

Automated Variational Ansatz Design

by

Matt Lourens

Dissertation presented for the degree of Doctor of Philosophy in Physics in the Faculty of Science at Stellenbosch University

Supervisors:

Prof. Francesco Petruccione
Prof. Ilya Sinayskiy

March 2026

https://scholar.sun.ac.za

Declaration

By submitting this dissertation electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Date: . . . . . March 2026 . . . . . .

Copyright © 2026 Stellenbosch University
All rights reserved.

https://scholar.sun.ac.za

Abstract

Automated Variational Ansatz Design

M. Lourens

Department of Physics, Stellenbosch University,
Private Bag X1, 7602 Matieland, South Africa.

Dissertation: PhD (Physics)

March 2026

We develop a systematic approach to ansatz design for quantum many-body problems. The approach is based on an abstract hypergraph language that compactly encodes modular, size-scalable and hierarchically composable wavefunction structures. With this language, evolutionary search can be carried out on few-body instances, yet the resulting wavefunctions extrapolate to arbitrary system size without modification. This enables efficient fitness evaluation and the capture of robust wavefunction structures. Applied to ground-state problems, the method autonomously rediscovers a mean-field description and extends it to incorporate correlations, producing simple, interpretable wavefunctions that remain analytically tractable. For both the Lipkin-Meshkov-Glick model and the transverse-field Ising model, the search converges on the same compact, two-parameter ansatz. Owing to its structure, we obtain closed-form expressions for energies and correlation functions. These results far surpass mean-field theory on finite systems and become exact or near-exact in the thermodynamic limit. The same framework also generates a quantum phase-recognition ansatz that outperforms a commonly used architecture while reducing the number of required variational parameters by two orders of magnitude. Finally, by training only on small-circuit examples, the method automatically produces general N-qubit realisations of standard quantum algorithms, namely the Deutsch-Jozsa algorithm, the quantum Fourier transform, and Grover's search. Taken together, our approach provides a concrete pathway towards a general, systematic theory of variational ansatz design.

https://scholar.sun.ac.za

Publications

This thesis is based on the following publications and manuscript.

1. **M. Lourens**, I. Sinayskiy, D. K. Park, C. Blank, and F. Petruccione, Hierarchical quantum circuit representations for neural architecture search, *npj Quantum Information* **9**, 79 (2023). DOI: 10.1038/s41534-023-00747-z.

2. **M. Lourens**, I. Sinayskiy, J. N. Kriel, and F. Petruccione, Generating Generalised Ground-State Ansatzes from Few-Body Examples, accepted for publication in *Physical Review Letters* (2025). DOI: 10.1103/5vc5-9f4d.

3. A. Rouillard, **M. Lourens**, and F. Petruccione, Automated Quantum Algorithm Synthesis, arXiv:2503.08449 (2025).

4. D. Slabbert, **M. Lourens**, and F. Petruccione, Pulsar classification: comparing quantum convolutional neural networks and quantum support vector machines, *Quantum Machine Intelligence* **6**, 56 (2024). DOI: 10.1007/s42484-024-00194-9.

https://scholar.sun.ac.za

Acknowledgements

I am forever grateful to the people and institutions that enabled and supported me throughout this journey.

I wish to thank Prof. Francesco Petruccione, whose unfailing optimism gave me the confidence to pursue what I otherwise would not have. I also thank Prof. Ilya Sinayskiy, who was always available to provide me with valuable insights, encouragement and support. I am grateful to Dr Johannes N. Kriel, whose keen insights and analytical capabilities taught me about the precision and detail that go into quality research. I also thank Prof. Hugo Touchette, who helped set this journey in motion.

I gratefully acknowledge the Oppenheimer Memorial Trust and the Unitary Fund for their financial support. I also acknowledge the computational resources provided by the Centre for High Performance Computing (CHPC), South Africa, and Hippo at the University of KwaZulu-Natal, South Africa.

I would also like to express my gratitude to Amy Rouillard, whose collaboration, support, and insights at every turn made this journey all the more worthwhile. Finally, I am sincerely grateful to my sister and parents, whose endless encouragement and support were invaluable to me.

https://scholar.sun.ac.za

Contents

Publications iii

Acknowledgements iv

List of Figures 1

List of Tables 3

Introduction 4

1 Aspects of Ansatz Design 6

2 Abstract Hypergraph Language 13
2.1 Hypergraphs . . . 13
2.2 Primitives . . . 16
2.3 Motifs . . . 18
2.4 Types of Primitives . . . 20
2.4.1 Initialisation . . . 20
2.4.2 Cycle . . . 21
2.4.3 Partition . . . 26
2.4.4 Mask . . . 28
2.4.5 Pivot . . . 32
2.4.6 Permute . . . 35
2.4.7 Default parameter values . . . 36
2.5 Hierarchical composition . . . 37
2.6 Domains of application . . . 39
2.6.1 Tensor networks . . . 40
2.6.2 Quantum circuits . . . 42

3 Ground-State Structure Search 46
3.1 Search strategy . . . 47
3.2 Genetic operations . . . 48

https://scholar.sun.ac.za

3.3 The discovered ansatz . . . 49
3.4 Robustness of the search . . . 52
3.5 Choice of operator basis . . . 53
3.6 Computational cost . . . 54

4 Analysis of a Universal Ansatz 57
4.1 Matrix product state representation . . . 57
4.2 Trace formulas for expectation values . . . 61
4.3 Transfer matrices - definitions and diagonalisation . . . 62
4.4 Expectation value calculations for the translationally invariant ansatz 64
4.5 Expectation value calculations for the translationally and parity symmetric ansatz . . . 65
4.6 Expectation value calculations for the original ansatz . . . 66
4.7 Thermodynamic limit of the average magnetisation per site . . . 67
4.8 Growth of bond dimension . . . 68

5 Predictions for Two Prototypical Spin Models 70
5.1 The Lipkin-Meshkov-Glick model . . . 72
5.1.1 The energy per spin in the thermodynamic limit . . . 74
5.1.2 Energy minimisation . . . 75
5.1.3 Symmetry projection . . . 76
5.2 The quantum transverse-field Ising model . . . 80
5.2.1 Energy minimisation . . . 82

6 Beyond Ground States 84
6.1 Quantum phase recognition . . . 84
6.2 Quantum algorithm synthesis . . . 86
6.2.1 Results . . . 87
6.3 Quantum machine learning . . . 91
6.3.1 Music genre classification . . . 92
6.3.2 Pulsar classification . . . 101

Conclusion 104

A Wildcard populate algorithm 119

https://scholar.sun.ac.za

List of Figures

2.1 Example of hypergraphs with same arity edges . . . 14
2.2 Example of hypergraphs with differing arities . . . 15
2.3 Example of size scaling . . . 16
2.4 Example of composition . . . 18
2.5 Example of multiplication . . . 19
2.6 Example of cycle steps . . . 22
2.7 Example of cycle strides . . . 23
2.8 Example of cycle arity . . . 24
2.9 Visualisation of arity $k = 1$ mask . . . 28
2.10 Visualisation of arity $k = 2$ mask . . . 29
2.11 Visualisation of the pivot primitive . . . 32
2.12 Comparison between the pivot and mask . . . 34
2.13 Example of hierarchical composition . . . 37
2.14 The tree of primitives . . . 39
2.15 Tensor network example . . . 41
2.16 Quantum Fourier transform example . . . 42
2.17 Representing quantum circuit ansatzes from literature . . . 44

3.1 Overview of using the AHL for ground-states . . . 47
3.2 The automatically generated ansatz for the LMG and TFIM models. 49
3.3 The robustness of the discovered ansatz under different search settings 52
3.4 Ansatz discovered for both models using the ladder-operator basis 53
3.5 Runtime of the evolutionary algorithm for different system sizes . . 54
3.6 Memory usage of the evolutionary algorithm for different system sizes 55

5.1 The discovered ansatz for the LMG and TFIM models . . . 71
5.2 RMS magnetisation for the LMG model . . . 74
5.3 Relative ground-state energy errors for the LMG model . . . 75
5.4 Long-range correlation function for the TFIM . . . 81
5.5 Relative energy errors for the TFIM . . . 82

https://scholar.sun.ac.za

6.1 Quantum phase recognition result 85
6.2 Discovered ansatz for quantum phase recognition 87
6.3 Motifs built by evolutionary algorithm 88
6.4 Evolution of fitness across generations (Grover rediscovery) 89
6.5 Example of an evolutionary path towards Grover's search algorithm 90
6.6 Runtime complexity for quantum algorithm synthesis 91
6.7 Machine learning pipeline for music genre classification 93
6.8 Possible mappings for the cycle in the music genre classification task 94
6.9 Reference architecture for music genre classification as hypergraph 95
6.10 Low-resolution image data classification results 97
6.11 Hyperparameters summary 100
6.12 QCNN for pulsar classification 101
6.13 The QSVM circuit 101
6.14 Pulsar classification results 103

https://scholar.sun.ac.za

List of Tables

2.1 Default cycle parameter values 36
2.2 Default mask parameter values 36
2.3 Default pivot parameter values 36
2.4 Default permute parameter values 37

3.1 Genetic operations for ground-state search 49

6.1 Performance of discovered ansatz for QPR 86
6.2 The parameter values of the cycle that determine the architecture in Ref. [153] 94
6.3 The parameter values of the mask that determine the architecture in Ref. [153] 95
6.4 Reference vs. random-search architecture results 96
6.5 The binary tree search space 99

https://scholar.sun.ac.za

Introduction

Every day, we are surrounded by a wealth of physical phenomena that emerge from the interactions of an incomprehensible number of degrees of freedom. In order to understand these physical systems, we build mathematical models to describe and predict their behaviour. While we understand the basic laws governing individual degrees of freedom, modelling their collective behaviour is incredibly challenging. More often than not we rely on a series of approximations to obtain qualitative and quantitative insights into these systems. This is especially true in quantum many-body theory, since we need an exponential number of coefficients to express the wavefunction. For instance, consider a one dimensional lattice with N sites, each with a d dimensional local state space $\{\sigma_i\}$. A generic state on the lattice can be written as

$$|\psi\rangle = \sum_{\sigma_0,\ldots,\sigma_{N-1}} \psi_{\sigma_0,\ldots,\sigma_{N-1}} |\sigma_0, \ldots, \sigma_{N-1}\rangle \,, \tag{1}$$

where each $\psi_{\sigma_0,\ldots,\sigma_{N-1}} \in \mathbb{C}$. This state contains a total of d^N complex amplitudes, which is exponential in the number of sites. Let us assume for a moment that $d = 2$. If you were to write down each amplitude on a small piece of paper and store it in a one cubic centimetre box, the observable universe would be filled at only $N = 287$ sites.

In order to understand quantum many-body systems, it is therefore necessary to find compressed representations of the relevant wavefunctions. These representations should capture the essential physical features of the systems of interest. Among the most successful and broadly applicable approaches to finding these representations are variational methods. In such methods one designs a suitable representation, called the ansatz, which incorporates variational parameters. By optimising these parameters with respect to an appropriate objective function, it is possible to obtain a solution that, ideally, provides qualitative insight and quantitative accuracy. A structurally simple ansatz with few parameters allows for analytic calculations and provides qualitative insights often at the expense of quantitative accuracy. Conversely, a structurally and variationally complex

https://scholar.sun.ac.za

ansatz requires a fully numeric approach but offers improved quantitative accuracy at the expense of qualitative insight. Balancing these qualitative and quantitative aspects is challenging. Consequently, constructing ansatzes that permit an analytic treatment while yielding accurate results over a wide range of system parameters is highly desirable.

Until recently, coming up with a good ansatz relied mostly on educated guesswork. In this thesis, I present a body of work [1–4] which aims to develop a systematic theory of ansatz design. Central to this work, and this thesis, is the abstract hypergraph language (AHL), implemented as an open source python package [1]. I developed the AHL to provide specific syntax and rules for compactly expressing ansatzes via modular building blocks. With the AHL we are able to automate certain aspects of ansatz design and strike a balance between the qualitative and quantitative extremes. This is demonstrated within the thesis. For quantum phase recognition, we automatically obtain an ansatz outperforming a popular design from literature [5], while reducing the required variational parameter count from 1308 to 11 [2]. For studying quantum many-body ground states, we show that a compact, accurate and interpretable ansatz can be discovered and written down in closed form by a computer-driven search [3]. For quantum algorithm design, we automatically generate the N-qubit realisations of popular quantum algorithms (Deutsch-Jozsa algorithm, quantum Fourier transform, Grover's search) by considering only examples up to five qubits [4].

https://scholar.sun.ac.za

Chapter 1

Aspects of Ansatz Design

When designing an ansatz, one ideally wants to strike a balance between qualitative and quantitative insights into physical systems. For qualitative insights, a simple ansatz is to construct a product state with minimal variational parameters, as is typical in mean-field theory (MFT) [6–10]. This approach, and its extensions [11–17], function by neglecting fluctuations and offer a low-cost procedure to obtain analytic insight into a system. For example, a well-known variational ansatz for a multi-electron wavefunction that satisfies the required antisymmetry principle is the Slater determinant [18, 19]. This ansatz is at the basis of the Hartree-Fock (HF) method [20–23], which approximates the ground-state electronic structure of systems like atoms and molecules. To achieve this, the method approximates the Coulomb electron-electron repulsion by a mean-field repulsion that each electron experiences. As a result of this approximation, the HF method is most effective for weakly correlated systems.

There are also the so-called post-Hartree-Fock methods that extend the HF method to better account for electron correlations [24]. This pattern of starting with the mean-field approximation and systematically extending it to incorporate correlations is a standard occurrence in ansatz design. We can think of such extensions abstractly as a mean-field ansatz combined with a correlator ansatz, and in this sense variational ansatzes are modular. Therefore, a crucial property of the AHL is its modularity: composing two ansatzes should yield another ansatz. In Chapter 3, I show how a computer-driven search was able to discover the mean-field solution and extend it to incorporate correlations, thereby stumbling upon this systematic pattern on its own. This shows the potential and utility of automating certain aspects of ansatz design.

From a technological point of view, solving the ground-state electronic structure problem is one of the most anticipated applications of quantum computing [25].

https://scholar.sun.ac.za

The timescale for the availability of fault-tolerant quantum computers is unclear, and current quantum devices suffer from high levels of noise and low coherence times. Therefore, algorithms have been developed that combine classical and quantum computation, called hybrid quantum classical (HQC) methods. Most popular among these is the variational quantum eigensolver (VQE) [26], which is used to find approximate ground states of quantum mechanical Hamiltonians, H. The VQE is an example of one of the variational methods alluded to above. The ansatz is a parameterised quantum circuit $U(\theta)$ executed on a quantum device that prepares the state

$$|\psi(\theta)\rangle = U(\theta)\,|\psi_0\rangle\,, \tag{1.1}$$

where $|\psi_0\rangle$ is typically a product state. The energy expectation value is then minimised by a classical optimiser to obtain

$$\theta^* = \text{argmin}_\theta\,\langle\psi(\theta)|H|\psi(\theta)\rangle\,, \tag{1.2}$$

which gives the approximate ground state $|\psi(\theta^*)\rangle$. For electronic structure problems, one needs to map fermionic operators to spin operators, usually via the Jordan-Wigner or Bravyi-Kitaev transformations. Many of these problems start with the unitary coupled cluster (UCC) ansatz that is then modified in a myriad of ways [27–39] for practical and performance considerations [40]. In particular, the adaptive ansatz approaches [35–39] illustrate that the ansatz structure can be updated iteratively, resulting in improved performance while lowering circuit depth and variational parameter count. The success of these methods further justifies automating certain aspects of ansatz design.

Beyond electronic structure problems, the VQE is also applicable to lattice models [41–50]. Here, the design of the quantum circuit ansatz remains crucial. In particular, Refs. [47–49] show that combining the adaptive ansatz approach with scalable circuits enables the consideration of smaller system sizes. This is computationally efficient and allows for the extrapolation of variational parameters to larger sizes. Using few-site information to build scalable wavefunctions has also proved effective in many-body methods such as density matrix embedding theory [51] and variational Monte Carlo [52]. The AHL shares this size scaling property. When initially introduced in Ref. [2], our circuits were also scalable, and we applied an evolutionary search algorithm to find a quantum phase recognition ansatz. The results are discussed in Chapter 6. Notably, we were able to improve upon the performance of a popular ansatz [5] while also reducing the variational parameter count from 1308 to 11 parameters.

Since optimising over ansatz structures is inherently computationally expensive, I believe the size scaling property of the AHL is crucial. Only small system sizes

https://scholar.sun.ac.za

need to be considered during optimisation because size scaling enables the extrapolation of both ansatz structure and ansatz parameters. This drastically reduces the required computational resources. The size scaling property enabled us to find an efficient ansatz for the ground state of two quantum spin models that extrapolates to the thermodynamic limit by considering only system sizes up to six spins [3]. These results are presented in Chapters 3-5. Similarly, we were able to find the general N-qubit realisations of three well-known quantum algorithms by only considering circuits of up to five qubits [4]. These algorithms are the quantum Fourier transform (QFT), the Deutsch–Jozsa algorithm, and Grover's search algorithm, and the results are discussed in Chapter 6.

The adaptive ansatz methods I have referenced up to this point work by updating the ansatz structure iteratively based on well-motivated heuristics. The method developed in our body of work [1–4] modifies the ansatz structure using a machine learning model. This is inspired by the success of neural architecture search (NAS) to automatically design neural network architectures. There is a large class of related methods in the quantum realm [53–63], typically called quantum architecture search (QAS). A common result in NAS and QAS is showcasing the ability to improve upon the performance of a manually designed ansatz, while at the same time reducing the structural or variational complexity. As mentioned before, the work in this thesis is based on an AHL to construct ansatzes with modular building blocks. The use of a symbolic language to construct ansatzes is common in program synthesis [64–66], where such a language is called a domain-specific language (DSL). The benefit of a compact representation (which the DSL enables) greatly reduces the size of the search space of possible programs. The AHL shares this compactness property, and we were the first to use NAS techniques based on a DSL in a quantum circuit [2, 4] and tensor network [3] design setting. The use of a DSL overcomes the two biggest issues currently facing QAS methods. It vastly reduces the search space size through its compactness and greatly reduces the computational cost through its size scaling.

The AHL was initially designed to build abstract quantum circuit structures [2] and is implemented as an open-source python package [1]. The Tequila framework [67] already demonstrated the power of a domain-specific language and abstract data structures in the context of variational quantum algorithm development. In our python package, we represent the circuit as a hypergraph where nodes represent qubits and edges represent gates. The hypergraph representation is quite general and in the latest version nodes represent indices of a multi-dimensional array and edges represent a function that is applied on a subset of elements in that array, thereby updating the array. Any pythonic objects can be stored in the array and any pythonic function can be mapped to the edges. There-

https://scholar.sun.ac.za

fore a very wide range of applications are possible. In particular, neural networks, tensor networks and quantum circuits can all be expressed as hypergraphs, and are in fact all used as wavefunction ansatzes for quantum many body states. In Ref. [3], we applied the AHL within the domain of tensor network states (TNS), motivated by their success in describing strongly correlated quantum many-body states.

Tensor network states are a powerful class of variational ansatzes that provide quantitative accuracy for strongly interacting systems [68]. A special case of TNS, matrix product states (MPSs) [69–71], naturally represent low-energy states of systems in 1D with local interactions [72, 73]. As an example, if we assume periodic boundary conditions for the one dimensional lattice from Eq. (1), then an MPS expresses the state in the form

$$|\psi\rangle = \sum_{\sigma_0,\dots,\sigma_{N-1}} \mathrm{Tr}(A_0^{\sigma_0} A_1^{\sigma_1} \cdots A_{N-1}^{\sigma_{N-1}}) \, |\sigma_0, \dots, \sigma_{N-1}\rangle \,, \tag{1.3}$$

where $A_i^{\sigma_i}$ are $d_i \times d_{i+1}$ matrices and indices are modulo N due to the periodic boundary condition. This representation is exact for sufficiently large d_i, $\forall i \in \{0, \dots, N-1\}$, called the bond dimensions. In Eq. (1.3) we see that an MPS enforces some structure on the coefficients from Eq. (1). In particular, this structure causes the correlation between two sites to decay exponentially with their seperation distance, which is typical for gapped non-critcal 1D ground states [74]. An MPS is most useful when it can accurately represent a state with bond dimensions that are numerically manageable. A powerful algorithm for finding such an MPS representation is the density matrix renormalization group (DMRG) [71, 75–78]. This method is very successful for one dimensional systems, and moderately successful for two dimensional systems [71]. The difference in success has to do with how the quantum entanglement in many-body states scales as dictated by the area laws [79].

The structure imposed by Eq. (1.3) has several convenient analytical properties. In Chapter 4, we analyse a TNS ansatz, discovered by our computer driven search, which can be transformed into an MPS form. This ansatz has a maximal bond dimension of two, and allows for analytic calculations. We obtain exact expressions for correlation functions and are able to further improve the ansatz by restoring the broken symmetries. In particular, this symmetry restoration is central to our method outlined in Ref. [3] because our search procedure typically yields compact ansatzes which break the model's underlying symmetries. By restoring these broken symmetries we can systematically improve the ansatz and move beyond the initial MPS form obtained from the search, as shown in Chapter 5.

https://scholar.sun.ac.za

The most natural generalisation of MPS to 2D is the so called projected entangled pair states (PEPS) [80–83]. The problem with PEPS is that the exact contraction of the network is inefficient [74]. Other generalisations involve different network geometries, for instance the multiscale-entanglement-renormalisation ansatz (MERA) [84–87], which is a hierarchical $D+1$ dimensional network. The MERA has several interresting properties. Its hierarchical structure enables the algebraic decay of correlations in any dimension D, as opposed to the exponential decay of correlations for the MPS [84], and contributes to the efficient contraction of the network. For these reasons, MERA is an efficient ansatz for simulating critical systems and performing entanglement renormalisation [88]. Similar to MERA, another generalisation of network structure is tree tensor networks (TTNs) [89–96]. As with MERA, TTNs better account for long range correlations and the hierarchical structure contributes to efficient network contraction. TTNs are also appliable to electronic structure problems [90, 97]. In Refs. [90, 97] each physical index of the TTN functions as a parametrised single particle basis state, and the appropriate network structure captures the entanglement topology. These generalisations show that hierarchical structures facilitate efficient calculations and better account for long range correlations. For this reason, the AHL is designed to express such networks compactly, thereby increasing the likelihood of a search algorithm building hierarchical networks.

Another class of generalisations are based on defining a correlator ansatz for which a variational Monte Carlo (VMC) algorithm [98] can be used to estimate physical observables. The correlator ansatzes can be MPSs, PEPS [99], entangled-plaquette states (EPS) [100], correlator product states (CPS) [101], string bond states (SBS) [102, 103] and complete graph tensor networks (CGTN) [104]. SBS are based on overlaying 1D MPSs on a higher dimensional lattice according to some string pattern. EPS groups local sites together into a block (for example a small PEPS in a large PEPS network), and extracts the ground state wavefunction for each block. The full wavefunction is then expressed as a product of theses block wavefunctions, which can be overlapping to form entangled plaquettes. CPS directly writes the wavefunction as a product of coefficients that depend on different subsets of sites, allowing for arbitrary correlations. In all of these methods, there is a choice of correlation pattern that needs to be made and these patterns can be represented diagrammaticically with hypergraphs. Therefore, the AHL is designed to capture common correlation patterns that appear across literature, enabling a search algorithm to automatically find patterns that work the best.

The use of VMC sampling to optimise the variational parameters of an ansatz is also found in the so-called neural quantum states (NQS) [105]. For NQS, neu-

https://scholar.sun.ac.za

ral networks are used as the wavefunction ansatzes [105–111]. The restricted Boltzmann machine is a widely used case where input nodes correspond to sites and are connected to hidden nodes to capture correlations. The resulting ansatz can again be expressed as a hypergraph, and in fact has strong connections to CPS [112], SBS and EPS [113].

The choice of a variational ansatz for a wavefunction is analogous to the choice of a specific neural network architecture in machine learning. In fact, as we just discussed, sometimes neural networks are directly used as ansatz wavefunctions. The field of machine learning went through its own arc of manual to automated ansatz design. Initially, features were designed by hand. In image classification, for example, these features typically captured aspects such as texture, shape, and edges. One of the main characteristics of modern machine learning is its ability to learn features from raw data, eliminating the need for manual feature design by experts [114]. AlexNet [115] demonstrated this and marked the shift in focus from manual feature design to manual architecture design [116]. Naturally, the next step is automated architecture design, which neural architecture search (NAS) aims to achieve [117], and has already produced state-of-the-art deep learning models [116, 118–120].

NAS consists of three main components: search space, search strategy, and performance estimation strategy [117]. The search space defines the set of possible architectures that a search algorithm can consider. Therefore, it is important to carefully design the search space to improve search efficiency and reduce computational complexity [121]. Search space design often involves encoding architectures using a cell-based representation. Usually, a set of primitive operations is combined into a cell to capture some design motif (compute graph). For example, in the context of convolutional neural networks these primitives can be convolution or pooling operations. Different cells are then stacked to form a complete architecture. Cell-based representations are popular because they can capture design motifs often seen in successfully hand-crafted architectures, such as repetitive and modular patterns. One problem with the cell-based representation is that the macro architecture, the sequence of cells, is fixed and must be chosen manually [117]. Recently, Ref. [121] proposed a hierarchical representation as a solution, where a cell sequence acts as the third level of a multi-level hierarchy. In this representation, lower-level motifs act as building blocks for higher-level motifs, allowing both macro and micro architecture to be learned. One of the core ideas in the hierarchical representation of Ref. [121] is the ability to express a motif in terms of submotifs. The AHL has two different ways of achieving this: one is standard composition and the other hierarchical composition. The details of composition are given in Chapter 2, but the main takeaway is that composition

https://scholar.sun.ac.za

allows for a very compact and modular representation of elaborate structures. Composition is also useful in the context of evolutionary algorithms, since the algorithm can find well-performing structures and use these as building blocks for more intricate structures.

The key aspects of variational ansatz design highlighted in this chapter are: modularity, size scaling, compactness, the capture of simple and hierarchical (correlation) patterns and hierarchical composition. The AHL shares all of these properties, and the mechanisms by which it does so are detailed in Chapter 2.

https://scholar.sun.ac.za

Chapter 2

Abstract Hypergraph Language

Here I present an *abstract hypergraph language* (AHL) that I developed to build ansatzes. An ansatz refers to a proposed form of a solution to a domain-specific problem. In later chapters, I show the applications of this AHL in the context of quantum many-body systems, quantum algorithms and quantum machine learning. In these applications, the AHL provides specific syntax and rules for compactly expressing ansatzes via modular building blocks. This AHL is then used to create a search space of ansatzes for an algorithm to explore. Within the context of quantum many-body systems, the ansatzes are tensor networks representing quantum many-body states. In quantum algorithms, the ansatzes are abstract quantum circuits representing quantum tasks. In quantum machine learning, the ansatzes are abstract quantum circuits representing classification models. The AHL is implemented as an open-source Python package [1], which we first presented in Ref. [2]. Initially, Ref. [2] introduced the concept of abstract quantum circuit structures that can be composed hierarchically to form new structures. Later, in Ref. [3], we extended that abstraction to include arbitrary tensor networks. Here, I present a more general form of the AHL, where it is used to build abstract hypergraphs. An abstract hypergraph refers to a class of hypergraphs that shares the same underlying structural pattern. In this general form, the AHL also finds applications in classical algorithms and sorting networks [1].

2.1 Hypergraphs

In this work, I will consider hypergraphs where there is an ordering for the nodes, the edges and the nodes in each edge. I define a hypergraph as the pair $\mathcal{G} \equiv (\mathbf{q}, E)$, where $\mathbf{q} \equiv (q_0, \dots, q_{N-1})^T$ is a sequence of N nodes represented as a column vector. The sequence E of R edges $\mathbf{e}_r$ is defined as $E \equiv (\mathbf{e}_0, \dots, \mathbf{e}_{R-1})$ and

https://scholar.sun.ac.za

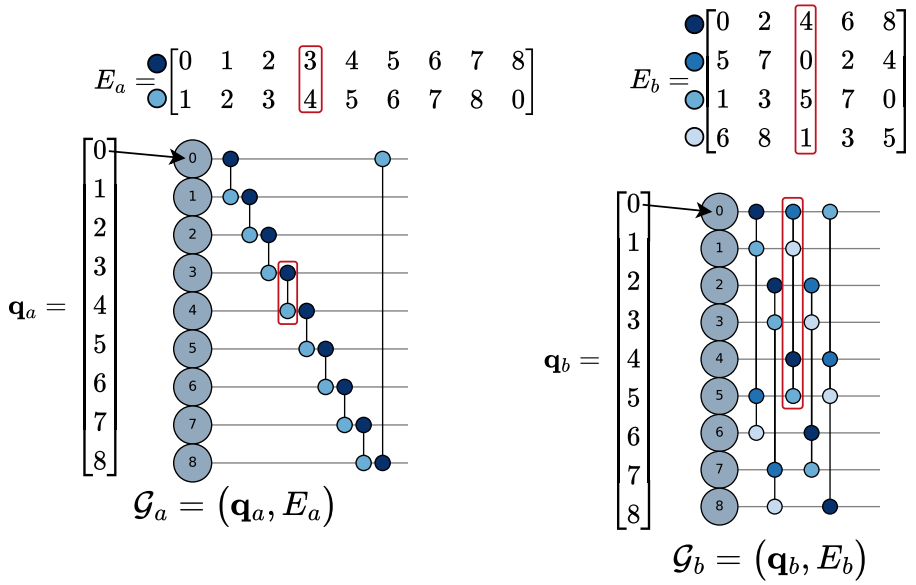


Figure 2.1: Two examples of hypergraphs, the edges in $\mathcal{G}_a$ all have an arity $k = 2$ and the edges in $\mathcal{G}_b$ all have an arity of $k = 4$. Nodes are ordered from top to bottom and represented as circles on the left of each graph. Edges are ordered from left to right. Each edge consists of a number of nodes indicated by circles, the ordering of the nodes in an edge is indicated with the darkness of the circle.

represented as a row vector. Each edge $\mathbf{e}_r$ is a sequence of k elements of $\mathbf{q}$, defined as $\mathbf{e}_r \equiv (q_{j_0}, \dots, q_{j_{k-1}})^T$, represented as a column vector. Here k is called the *arity* of an edge $\mathbf{e}_r$. If every edge $\mathbf{e}_r$ in the sequence of edges E has the same arity k then I will treat E as a $k \times R$ matrix, called the *connectivity matrix*. In Fig. 2.1, I show two examples of hypergraphs. These have the sequence of nodes $\mathbf{q}_a$ and $\mathbf{q}_b$ and connectivity matrices E_a and E_b respectively. I will represent these hypergraphs as

$$\underset{\mathcal{G}_a = (\mathbf{q}_a, E_a)}{\begin{bmatrix}0\\1\\2\\3\\4\\5\\6\\7\\8\end{bmatrix}_{\mathbf{q}_a}, \begin{bmatrix}0&1&2&3&4&5&6&7&8\\1&2&3&4&5&6&7&8&0\end{bmatrix}_{E_a}} \quad \text{and} \quad \underset{\mathcal{G}_b = (\mathbf{q}_b, E_b)}{\begin{bmatrix}0\\1\\2\\3\\4\\5\\6\\7\\8\end{bmatrix}_{\mathbf{q}_b}, \begin{bmatrix}0&2&4&6&8\\5&7&0&2&4\\1&3&5&7&0\\6&8&1&3&5\end{bmatrix}_{E_b}}. \tag{2.1}$$

https://scholar.sun.ac.za

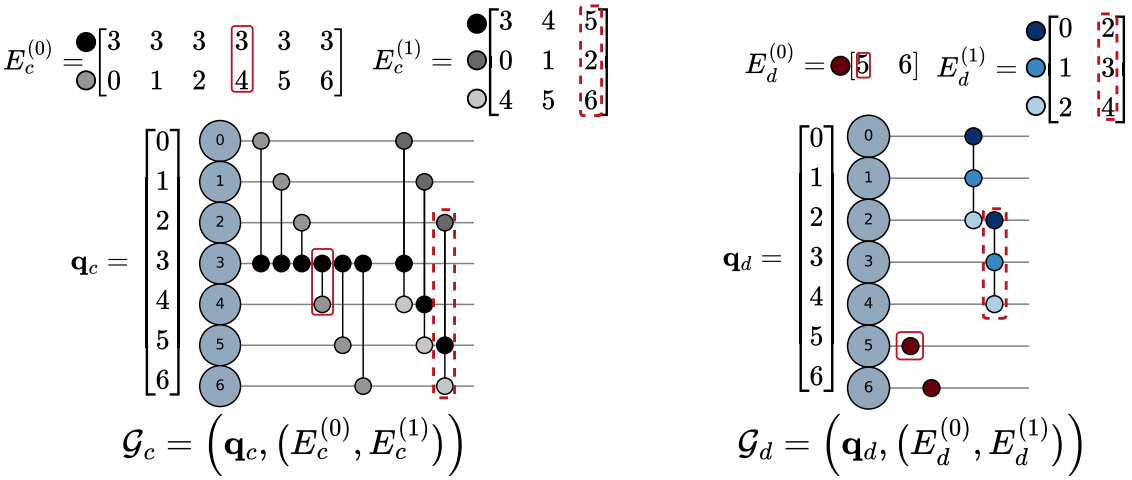


Figure 2.2: Two examples of hypergraphs $\mathcal{G}_c$ and $\mathcal{G}_d$ with differing edge arities in each hypergraph. The edges in $\mathcal{G}_c$ have arity either $k = 2$ or $k = 3$ and the edges in $\mathcal{G}_d$ have arity either $k = 1$ or $k = 3$. The edges of each hypergraph are specified as a sequence of connectivity matrices.

Both $\mathcal{G}_a$ and $\mathcal{G}_b$ have $N_a = N_b = 9$ nodes. The depiction in Fig. 2.1 captures the ordering within an edge and between the edges. The ordering between the edges is captured horizontally, ordered from left to right. This matches the ordering of the columns in the connectivity matrix, for example, in $\mathcal{G}_b$ the first edge is given by $\mathbf{e}_0 = (0, 5, 1, 6)^T$ and the second edge by $\mathbf{e}_1 = (2, 7, 3, 8)^T$, and so on. The ordering within an edge is depicted in Fig. 2.1 by the colour gradient of the dots, from dark to light. For example, the third edge of $\mathcal{G}_b$ is $\mathbf{e}_2 = (4, 0, 5, 1)^T$ therefore the corresponding darkest dot is on node 4, the second darkest on node 0, the third on node 5, and the lightest on node 1.

In Fig. 2.2 I show two hypergraphs $\mathcal{G}_c$ and $\mathcal{G}_d$ with differing edge arities in each hypergraph, i.e. they do not have the same number of elements in each of their edges. But notice that there are groups of edges within each hypergraph that do, and so I will keep track of their edges as a sequence of connectivity matrices. The

https://scholar.sun.ac.za

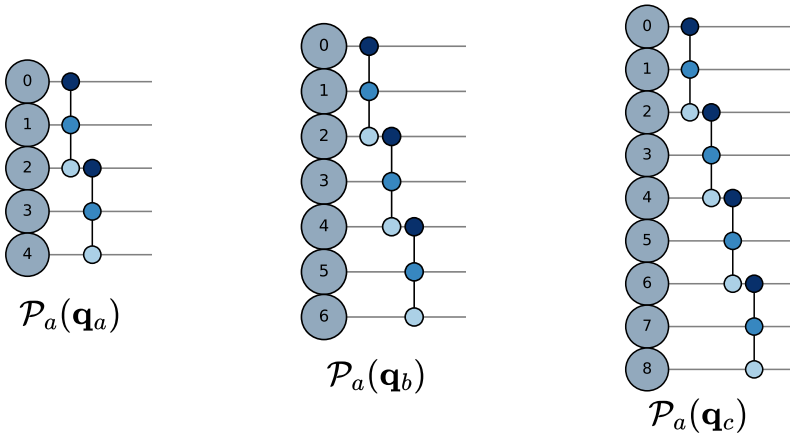


Figure 2.3: The same primitive $\mathcal{P}_a$ instantiated on node sequences of different lengths.

hypergraphs $\mathcal{G}_c$ and $\mathcal{G}_d$ now reads

$$\mathcal{G}_c = \left(\mathbf{q}_c, \left(E_c^{(0)}, E_c^{(1)}\right)\right) \qquad \mathcal{G}_d = \left(\mathbf{q}_d, \left(E_d^{(0)}, E_d^{(1)}\right)\right)$$

$$\overset{\mathbf{q}_c}{\begin{bmatrix}0\\1\\2\\3\\4\\5\\6\end{bmatrix}}, \left(\overset{E_c^{(0)}}{\begin{bmatrix}3&3&3&3&3&3\\0&1&2&4&5&6\end{bmatrix}}, \overset{E_c^{(1)}}{\begin{bmatrix}3&4&5\\0&1&2\\4&5&6\end{bmatrix}}\right) \text{ and } \overset{\mathbf{q}_d}{\begin{bmatrix}0\\1\\2\\3\\4\\5\\6\end{bmatrix}}, \left(\overset{E_d^{(0)}}{\begin{bmatrix}5&6\end{bmatrix}}, \overset{E_d^{(1)}}{\begin{bmatrix}0&2\\1&3\\2&4\end{bmatrix}}\right). \tag{2.2}$$

In these examples the subscript α identifies the graph $\mathcal{G}_\alpha$ with its associated nodes $\mathbf{q}_\alpha$ and edges $E_\alpha^{(i)}$. The superscript i in $E_\alpha^{(i)}$ indicates the order of the different connectivity matrices in the sequence. The reason for grouping edges with the same number of elements into connectivity matrices $E_\alpha^{(i)}$ is that the building blocks within the AHL generate hypergraphs with edges that all have the same arity. The composition of these building blocks then lead to hypergraphs with edges that have different arities. The hypergraphs $\mathcal{G}_c$ and $\mathcal{G}_d$ in Fig. 2.2 are examples of this case, they consist of two sub-hypergraphs that share the same nodes.

2.2 Primitives

Primitives are the basic building blocks within the AHL. A primitive is an abstract hypergraph with properties that specify its instantiation on different numbers of

https://scholar.sun.ac.za

nodes. For example, in Fig. 2.3 I show the same primitive $\mathcal{P}_a$ instantiated on the sequences of nodes

$$\mathbf{q}_a = (0, 1, 2, 3, 4)^T, \tag{2.3}$$
$$\mathbf{q}_b = (0, 1, 2, 3, 4, 5, 6)^T, \tag{2.4}$$
$$\mathbf{q}_c = (0, 1, 2, 3, 4, 5, 6, 7, 8)^T. \tag{2.5}$$

All three hypergraphs in Fig. 2.3 share the same structure. Each edge covers three consecutive nodes and the previous edge ends where the next starts. This structure is encoded in the primitive $\mathcal{P}_a$ as a rule that generates the connectivity matrices

$$E_a = \begin{bmatrix} 0 & 2 \\ 1 & 3 \\ 2 & 4 \end{bmatrix}, \tag{2.6}$$

$$E_b = \begin{bmatrix} 0 & 2 & 4 \\ 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}, \tag{2.7}$$

$$E_c = \begin{bmatrix} 0 & 2 & 4 & 6 \\ 1 & 3 & 5 & 7 \\ 2 & 4 & 6 & 8 \end{bmatrix}. \tag{2.8}$$

This rule is given by

$$n(j, r) = j + 2r \tag{2.9}$$
$$(E_\alpha)_{j,r} = \mathbf{q}_\alpha[n(j, r)], \tag{2.10}$$

where $\alpha \in \{a, b, c\}$ and $\mathbf{q}_\alpha[n] \equiv (q_\alpha)_n$ refers to the element at index n of $\mathbf{q}_\alpha$. Here $j \in \{0, \dots, k-1\}$, where k is the arity, and $r \in \{0, \dots, R_\alpha - 1\}$ where the number of edges R_α is given by

$$R_\alpha = \left\lfloor \frac{N_\alpha - k}{2} \right\rfloor + 1, \tag{2.11}$$

and N_α denotes the number of nodes in $\mathbf{q}_\alpha$. Each connectivity matrix E_α has the shape $k \times R_\alpha$. Every edge covers three nodes because the arity is $k = 3$. These nodes are consecutive because any two successive elements in an edge $\mathbf{e}_r$ are one node apart, i.e. $n(j+1, r) - n(j, r) = 1$ from Eq. (2.9). The next edge starts where the previous ends. From Eq. (2.9) we see that edge $\mathbf{e}_r$ ends at $n(k-1, r)$ and the next edge starts at $n(0, r+1)$, so that the difference $n(k-1, r) - n(0, r+1) = k - 3 = 0$.

https://scholar.sun.ac.za

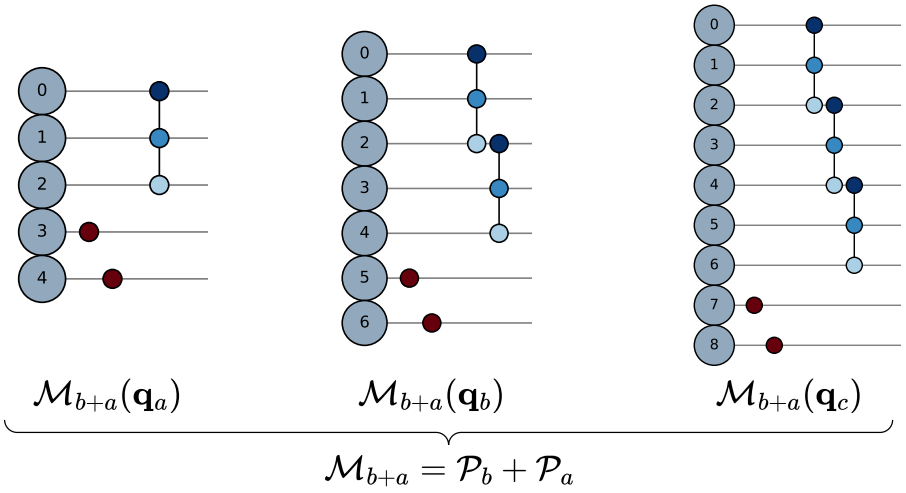


Figure 2.4: Three examples of a motif instantiated on different numbers of nodes.

An important property of a primitive is to capture some underlying structural pattern in a hypergraph such that the pattern can be extrapolated to any sequence of nodes. I refer to this as the size-scaling property. All primitives generate a $k \times R(N)$ connectivity matrix E. The number of edges $R(N)$ depends on the number of nodes N. However, the arity k of all edges in the connectivity matrix E is fixed, and therefore I will also refer to k as the arity of the primitive.

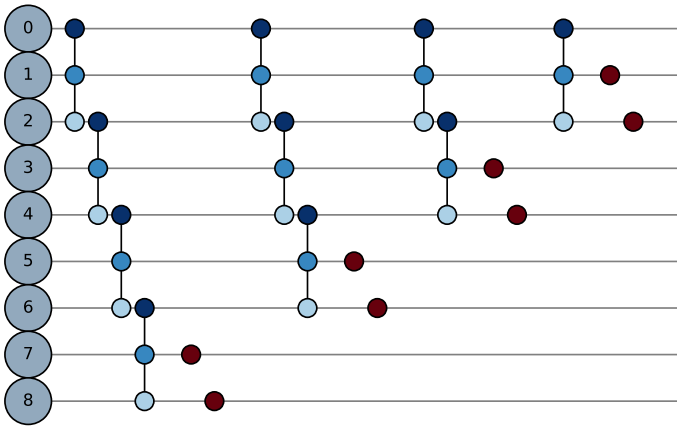
2.3 Motifs

Primitives can be composed to form sequences of primitives called motifs. In turn, multiple motifs can be composed to form sequences of motifs. A motif is therefore a sequence of abstract hypergraphs. In this sequence, neighbouring primitives are linked so that a motif itself forms a single abstract hypergraph. The key point is that a primitive in a motif determines which of its nodes will be available to the subsequent primitive. Let $\mathcal{P}_a$ and $\mathcal{P}_b$ be two primitives, then the motif that consists of $\mathcal{P}_b$ followed by $\mathcal{P}_a$ is written as

$$\mathcal{M}_{b+a} \equiv \mathcal{P}_b + \mathcal{P}_a \equiv (\mathcal{P}_b, \mathcal{P}_a). \tag{2.12}$$

This motif consists of two primitives, and I will use an index $i \in \{0, 1\}$ to indicate their order. For instance, $\mathcal{M}_{b+a}[0] \equiv \mathcal{P}_b$ and $\mathcal{M}_{b+a}[1] \equiv \mathcal{P}_a$. The same index i is used to refer to the hypergraphs that would result from each primitive, $\mathcal{G}_0 = (\mathbf{q}_0, E_0)$ and $\mathcal{G}_1 = (\mathbf{q}_1, E_1)$. If we require the nodes in $\mathbf{q}_1$ from the second hypergraph to be a subset of the nodes in $\mathbf{q}_0$ from the first hypergraph, then we can combine the two into one hypergraph $\mathcal{G}_{b+a} = (\mathbf{q}_0, (E^{(0)}_{b+a}, E^{(1)}_{b+a}))$. This is how

https://scholar.sun.ac.za



$$\mathcal{M}_d(\mathbf{q}_c)$$

$$\mathcal{M}_d = (\mathcal{P}_a + \mathcal{P}_b) \times 4$$

Figure 2.5: Example of motif multiplication. The sequence of primitives $\mathcal{P}_a$ and $\mathcal{P}_b$ is repeated 4 times.

the two examples from Fig. 2.2 were formed. Each results from a motif that consists of two primitives.

Consider the hypergraphs in Fig. 2.4. All three hypergraphs come from the single motif in Eq. (2.12) instantiated on the sequences of nodes $\mathbf{q}_a = (0, 1, 2, 3, 4)^T$, $\mathbf{q}_b = (0, 1, 2, 3, 4, 5, 6)^T$ and $\mathbf{q}_c = (0, 1, 2, 3, 4, 5, 6, 7, 8)^T$. Fig. 2.4 shows that a motif, like a primitive, can be instantiated on any number of nodes while maintaining the underlying pattern. In other words, motifs retain the size-scaling property. Here $\mathcal{P}_a$ is the primitive from Fig. 2.3 and $\mathcal{P}_b$ is a new primitive that makes all its nodes available except the last two. Consider the first hypergraph on the left of Fig. 2.4. The first primitive $\mathcal{M}_{b+a}[0]$ in the motif $\mathcal{M}_{b+a}$ instantiates a hypergraph $\mathcal{G}_0$ with $\mathbf{q}_0 \equiv (0, 1, 2, 3, 4)^T$, where $\mathbf{q}_0$ is defined as the sequence of nodes on which the motif was instantiated, in this case $\mathbf{q}_a$. The primitive $\mathcal{M}_{b+a}[0]$ makes a subsequence of its nodes available to the next primitive $\mathcal{M}_{b+a}[1]$. This subsequence is denoted as $\mathbf{q}'_0$, in this case it is given by $\mathbf{q}'_0 = (0, 1, 2)^T$ since the first primitive $\mathcal{M}_{b+a}[0]$ hides the last two nodes of $\mathbf{q}_0$. The sequence of nodes $\mathbf{q}'_0$ defines the sequence of nodes $\mathbf{q}_1$ with which the next primitive $\mathcal{M}_{b+a}[1]$ will instantiate the hypergraph $\mathcal{G}_1 = (\mathbf{q}_1, E_1)$. Since $\mathcal{G}_0$ and $\mathcal{G}_1$ share the same nodes, we can combine them into one hypergraph $\mathcal{G}_{b+a} = (\mathbf{q}_0, (E^{(0)}_{b+a}, E^{(1)}_{b+a}))$ where $E^{(0)}_{b+a}$ denotes the connectivity matrix of $\mathcal{G}_0$ and $E^{(1)}_{b+a}$ the connectivity matrix of $\mathcal{G}_1$.

Let $\mathcal{M}$ denote a motif that contains N_p primitives indexed by $i \in \{0, \dots, N_p - 1\}$.

https://scholar.sun.ac.za

For any two subsequent primitives $\mathcal{M}[i]$ and $\mathcal{M}[i+1]$ the nodes with which $\mathcal{M}[i+1]$ instantiates the hypergraph $\mathcal{G}_{i+1}$ is linked to the nodes of $\mathcal{M}[i]$ by

$$\mathbf{q}_{i+1} \equiv \mathbf{q}'_i. \tag{2.13}$$

Here, $\mathbf{q}'_i$ is a subsequence of $\mathbf{q}_i$ that the primitive $\mathcal{M}[i]$ makes available to the primitive $\mathcal{M}[i+1]$. The single hypergraph that results from a motif $\mathcal{M}$ instantiated with the initial sequence of nodes $\mathbf{q}_0$ is then given by

$$\mathcal{G} \equiv \left(\mathbf{q}_0, (E^{(0)}, \dots, E^{(N_p-1)})\right). \tag{2.14}$$

This composition rule allows us to compactly express several underlying structures in a hypergraph. We can further compactify our notation by introducing a repetition operation, denoted as multiplication by

$$\mathcal{P} \times 2 \equiv \mathcal{P} + \mathcal{P}, \tag{2.15}$$

for primitives and

$$\mathcal{M} \times 2 \equiv \mathcal{M} + \mathcal{M}, \tag{2.16}$$

for motifs. In Fig. 2.5, I show an example of repetition through multiplication. Here, the same primitives $\mathcal{P}_a$ and $\mathcal{P}_b$ from Eq. (2.12) are combined but in a different order and this combination is repeated four times. The result is a sequence of eight primitives, where each $\mathcal{P}_b$ hides the last two nodes for the subsequent primitives. The power of motifs is that they allow us to compactly express underlying structures in hypergraphs without referring to their nodes.

2.4 Types of Primitives

2.4.1 Initialisation

The purpose of initialisation is to specify the initial sequence of nodes $\mathbf{q}_0$ with which to instantiate a hypergraph from a motif. Up until now, I have denoted the instantiation of a hypergraph loosely as either $\mathcal{P}(\mathbf{q})$ or $\mathcal{M}(\mathbf{q})$ depending on whether it came from a primitive or motif. It is convenient to reserve a special primitive $\mathcal{P}_0$, called *init*, for instantiation. Any hypergraph $\mathcal{G}$ can be expressed in the AHL as some motif $\mathcal{M}$ combined with the init primitive $\mathcal{P}_0$

$$\mathcal{G} \equiv \mathcal{P}_0(\mathbf{q}_0) + \mathcal{M}. \tag{2.17}$$

The init primitive $\mathcal{P}_0$ takes the sequence of nodes $\mathbf{q}_0$ and trivially makes the same sequence available for the first primitive in the motif that follows, i.e. $\mathbf{q}'_0 \equiv \mathbf{q}_0$.

https://scholar.sun.ac.za

The init primitive $\mathcal{P}_0$ must always occur first and only once in the sequence when initialising a hypergraph. Therefore, the statement $\mathcal{M} + \mathcal{P}_0$ is undefined. In most examples, the following shorthand is used

$$\mathcal{G} = \mathcal{P}_0(N) + \mathcal{M}, \tag{2.18}$$

where N denotes the number of nodes and initialises $\mathbf{q}_0$ as

$$\mathbf{q}_0 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ N-1 \end{bmatrix}. \tag{2.19}$$

2.4.2 Cycle

The cycle primitive creates a hypergraph by traversing its sequence of nodes and generating an edge at each node that is visited. Consider a hypergraph $\mathcal{G}$ created by some motif $\mathcal{M}$ as per Eq. (2.17) with N initial nodes contained in $\mathbf{q}_0$. Let $\mathbf{q}_i$ be the sequence of N_i available nodes for an arity k cycle $\mathcal{M}[i]$ in the motif $\mathcal{M}$. This cycle will generate the connectivity matrix $E^{(i)}$ as follows

$$n_i(j,r) \equiv (h_0 + h_1 j + h_2 r) \bmod N_i, \tag{2.20}$$

$$E^{(i)}_{j,r} \equiv \mathbf{q}_i[n_i(j,r)], \tag{2.21}$$

where h_0, h_1 and h_2 are called the *offset*, *stride* and *step*, respectively. Here, $h_0 \in [0, N]$, $h_1 \in [1, N]$, $h_2 \in [1, N]$, $j \in \{0, \dots, k-1\}$ and $r \in \{0, \dots, R-1\}$ with the number of edges R determined by

$$R \equiv \left\lfloor \frac{N_i - 1 - h_0 - (k-1)h_1\delta_b}{h_2} \right\rfloor + 1. \tag{2.22}$$

and $\delta_b \in \{0, 1\}$ is a boundary condition parameter. The cycle leaves all of its nodes available for subsequent primitives, therefore

$$\mathbf{q}'_i = \mathbf{q}_i. \tag{2.23}$$

Eq. (2.21) specifies the construction of the connectivity matrix $E^{(i)}$, with elements $E^{(i)}_{j,r}$ indexed by $n_i(j,r)$ defined in Eq. (2.20). This index function $n_i(j,r) \in \{0, \dots, N_i - 1\}$ is the central object that encodes the structure of hypergraphs formed by cycles. There is a one-to-one correspondence between the nodes in $\mathbf{q}_i$ and their indices $n_i(j,r)$, so that $n_i(j,r)$ is a unique label for each node $\mathbf{q}_i[n_i(j,r)]$. Based on this equivalence and for convenience, I will use the node index $n_i(j,r)$

https://scholar.sun.ac.za

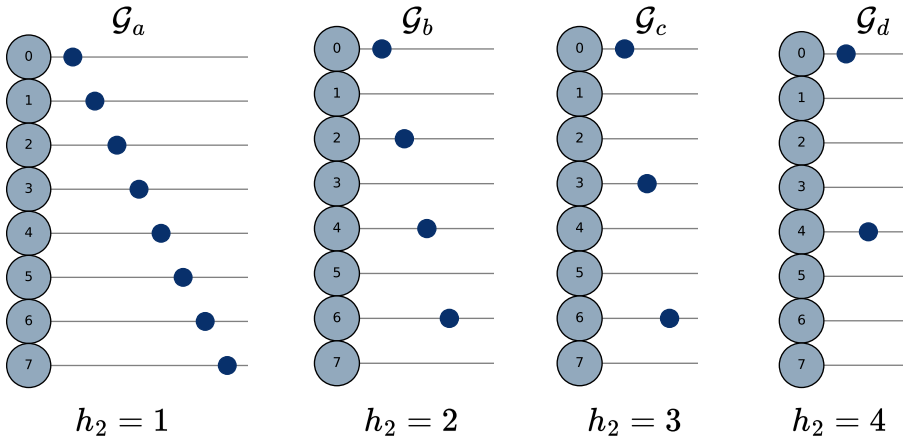


Figure 2.6: Four cycles of arity $k = 1$ and $h_0 = 0$ instantiated on eight nodes with differing step parameter values. When $k = 1$, both Eqs. (2.20) and (2.22) become independent of δ_b and h_1, therefore their values do not influence the hypergraphs in this example.

instead of the node value $\mathbf{q}_i[n_i(j, r)]$ to refer to a specific node.

As a first example of the cycle's functionality, let us consider four different values of the step parameter h_2. The other cycle parameter values are fixed at $h_0 = 0$, $h_1 = 1$ and $N_i = 8$. In Fig. 2.6, I show four hypergraphs $\mathcal{G}_\alpha = (\mathbf{q}_0, E^{(0)}_\alpha)$ for $\alpha \in \{a, b, c, d\}$ resulting from a cycle of arity $k = 1$ on the sequence of nodes $\mathbf{q}_0 = (0, 1, 2, 3, 4, 5, 6, 7)^T$. The four connectivity matrices are given by

$$E^{(0)}_a = [0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7], \tag{2.24}$$

$$E^{(0)}_b = [0 \quad 2 \quad 4 \quad 6], \tag{2.25}$$

$$E^{(0)}_c = [0 \quad 3 \quad 6], \tag{2.26}$$

$$E^{(0)}_d = [0 \quad 4]. \tag{2.27}$$

These cycle primitives encode different traversals: when $h_2 = 1$ every node is visited, when $h_2 = 2$ every second node is visited, when $h_2 = 3$ every third node and so on. This example is a special case since each edge has a single element ($k = 1$) corresponding to the visited node. In general, the term *visited node* refers to the first element $\mathbf{e}_r[0]$ of each edge $\mathbf{e}_r$ in the connectivity matrix $E^{(i)}$. The visited nodes therefore correspond to the nodes $n_i(0, r)$ from Eq. (2.20). For $k > 1$, I will still refer to the first element of an edge as the visited node because it serves as an anchor for each step of the traversal. It is in this sense that the h_2 parameter functions as a step size measured in number of nodes. In all four examples from Fig. 2.6, the traversal starts at the node $n_i(0, 0) = 0$, then node $n_i(0, 1) = h_2$ is

https://scholar.sun.ac.za

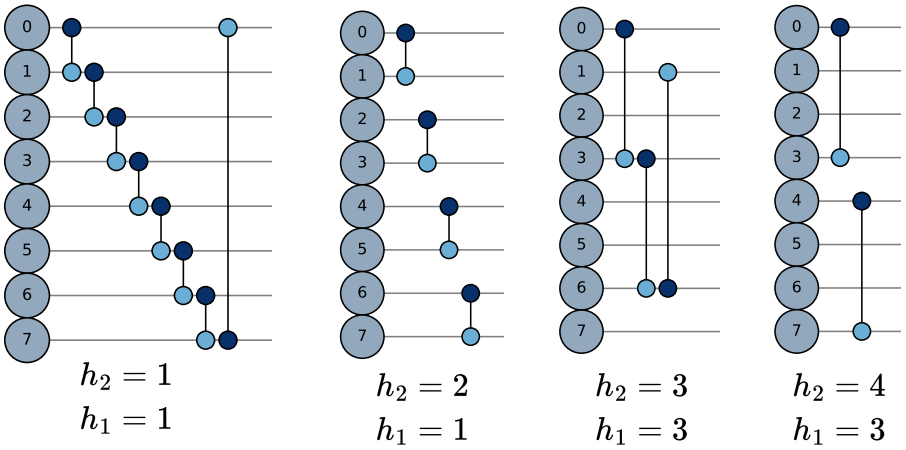


Figure 2.7: Four cycles of arity $k = 2$ instantiated on eight nodes with differing stride and step parameter values. Here $\delta_b = 0$ and $h_0 = 0$

visited, then node $n_i(0, 2) = 2h_2$ up until the final visited node $n_i(0, R-1) = (R-1)h_2$, according to Eq. (2.20). Here R is the number of nodes visited, which is the same as the number of edges in the connectivity matrix $E^{(i)}$. In this example the offset is fixed at $h_0 = 0$ and the arity at $k = 1$, substituting this into Eq. (2.22) gives

$$R = \left\lfloor \frac{N_i - 1}{h_2} \right\rfloor + 1, \tag{2.28}$$

which is the number of nodes that can be visited (including the first) before going past the final node $N_i - 1$. The stride and boundary parameters do not appear in Eq. (2.28) because of the arity value of $k = 1$. The offset parameter h_0 changes the starting point of the traversal. If the examples in Fig. 2.6 had a nonzero offset of h_0, the traversal would have started at node h_0 then visited node $h_0 + h_2$, then $h_0 + 2h_2$ and so on. The number of visited nodes is then updated by reducing the total number of nodes by the offset

$$R = \left\lfloor \frac{N_i - 1 - h_0}{h_2} \right\rfloor + 1. \tag{2.29}$$

Nodes are always visited in increasing order, i.e. $n_i(0, r) < n_i(0, r + 1)$, and the traversal ends before passing the final node, i.e. $n_i(0, R - 1) < N_i$. In Fig. 2.7, I show the same four examples from Fig. 2.6 but with an arity of $k = 2$. Notice that the same nodes are visited, i.e. the location of the darkest dots in Fig. 2.6 and

https://scholar.sun.ac.za

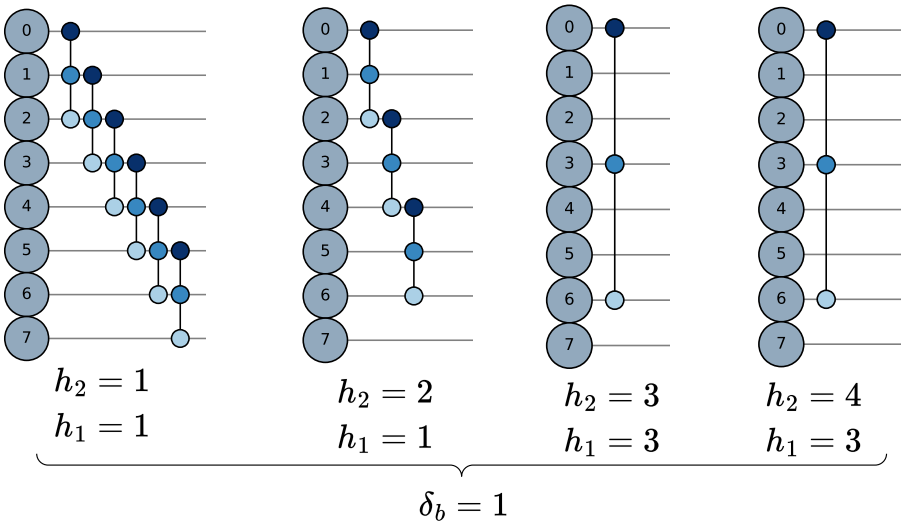


Figure 2.8: Four cycles of arity three instantiated on eight nodes with differing stride and step parameter values.

Fig. 2.7 is the same. We can also see this in the updated connectivity matrices

$$E^{(0)}_{a'} = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & 0 \end{bmatrix}, \tag{2.30}$$

$$E^{(0)}_{b'} = \begin{bmatrix} 0 & 2 & 4 & 6 \\ 1 & 3 & 5 & 7 \end{bmatrix}, \tag{2.31}$$

$$E^{(0)}_{c'} = \begin{bmatrix} 0 & 3 & 6 \\ 3 & 6 & 1 \end{bmatrix}, \tag{2.32}$$

$$E^{(0)}_{d'} = \begin{bmatrix} 0 & 4 \\ 3 & 7 \end{bmatrix}, \tag{2.33}$$

where the corresponding first rows of Eqs. (2.24)-(2.27) and Eqs. (2.30)-(2.33) are the same. Consider the two examples on the left of Fig. 2.7 with stride values of $h_1 = 1$. We see that each visited node is connected to the next node in the sequence. Then in the two examples on the right of Fig. 2.7 with stride values of $h_1 = 3$, we see that each visited node is connected to the third next node in the sequence. It might look as if the example where $h_1 = h_2 = 3$ in Fig. 2.7 does not adhere to this rule, but the boundary condition is periodic, i.e. $\delta_b = 0$, which treats the nodes as if they are on a ring. On this ring, the third next node from node 6 is node 1, i.e. $6 + 3 \bmod 8 = 1$.

The corresponding examples from Figs. 2.6 and 2.7 share the same number of edges R. This can be seen in Eq. (2.22) after substituting in the boundary condition $\delta_b = 0$ and $h_0 = 0$ so that R only depends on h_2. When the boundary

https://scholar.sun.ac.za

condition is not periodic, i.e. $\delta_b = 1$, then the number of edges (visited nodes) R depends on the step, stride, offset and arity. In Fig. 2.8, I show the same examples from Figs. 2.6 and 2.7 but with the arity increased to $k = 3$ and with an open boundary condition of $\delta_b = 1$. When comparing the $h_1 = h_2 = 1$ examples between Figs. 2.7 and 2.8, we can see that the number of visited nodes reduced from $R = 8$ to $R = 6$. This is because if node 6 were visited, the edge would cause a wrap around, i.e. $\mathbf{e}_6 = (6, 7, 0)^T$, which $\delta_b = 1$ forbids. The same can be checked for the example $h_2 = 2$ and $h_1 = 1$. Substituting in the boundary and arity parameter values, $\delta_b = 1, k = 3$, into Eq. (2.22) for the examples in Fig. 2.8 we find

$$R = \left\lfloor \frac{N_i - 1 - h_0 - 2h_1}{h_2} \right\rfloor + 1. \tag{2.34}$$

Comparing this to Eq. (2.29) we see that there is a new term that reduces the total number of nodes by $2h_1$. This term can be thought of as the node coverage of an edge; more generally, it is $(k-1)h_1$.

The two examples on the right of Fig. 2.8 correspond to the same instantiated hypergraph for different cycle parameters. The fact that certain parameter values of primitives lead to the same instantiated hypergraph for certain numbers of nodes is expected, but this will not be the case for all numbers of nodes.

The resulting hypergraphs from Figs. 2.6-2.8 are determined by the nodes $n_i(j, r)$ from Eq. (2.20) and the number of visited nodes R from Eq. (2.22). In general, the traversal starts at node h_0, and the $(r+1)^{\text{th}}$ visited node is $n_i(0, r) = h_0 + h_2 r$, where I omit the modulo operation because $n_i(0, r) < N_i$ [1]. Here h_0 functions as an offset for the starting point and h_2 as a step size measured in number of nodes. At the $(r + 1)^{\text{th}}$ visited node an edge $\mathbf{e}_r$ is generated. Its $(j + 1)^{\text{th}}$ element is given by $\mathbf{q}_i[n_i(j, r)]$ with the node $n_i(j, r) = (h_0 + h_1 j + h_2 r) \bmod N_i$ as per Eqs. (2.20) and (2.21).

In some applications, it is useful to change the ordering between the edges $\mathbf{e}_r$ in the connectivity matrix E. Within the AHL, this is enabled with a parameter called the *edge order*. This amounts to permuting the columns of the connectivity matrix. For example, if a cycle produces the connectivity matrix

$$E = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 2 & 3 \end{bmatrix}, \tag{2.35}$$

[1] This can be seen from Eq. (2.22) by noting that $R' \equiv \frac{N_i - 1 - h_0}{h_2} + 1 \geq R$, and that substituting R' into Eq. (2.20) gives $n_i(0, R' - 1) = N_i - 1$ from which we conclude that $n_i(0, r) < N_i$.

https://scholar.sun.ac.za

then specifying the edge order $(2, 0, 1)$ is equivalent to applying the permutation

$$E' = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}, \tag{2.36}$$

$$= \begin{bmatrix} 2 & 0 & 1 \\ 3 & 1 & 2 \end{bmatrix}. \tag{2.37}$$

Here E' is the new connectivity matrix for the cycle.

2.4.3 Partition

The other two primitives, mask and pivot, rely on partitioning their sequence of nodes into two subsequences. The idea is to use the position of bits within a bitstring to indicate the partition. For example, consider the sequence of nodes

$$\mathbf{q} = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}, \tag{2.38}$$

the bitstring $b = 10101$ specifies the two subsequences

$$\mathbf{q}^a = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \quad \text{and} \quad \mathbf{q}^b = \begin{bmatrix} 0 \\ 2 \\ 4 \end{bmatrix}. \tag{2.39}$$

Here, the zeros of the bitstring b specify which nodes belong to $\mathbf{q}^a$ and the ones in bitstring b specify the nodes belonging to $\mathbf{q}^b$. In general, let $\mathbf{q}_i$ be the N_i available nodes for a primitive $\mathcal{M}[i]$ in the motif $\mathcal{M}$. If we denote the indices of the sequence of nodes $\mathbf{q}_i$ by $n \in \{0, \dots, N_i - 1\}$, then the bitstring $b = b_0 \dots b_{N_i - 1}$ partitions these indices as the ordered sets

$$I_a \equiv \{n : n \in \{0, \dots, N_i - 1\}, b_n = 0\} = \{n_0 < n_1 \cdots < n_{N_a - 1}\}, \tag{2.40}$$

$$I_b \equiv \{n' : n' \in \{0, \dots, N_i - 1\}, b_{n'} = 1\} = \{n'_0 < n'_1 \cdots < n'_{N_b - 1}\}, \tag{2.41}$$

where

$$N_a \equiv N_i - N_b, \qquad \text{Number of 0's} \tag{2.42}$$

$$N_b \equiv \sum_n b_n, \qquad \text{Number of 1's} \tag{2.43}$$

https://scholar.sun.ac.za

denote the number of nodes in $\mathbf{q}_i^a$ and $\mathbf{q}_i^b$ respectively. The nodes from $\mathbf{q}_i$ appear in $\mathbf{q}_i^a$ and $\mathbf{q}_i^b$ as

$$\mathbf{q}_i^a \equiv \Big(\mathbf{q}_i[n]\Big)^T_{n\in I_a}, \tag{2.44}$$

$$\mathbf{q}_i^b \equiv \Big(\mathbf{q}_i[n]\Big)^T_{n\in I_b}. \tag{2.45}$$

Within the AHL, the bitstring b is generated by a base pattern b^p that is independent of the number of nodes. There are two options for specifying a base pattern. The first is a string containing only bits, and the second is a string containing bits and *wild card characters*. The first case is simplest: the base pattern is repeatedly concatenated with itself until its length equals the current number of nodes N_i. If the bitstring overshoots N_i, trim the tail so that it does not exceed N_i. For example, the base pattern $b^p =$ `010` produces the bitstrings

b^p	N_i	bitstring
010	5	01001
	6	010010
	7	0100100

(2.46)

The second way to generate the bitstring is with the wild card characters ! and $*$. The ! fills the string with ones, and the $*$ fills the string with zeros. Some typical examples include

b^p	N_i	bitstring
*!	8	00001111
*!	4	0011
1*1	7	1000001
!	12	000111111000

(2.47)

From Eq. (2.47) we see that for any number of nodes the base pattern `*!` will partition the nodes into a first and last half. The pattern `1*1` will partition the first and last nodes from the rest, and `*!*` partitions the middle from the rest. The full algorithm for determining the bitstring from a wildcard pattern is given in Appendix A. The basic idea is to search for the first $*$ and when found, insert `0` adjacent to it. With the updated string search for the first `!` and when found, insert `1` adjacent to it. This process starts with the base pattern and updates it until its length (excluding the $*$ and `!`) equals N_i. There are some minor nuances and a few arbitrary design choices. The algorithm starts with the search for $*$ as opposed to `!`, leading to more zeros in strings that cannot be filled symmetrically. The insertion of a bit is always adjacent to the right of the wildcard. The search for a wild character also alternates between searching from the left and right of the string to handle cases with two $*$ or two `!`.

https://scholar.sun.ac.za

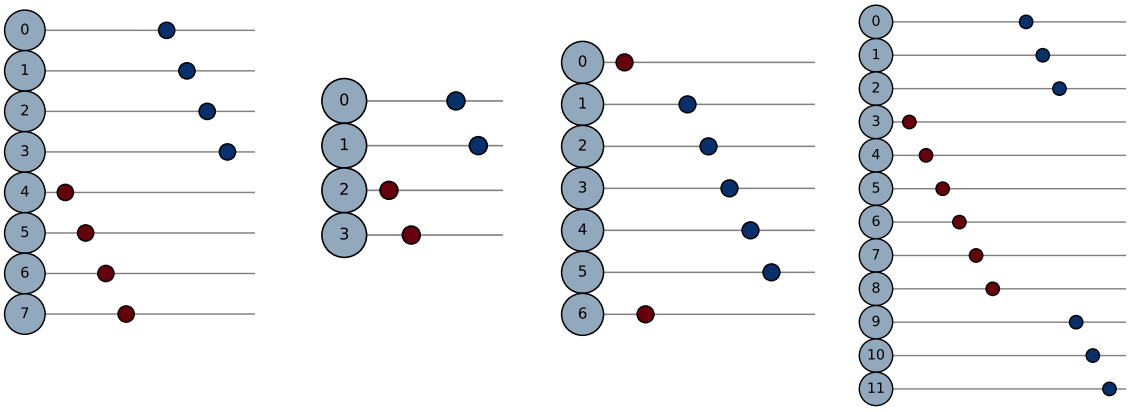


Figure 2.9: Four hypergraphs created from a motif that consists of an arity $k = 1$ mask followed by an arity $k = 1$ cycle. The edges generated by the mask are shown in red, and the corresponding nodes are unavailable for subsequent primitives. The edges from the cycle are shown in blue, indicating which nodes they had available.

2.4.4 Mask

The mask primitive partitions its sequence of nodes $\mathbf{q}_i$ into two subsequences $\mathbf{q}_i^a$ and $\mathbf{q}_i^b$ and makes the sequence $\mathbf{q}_i^a$ available for subsequent primitives, i.e.

$$\mathbf{q}_i' \equiv \mathbf{q}_i^a. \tag{2.48}$$

The partition is specified with a base pattern b^p as discussed in Section 2.4.3. For example, consider the four base patterns from Eq. (2.47). Providing these base patterns to an arity $k = 1$ mask produces the hypergraphs seen in Fig. 2.9. Here I show four hypergraphs generated from a motif of the form $\mathcal{M} = \mathcal{P}_m + \mathcal{P}_c$. This motif consists of a mask $\mathcal{P}_m$ with the particular base pattern b^p followed by the cycle $\mathcal{P}_c$ from Fig. 2.6 (a). The cycle $\mathcal{P}_c$ only serves to show which nodes it had available, since it places an arity one edge on each available node. This example in Fig. 2.9 shows the masked subsequence $\mathbf{q}_i^b$ with red dots and the remaining subsequence $\mathbf{q}_i^a$ with blue dots. Since the mask hides the nodes in $\mathbf{q}_i^b$ we can interpret the two examples on the left of Fig. 2.9 as always hiding the last half of the nodes. The $b^p = 1*1$ example hides the first and last nodes from the rest, and $b^p = *!*$ hides the middle from the ends.

Masks may have an arity $k > 1$. For example, in Fig. 2.10, I have increased the arity of the mask primitives in Fig. 2.9 to $k = 2$. We see that the two subsequences, $\mathbf{q}_i^a$ and $\mathbf{q}_i^b$, are connected. The first element $\mathbf{e}_r[0]$ of each edge $\mathbf{e}_r$ created by the

https://scholar.sun.ac.za

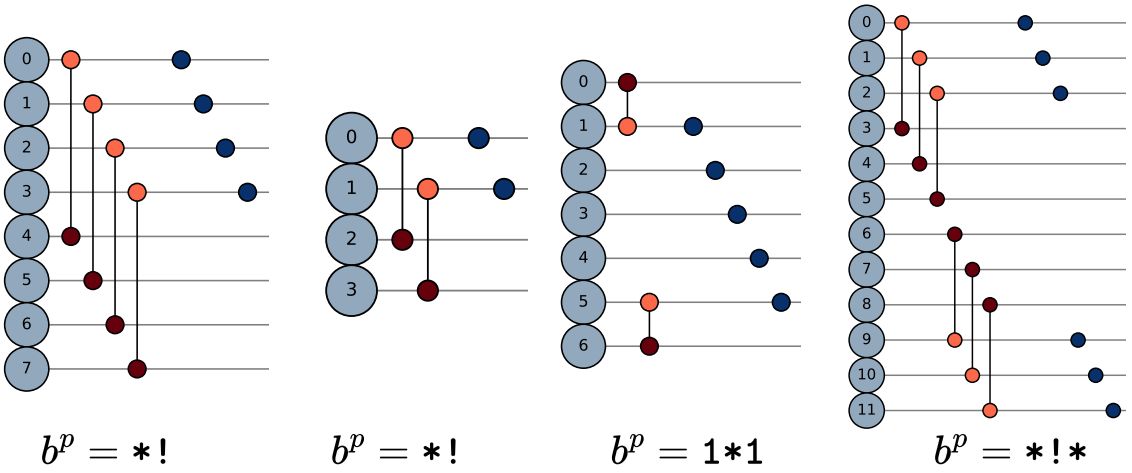


Figure 2.10: Four hypergraphs created from a motif that consists of an arity $k = 2$ mask followed by an arity $k = 1$ cycle. The edges generated by the mask are shown in red, and the corresponding dark nodes are unavailable for subsequent primitives.

mask comes from the $\mathbf{q}_i^b$ sequence and the second element $\mathbf{e}_r[1]$ from the $\mathbf{q}_i^a$ sequence. The connection between the two sequences is specified with a bitstring b^e, called the edge pattern, in this example $b^e = 10$. Generally speaking, if $b_n^e = 1$ then the edge element $\mathbf{e}_r[n]$ comes from the $\mathbf{q}_i^b$ sequence and if $b_n^e = 0$, the edge element $\mathbf{e}_r[n]$ comes from the $\mathbf{q}_i^a$ sequence. Notice also how each masked node, a node in $\mathbf{q}_i^b$, is connected to a remaining node, a node in $\mathbf{q}_i^a$, but not vice versa. This will generally be the case: the mask primitive will always try to connect up the masked nodes to the remaining ones.

Let us focus on the hypergraph on the left of Fig. 2.10 with a base pattern of $b^p = *!$. The first primitive $\mathcal{M}[0]$ in this motif $\mathcal{M} = \mathcal{P}'_m + \mathcal{P}_c$ is a mask which creates the hypergraph $\mathcal{G}_0 = (\mathbf{q}_0, E^{(0)})$. The initial nodes are

$$\mathbf{q}_0 = (0, 1, 2, 3, 4, 5, 6, 7)^T, \tag{2.49}$$

and the connectivity matrix given by

$$E^{(0)} = \begin{bmatrix} 4 & 5 & 6 & 7 \\ 0 & 1 & 2 & 3 \end{bmatrix}. \tag{2.50}$$

This mask makes the subsequence $\mathbf{q}_0^a$, which is the first half of $\mathbf{q}_0$, available to the cycle $\mathcal{M}[1]$, i.e.

$$\mathbf{q}'_0 = \mathbf{q}_0^a = \mathbf{q}_1 = (0, 1, 2, 3)^T. \tag{2.51}$$

https://scholar.sun.ac.za

The cycle $\mathcal{M}[1]$ then creates the hypergraph $\mathcal{G}_1 = (\mathbf{q}_1, E^{(1)})$ where the connectivity matrix is given by

$$E^{(1)} = \begin{bmatrix} 0 & 1 & 2 & 3 \end{bmatrix}. \tag{2.52}$$

The resulting hypergraph from the motif $\mathcal{M}$ is therefore

$$\mathcal{G} = \Big(\mathbf{q}_0, (E^{(0)}, E^{(1)})\Big). \tag{2.53}$$

Continuing with the example from Fig. 2.10, with the base pattern of $b^p = *!$, let us look at how $E^{(0)}$ is generated. The two subsequences from the partition is given by $\mathbf{q}_0^a = (0,1,2,3)^T$ and $\mathbf{q}_0^b = (4,5,6,7)^T$. The arity of the mask is $k = 2$, so that each edge $\mathbf{e}_r$ contains two nodes. The $b^e = 10$ bitstring partitions each edge into two parts, one with size $k_a = 1$ and the other with size $k_b = 1$. Here k_a indicates the number of nodes in each edge $\mathbf{e}_r$ belonging to $\mathbf{q}_0^a$ and k_b the number of nodes in each edge belonging to $\mathbf{q}_0^b$. Therefore, we have that $k = k_a + k_b$. The mask primitive will generate a connectivity matrix $E_\alpha^{(0)}$ on each subsequence $\mathbf{q}_0^\alpha$ through an independent arity k_α cycle for $\alpha \in \{a, b\}$. In our example from Fig. 2.10, with the base pattern of $b^p = *!$, the same cycle is used for both $\mathbf{q}_0^a$ and $\mathbf{q}_0^b$ with $h_0 = 0, h_1 = 1, h_2 = 1$ and $\delta_b = 0$. Applying Eqs. (2.20)-(2.22) on each $\mathbf{q}_0^\alpha$ with these cycle parameters gives the connectivity matrices

$$E_a^{(0)} = \begin{bmatrix} 0 & 1 & 2 & 3 \end{bmatrix}, \tag{2.54}$$

$$E_b^{(0)} = \begin{bmatrix} 4 & 5 & 6 & 7 \end{bmatrix}. \tag{2.55}$$

The connectivity matrices from Eqs. (2.54) and (2.55) are then combined into one connectivity matrix $E^{(0)}$. We see that Eqs. (2.54) and (2.55) were zipped together into Eq. (2.50). The first row of $E^{(0)}$ comes from Eq. (2.55) because the first bit b_0^e in the bitstring b^e indicates that those elements should come from the $\mathbf{q}_0^b$ subsequence.

In general, consider a hypergraph $\mathcal{G}$ created by some motif $\mathcal{M}$ as per Eq. (2.17) with N initial nodes contained in $\mathbf{q}_0$. Let $\mathbf{q}_i$ be the sequence of N_i available nodes for an arity k mask $\mathcal{M}[i]$ in the motif $\mathcal{M}$. Then the mask will partition its sequence of nodes $\mathbf{q}_i$ into two subsequences $\mathbf{q}_i^a$ and $\mathbf{q}_i^b$ as specified through the base pattern b^p. Each edge $\mathbf{e}_r$ in the resulting connectivity matrix $E^{(i)}$ will also be partitioned as specified with the edge pattern $b^e = b_0^e \cdots b_{k-1}^e$. The edge pattern determines

$$k_a \equiv k - k_b, \tag{2.56}$$

$$k_b \equiv \sum_{n=0}^{k-1} b_n^e, \tag{2.57}$$

https://scholar.sun.ac.za

which are the effective arities of the two independent cycles that will create the connectivity matrices $E_a^{(i)}$ and $E_b^{(i)}$. These independent cycles have the parameters $h_0^a, h_1^a, h_2^a, \delta_b^a$ and $h_0^b, h_1^b, h_2^b, \delta_b^b$ which generate $E_a^{(i)}$ and $E_b^{(i)}$ through Eqs. (2.20) and (2.21) on the subsequences $\mathbf{q}_i^a$ and $\mathbf{q}_i^b$. The number of edges R_a and R_b in each connectivity matrix $E_a^{(i)}$ and $E_b^{(i)}$ is given by Eq. (2.22). Furthermore, let

$$r' \equiv (r + h_1^c) \bmod R_a, \tag{2.58}$$

with $h_1^c \in [0, N]$ denoting a stride between partitions. The resulting connectivity matrix $E^{(i)}$ is determined by

$$E_{j,r}^{(i)} \equiv \begin{cases} (E_a^{(i)})_{n_0(j),r'} & \text{if } b_j^e = 0, \\ (E_b^{(i)})_{n_1(j),r} & \text{if } b_j^e = 1, \end{cases} \tag{2.59}$$

where

$$n_0(j) \equiv j - n_1(j), \tag{2.60}$$

$$n_1(j) \equiv \sum_{n=0}^{j-1} b_n^e. \tag{2.61}$$

returns the number of zeros or ones in b^e up to and excluding the index j and $n_1(0) \equiv 0$. In Eq. (2.59) the indices take on the values $j \in \{0, \dots, k-1\}$ and $r \in \{0, \dots, R_b - 1\}$. Notice that the number of edges is based on the sequence $\mathbf{q}_i^b$, which allows the mask primitive to connect up all the masked nodes to the remaining ones. The index r' from Eq. (2.58) handles the case for when there are fewer remaining nodes than masked nodes, i.e. $R_a < R_b$, and ensures all masked nodes are covered.

There is another type of mask primitive, called the *unmask*, which unhides the nodes that a previous mask hid. For example, let $\mathcal{P}_m, \mathcal{P}_{m'}$ and $\mathcal{P}_c$ denote two different mask primitives and a cycle, respectively. Furthermore, let $\mathcal{P}_u$ denote an unmask primitive. Consider the following motif

$$\mathcal{M} = \mathcal{P}_m + \mathcal{P}_c + \mathcal{P}_{m'} + \mathcal{P}_c + \mathcal{P}_u + \mathcal{P}_c. \tag{2.62}$$

The sequence of nodes $\mathbf{q}_1$ for the first cycle $\mathcal{M}[1]$ will be a subsequence of $\mathbf{q}_0$ due to the first mask, i.e. $\mathbf{q}_1 = \mathbf{q}_0'$. The sequence of nodes $\mathbf{q}_3$ for the second cycle $\mathcal{M}[3]$ will be a subsequence of $\mathbf{q}_2$ due to the second mask, i.e. $\mathbf{q}_3 = \mathbf{q}_2'$. The unmask primitive $\mathcal{M}[4]$ will unhide the nodes that the mask $\mathcal{M}[2] = \mathcal{P}_{m'}$ hid. Therefore, the sequence of nodes $\mathbf{q}_5$ for the last cycle $\mathcal{M}[5]$ will be the sequence $\mathbf{q}_1$. In fact, we have that $\mathbf{q}_5 = \mathbf{q}_2 = \mathbf{q}_1$.

https://scholar.sun.ac.za

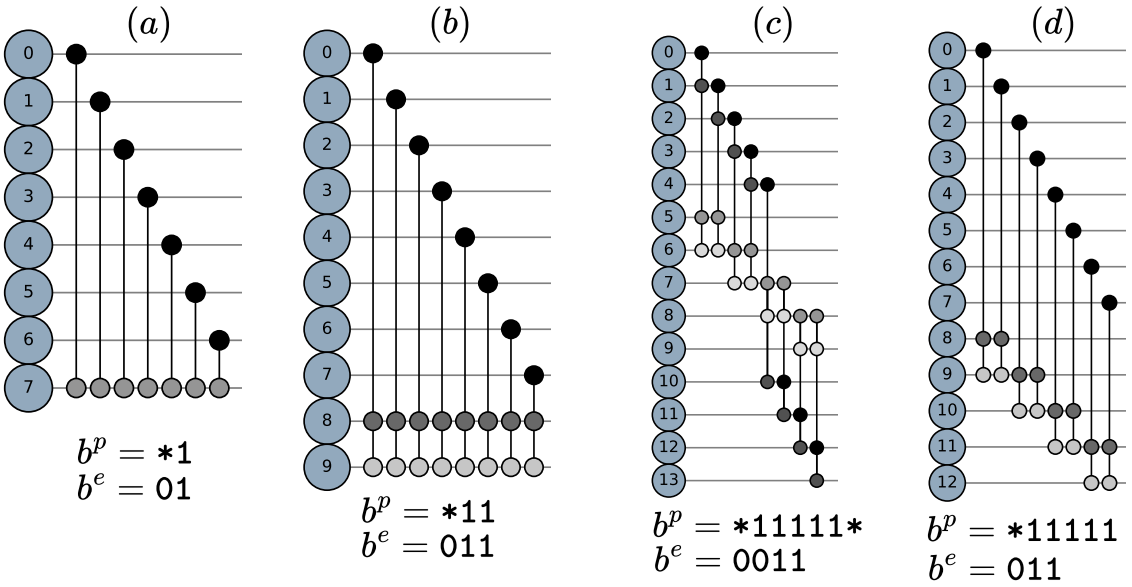


Figure 2.11: For examples of hypergraphs that were created by different pivot primitives.

2.4.5 Pivot

The pivot primitive partitions its sequence of nodes $\mathbf{q}_i$ into two subsequences $\mathbf{q}_i^a$ and $\mathbf{q}_i^b$ and makes all its nodes available for subsequent primitives, i.e.

$$\mathbf{q}_i' \equiv \mathbf{q}_i. \tag{2.63}$$

I refer to the nodes in the $\mathbf{q}_i^b$ sequence as the pivoted nodes, and to the nodes in $\mathbf{q}_i^a$ as the remaining nodes. The pivot is similar to the mask; both partition their nodes into two subsequences. The mask leaves only one subsequence available for subsequent primitives, while the pivot leaves both available. The other difference is in how the two edge partitions are connected into the connectivity matrix. In Fig. 2.11, I show four examples of hypergraphs created from different pivot primitives. On the left (a), the base pattern is $b^p = *1$ which partitions the last node from the rest. Here, the pivot connects each remaining node with the pivoted node (the last node). This will generally be the case: the pivot primitive will always try to connect the remaining nodes to the pivoted ones.

As with the mask, the edge pattern b^e specifies a partition for each edge. In the example of Fig. 2.11(a), $b^e = 01$, which means the first element of each edge is a remaining node and the second element a pivot node. Generally, $b_n^e = 1$ indicates that the edge element $\mathbf{e}_r[n]$ is a pivot node and if $b_n^e = 0$, the edge element $\mathbf{e}_r[n]$ is a remaining node. In Fig. 2.11(b), I show an arity $k = 3$ pivot, where the

https://scholar.sun.ac.za

sub-arities are $k_a = 1$ and $k_b = 2$, as determined by the edge pattern b^e from Eqs. (2.56) and (2.57). We see a similar pattern as in example (a); each remaining node is connected to a pivot node pair. These two examples in Fig. 2.11 (a) and (b) show a special case of the pivot where the number of pivot nodes in any edge is the same as the total number of pivot nodes, i.e. $k_b = N_b$. Here N_b is the size of the $\mathbf{q}_i^b$ sequence as given by Eq. (2.43).

The hypergraph in Fig. 2.11(c) shows an example where this is not the case. Here, the middle five nodes are partitioned from the rest of the nodes. As with the mask, there will be two independent cycles on each subsequence that will determine the connectivity matrix. In this example $k_b = 2$ and $k_a = 2$, therefore the pivoted edges are determined by an arity $k_b = 2$ cycle and the remaining edges by an arity $k_a = 2$ cycle. Notice that similar pivot node pairs are grouped together in this example, i.e. the node pair $(5, 6)^T$ is present in the first and second edge and therefore next to each other. Similarly, the third and fourth edges both contain the same node pair $(6, 7)^T$, and the fifth and sixth edges contain $(8, 9)^T$. Generally, "like pivots" are grouped together, and this feature distinguishes the pivot from the mask.

Let us look at the example of Fig. 2.11(d) to further detail this difference. Denote the hypergraph as $\mathcal{G} = (\mathbf{q}_0, E^{(0)})$. There will be two independent cycles on each subsequence $\mathbf{q}_0^a$ and $\mathbf{q}_0^b$. Each cycle has its own parameter values and creates the sub-connectivity matrix $E_a^{(0)}$ or $E_b^{(0)}$. Both sub-connectivity matrices $E_a^{(0)}$ and $E_b^{(0)}$ are joined into $E^{(0)}$ based on the edge pattern b^e. The difference between the mask and pivot will be in how these two sub-connectivity matrices are joined up. Specifically, for this example in Fig. 2.11(d), the two sub-connectivity matrices created by the independent cycles on each subsequence are given by

$$E_a^{(0)} = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{bmatrix}, \tag{2.64}$$

$$E_b^{(0)} = \begin{bmatrix} 8 & 9 & 10 & 11 \\ 9 & 10 & 11 & 12 \end{bmatrix}, \tag{2.65}$$

and joined into the single connectivity matrix

$$E^{(0)} = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 8 & 8 & 9 & 9 & 10 & 10 & 11 & 11 \\ 9 & 9 & 10 & 10 & 11 & 11 & 12 & 12 \end{bmatrix}. \tag{2.66}$$

Compare this to how these two sub-connectivity matrices from Eqs. (2.64) and (2.65)

https://scholar.sun.ac.za

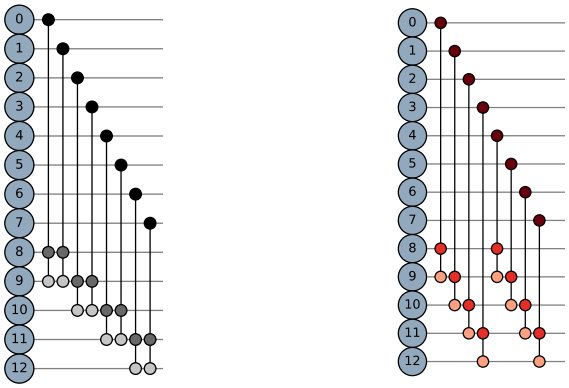


Figure 2.12: The difference between a pivot and mask where the same two sub-connectivity matrices are joined up in different ways.

would have been joined up by the mask

$$E^{(0)}_{\text{mask}} = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 & 8 & 9 & 10 & 11 \\ 9 & 10 & 11 & 12 & 9 & 10 & 11 & 12 \end{bmatrix}. \tag{2.67}$$

In Fig. 2.12, I further highlight this difference between the mask and the pivot. We see that the pivot groups "like pivots" together. For instance, the two pivoted node pairs $(8,9)^T$ are adjacent whereas the two masked node pairs $(8,9)^T$ are not.

In general, consider a hypergraph $\mathcal{G}$ created by some motif $\mathcal{M}$ as per Eq. (2.17) with N initial nodes contained in $\mathbf{q}_0$. Let $\mathbf{q}_i$ be the sequence of N_i available nodes for an arity k pivot $\mathcal{M}[i]$ in the motif $\mathcal{M}$. Then the pivot will partition its sequence of nodes $\mathbf{q}_i$ into two subsequences $\mathbf{q}^a_i$ and $\mathbf{q}^b_i$ as specified through the base pattern b^p. Each edge $\mathbf{e}_r$ in the resulting connectivity matrix $E^{(i)}$ will also be partitioned as specified with the edge pattern $b^e = b^e_0 \cdots b^e_{k-1}$. The edge pattern determines

$$k_a = k - k_b, \tag{2.68}$$

$$k_b = \sum_{n=0}^{k-1} b^e_n, \tag{2.69}$$

which are the effective arities of the two independent cycles that will create the connectivity matrices $E^{(i)}_a$ and $E^{(i)}_b$. These independent cycles have the parameters $h^a_0, h^a_1, h^a_2, \delta^a_b$ and $h^b_0, h^b_1, h^b_2, \delta^b_b$ which generate $E^{(i)}_a$ and $E^{(i)}_b$ through Eqs. (2.20) and (2.21) on the subsequences $\mathbf{q}^a_i$ and $\mathbf{q}^b_i$. The number of edges R_a and R_b in

https://scholar.sun.ac.za

each connectivity matrix $E_a^{(i)}$ and $E_b^{(i)}$ is given by Eq. (2.22). Let c and d denote the quotient and remainder of R_a/R_b, so that $R_a = cR_b + d$. Furthermore, let

$$r' \equiv \begin{cases} \left(\left\lfloor \dfrac{r}{c+1} \right\rfloor + h_1^c\right) \bmod R_b & \text{if } r < (c+1)d, \\ \left(\left\lfloor \dfrac{r-d}{c} \right\rfloor + h_1^c\right) \bmod R_b & \text{otherwise,} \end{cases} \tag{2.70}$$

where $h_1^c \in [0, N]$ denotes a stride between partitions. The resulting connectivity matrix $E^{(i)}$ is determined by

$$E_{j,r}^{(i)} \equiv \begin{cases} \left(E_a^{(i)}\right)_{n_0(j),r} & \text{if } b_j^e = 0, \\ \left(E_b^{(i)}\right)_{n_1(j),r'} & \text{if } b_j^e = 1, \end{cases} \tag{2.71}$$

where $n_0(j)$ and $n_1(j)$ are given by Eqs. (2.60) and (2.61). In Eq. (2.71) the indices take on the values $j \in \{0, \dots, k-1\}$ and $r \in \{0, \dots, R_a - 1\}$. The definition of r' in Eq. (2.70) indexes the $E_b^{(i)}$ sub-connectivity matrix in Eq. (2.71). This definition causes the "like pivots" to be grouped together. In Eq. (2.71), we see that the connectivity matrix $E^{(i)}$ shares its r index with the sub-connectivity matrix $E_a^{(i)}$. This causes the pivot to try and connect each remaining node with a pivot node, and not vice versa.

2.4.6 Permute

The permute primitive creates a connectivity matrix containing permutations or combinations of its nodes. Let P_k (C_k) denote the set of possible permutations (combinations) of k elements

$$P_k \equiv \left\{(a_0, \dots, a_{k-1}) \in \{0, \dots, N_i - 1\}^k : \ a_j \text{ all distinct}\right\}, \tag{2.72}$$

$$C_k \equiv \left\{(a_0, \dots, a_{k-1}) \in \{0, \dots, N_i - 1\}^k : \ a_0 < a_1 < \cdots < a_{k-1}\right\}. \tag{2.73}$$

Equip these sets with a lexicographic order indexed by r

$$\{p^{(r)}\}_{r=0}^{R-1} \equiv \begin{cases} P_k, & \delta_p = 0, \\ C_k, & \delta_p = 1, \end{cases} \tag{2.74}$$

where δ_p distinguishes between permutations and combinations and R is determined by

$$R \equiv \begin{cases} \frac{N_i!}{(N_i-k)!}, & \delta_p = 0, \\ \binom{N_i}{k}, & \delta_p = 1. \end{cases} \tag{2.75}$$

https://scholar.sun.ac.za

Consider a hypergraph $\mathcal{G}$ created by some motif $\mathcal{M}$ as per Eq. (2.17) with N initial nodes contained in $\mathbf{q}_0$. Let $\mathbf{q}_i$ be the sequence of N_i available nodes for an arity k permute primitive $\mathcal{M}[i]$ in the motif $\mathcal{M}$. This permute primitive will generate the connectivity matrix $E^{(i)}$ as follows

$$E^{(i)}_{j,r} \equiv \mathbf{q}_i[p^{(r)}_j], \tag{2.76}$$

where $j \in \{0, 1, \dots, k-1\}$ and $r \in \{0, 1, \dots, R-1\}$. In other words, each column of $E^{(i)}$ is just one element from either P_k or C_k.

2.4.7 Default parameter values

I will frequently be referring to primitives for the rest of the thesis and it will be useful to have a notion of the default cycle, mask, pivot and permute. Here, I provide the default parameter values that each primitive assumes if not specified. These values are in accordance with the python package [1].

Cycle					
Parameter	k	δ_b	h_0	h_1	h_2
Value	2	0	0	1	1

Table 2.1: Default cycle parameter values

Mask												
Parameter	k	δ^a_b	δ^b_b	b^p	b^e	h^a_0	h^a_1	h^a_2	h^b_0	h^b_1	h^b_2	h^c_1
Value	2	1	1	`1*`	`10`	0	1	1	0	1	1	0

Table 2.2: Default mask parameter values

Pivot												
Parameter	k	δ^a_b	δ^b_b	b^p	b^e	h^a_0	h^a_1	h^a_2	h^b_0	h^b_1	h^b_2	h^c_1
Value	2	1	1	`1*`	`10`	0	1	1	0	1	1	0

Table 2.3: Default pivot parameter values

https://scholar.sun.ac.za

Permute		
Parameter	k	δ_p
Value	2	1

Table 2.4: Default permute parameter values

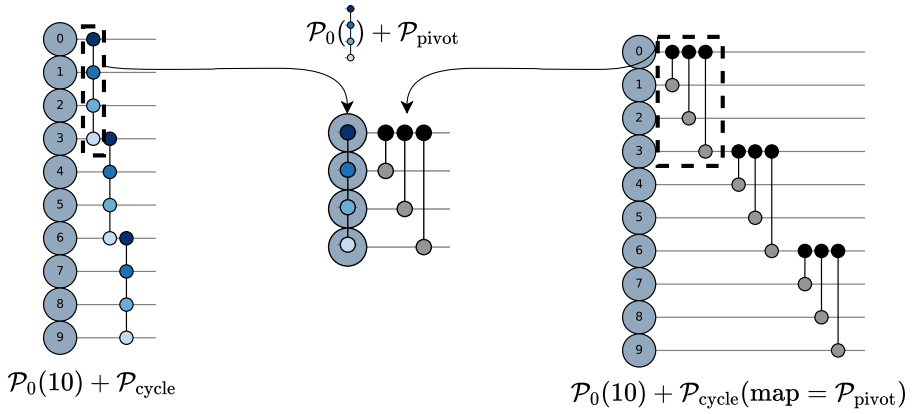


Figure 2.13: Left: hypergraph generated by an arity $k = 4$ cycle primitive. Right: the same cycle uses a pivot primitive as a mapping on each of its edges, producing a hierarchically composed hypergraph.

2.5 Hierarchical composition

Consider the hypergraph $\mathcal{G}_{\text{orig}} = (\mathbf{q}_0, E^{(0)})$ on the left of Fig. 2.13. It was created by the cycle primitive $\mathcal{P}_{\text{cycle}}(k = 4, \delta_b = 1, h_2 = 3)$, where the parameter values that are different from the primitive defaults are listed in brackets. Its sequence of nodes and connectivity matrix are given by

$$\mathbf{q}_0 = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)^T, \tag{2.77}$$

$$E^{(0)} = \begin{bmatrix} 0 & 3 & 6 \\ 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}. \tag{2.78}$$

In Eq. (2.78) we see that each edge $\mathbf{e}_r$ in $E^{(0)}$ is a subsequence of the sequence of nodes $\mathbf{q}_0$, by definition. If we treat each edge $\mathbf{e}_r$ as an initial sequence of nodes for a default pivot, i.e. $\mathcal{G}_r = \mathcal{P}_0(\mathbf{e}_r) + \mathcal{P}_{\text{pivot}}(k = 2)$, then we obtain the following

https://scholar.sun.ac.za

hypergraphs

$$\underset{\mathbf{e}_0}{\mathcal{G}_0} = (\mathbf{e}_0, E_0) \qquad \underset{\mathbf{e}_1}{\mathcal{G}_1} = (\mathbf{e}_1, E_1) \qquad \underset{\mathbf{e}_2}{\mathcal{G}_2} = (\mathbf{e}_2, E_2)$$

$$\begin{bmatrix}0\\1\\2\\3\end{bmatrix}, \overset{E_0}{\begin{bmatrix}0&0&0\\1&2&3\end{bmatrix}}, \quad \begin{bmatrix}3\\4\\5\\6\end{bmatrix}, \overset{E_1}{\begin{bmatrix}3&3&3\\4&5&6\end{bmatrix}}, \quad \begin{bmatrix}6\\7\\8\\9\end{bmatrix}, \overset{E_2}{\begin{bmatrix}6&6&6\\7&8&9\end{bmatrix}}. \tag{2.79}$$

Since all the elements in E_r, for $r \in \{0,1,2\}$, are nodes from the sequence $\mathbf{q}_0$, we may form a new hypergraph $\mathcal{G}_{\text{new}} = (\mathbf{q}_0, E^{(0)}_{\text{new}})$ by combining these connectivity matrices into one

$$E^{(0)}_{\text{new}} = \begin{bmatrix}0&0&0&3&3&3&6&6&6\\1&2&3&4&5&6&7&8&9\end{bmatrix}. \tag{2.80}$$

Eq. (2.80) is the connectivity matrix for the hypergraph on the right of Fig. 2.13. Loosely speaking, we have created a hypergraph by cycling a pivot. I call this *hierarchical composition* within the AHL.

Hierarchical composition is accomplished with a new parameter that all primitives have, called *map*. The main idea is that an arbitrary motif can be provided to a primitive as a mapping. The resulting hypergraph is then formed by instantiating that motif (the mapping) on each edge in the connectivity matrix produced by the primitive. In general, this map parameter points to a function that is applied on each edge in a connectivity matrix of a primitive. In the next section we will see how an operator acting on a quantum state can also be provided as a mapping. The example from Fig. 2.13 shows a special case: the mapping provided to the cycle is a single primitive (a pivot), and this primitive (pivot) does not have its own mapping. This will not generally be the case. The mapping provided to a primitive can be a motif that consists of multiple primitives. These primitives can themselves have motifs as mappings and so on. The result is a hierarchy of primitives, of which I show an example in Fig. 2.14. Here, the following motifs are constructed

$$\mathcal{M}^{(0)}_1 = \mathcal{P}^{(0)}_1 + \mathcal{P}^{(1)}_1 + \mathcal{P}^{(2)}_1, \tag{2.81}$$
$$\mathcal{M}^{(0)}_2 = \mathcal{P}^{(0)}_2 + \mathcal{P}^{(1)}_2, \tag{2.82}$$
$$\mathcal{M}^{(1)}_2 = \mathcal{P}^{(2)}_2 + \mathcal{P}^{(3)}_2, \tag{2.83}$$
$$\mathcal{M}^{(0)}_3 = \mathcal{P}^{(0)}_3 + \mathcal{P}^{(1)}_3, \tag{2.84}$$

https://scholar.sun.ac.za

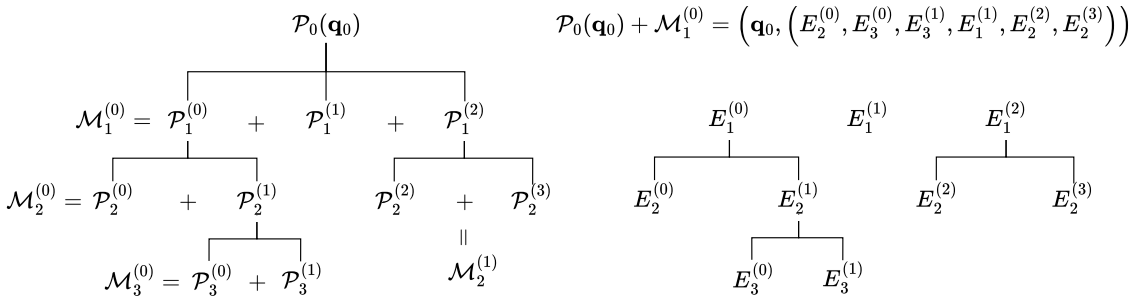


Figure 2.14: Hierarchy of primitives (left) and corresponding hierarchy of connectivity matrices (right); the top expression shows the depth-wise enumeration that produces the final connectivity matrix.

with the following primitives having the mappings

$$\mathcal{P}_1^{(0)} = \mathcal{P}_1^{(0)}(\text{map} = \mathcal{M}_2^{(0)}), \tag{2.85}$$

$$\mathcal{P}_1^{(2)} = \mathcal{P}_1^{(2)}(\text{map} = \mathcal{M}_2^{(1)}), \tag{2.86}$$

$$\mathcal{P}_2^{(1)} = \mathcal{P}_2^{(1)}(\text{map} = \mathcal{M}_3^{(0)}). \tag{2.87}$$

Here I am using a subscript to denote the level of the hierarchy (increasing from top to bottom) and a superscript to indicate the order of a primitive on that level. At the top of the hierarchy, level zero, is the initialisation primitive. The motif $\mathcal{M}_1^{(0)}$ is added to the init primitive which creates level one of the tree. The primitives $\mathcal{P}_1^{(0)}$ and $\mathcal{P}_1^{(2)}$ both have motifs as mappings, causing two separate branches into level two. The primitive $\mathcal{P}_2^{(1)}$ has the motif $\mathcal{M}_3^{(0)}$ as a mapping, creating level three. The final sequence of connectivity matrices, seen on the right of Fig. 2.14, is obtained by traversing this tree depth-wise and listing the matrices in the order induced by the hierarchy.

2.6 Domains of application

The AHL attempts to capture simple hypergraph patterns and makes no reference to a specific domain. Whenever the form of a solution to a problem can benefit from being expressed as a hypergraph, then the AHL might be used to search for such forms. Here I discuss the domains of application for the AHL that we have considered so far.

https://scholar.sun.ac.za

2.6.1 Tensor networks

We can build tensor networks using the abstract hypergraph language (AHL). Consider a hypergraph $\mathcal{G} = (\mathbf{q}_0, (E^{(i)}))$ created by a motif $\mathcal{M}$. Each node in $\mathbf{q}_0$ labels an open leg of an initial tensor network. If we associate a tensor with each edge in the hypergraph, then the edge elements indicate the connectivity of that tensor within the tensor network. The order between edges is important here, two consecutive edges with overlapping edge elements indicate a connection between them.

Let us use the AHL to build tensor networks that represent quantum states. A generic pure quantum state on a one-dimensional lattice with N sites, each with a local d-dimensional state space $\{\sigma_i\}$, can be written as

$$|\psi\rangle = \sum_{\sigma_0,\cdots,\sigma_{N-1}} \psi_{\sigma_0,\cdots,\sigma_{N-1}} |\sigma_0,\cdots,\sigma_{N-1}\rangle , \tag{2.88}$$

where $\psi_{\sigma_0,\cdots,\sigma_{N-1}} \in \mathbb{C}$. I will adopt the following compact notation for the action of a general k site operator $O_{n_0,\ldots,n_{k-1}}$. If $k = 1$ and O_n acts on site n of the lattice then

$$O_n |\psi\rangle = \underbrace{I \otimes \cdots \otimes I}_{n} \otimes O \otimes \underbrace{I \otimes \cdots \otimes I}_{N-n-1} |\psi\rangle , \tag{2.89}$$

where I denotes the identity operator. This will be extended when $k > 1$ in the same way, with identity operators applied to the sites $n \notin (n_0, \ldots, n_{k-1})$. Now consider a hypergraph $\mathcal{G}$ created by some motif $\mathcal{M}$ as per Eq. (2.17) with N initial nodes contained in $\mathbf{q}_0$. Let $E^{(i)}$ be the $k_i \times R_i$ connectivity matrix generated by the primitive $\mathcal{M}[i]$ in the motif $\mathcal{M}$. Associate a k_i site operator $O^{(i)}_{n_0,\ldots,n_{k_i-1}}$ with each primitive $\mathcal{M}[i]$ in the motif $\mathcal{M}$. Then the hypergraph $\mathcal{G}$ builds the quantum state

$$|\psi\rangle = \left(\overleftarrow{\prod_{E^{(i)}\in\mathcal{G}}} \overleftarrow{\prod_{\mathbf{e}_r\in E^{(i)}}} O^{(i)}_{\mathbf{e}_r} \right) |\psi_0\rangle , \tag{2.90}$$

where $|\psi_0\rangle$ is some initial state on a one-dimensional lattice with N sites. The operator products in Eq. (2.90) are ordered with the r and i indices increasing from right to left. Here we see that the motif $\mathcal{M}$ is a tensor network representing $|\psi\rangle$. Since the motif is independent of system size, so is the tensor network. Therefore the motif also specifies how $|\psi\rangle$ scales with increasing system size. If all the operators $O^{(i)}_{\mathbf{e}_r}$ are unitary, then we can regard the motif $\mathcal{M}$ as an abstract quantum circuit. When I refer to the operator associated with a primitive, I mean the operator passed through its map argument. For hierarchical composition discussed

https://scholar.sun.ac.za

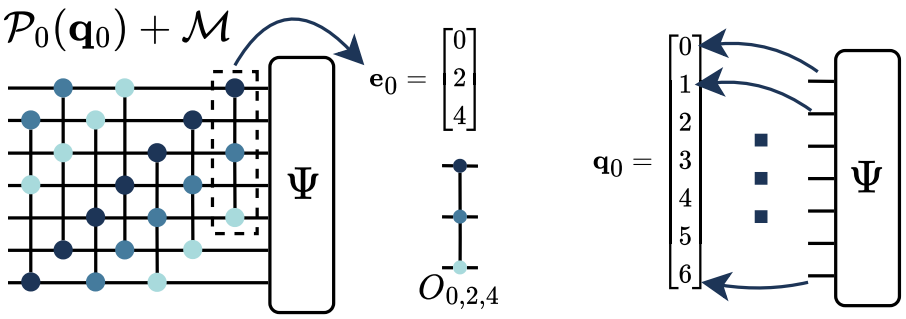


Figure 2.15: An example of how a hypergraph created by the AHL corresponds to a tensor network. On the right, the diagrammatic representation of a generic quantum state on a one-dimensional lattice with $N = 7$ sites. Each site is associated with a node from the hypergraph. On the left, a hypergraph created by the motif $\mathcal{M}$. If we associate each edge in the hypergraph with an operator O_{n_0,n_1,n_2}, then it encodes a tensor network.

in Section 2.5, the associated operator will be the one provided to the mapping of the primitive at the deepest level of the hierarchy. This is analogous to how operators are specified in the python package.

In Eq. (2.90), the tensor network is built from the bottom up. This does not necessarily determine the order in which tensor contraction will be performed when computing quantities. Practically, the python package [1] allows us to build the network as in Eq. (2.90) to obtain a tensor network object. The resulting object is then manipulated with a tensor network package, like quimb [122], that can find efficient contraction paths for computing quantities.

In Fig. 2.15, I show how the hypergraph $\mathcal{G} = \mathcal{P}_0(\mathbf{q}_0) + \mathcal{M}$ corresponds to a tensor network. On the right, I show the initial tensor network, Ψ, representing a quantum state on a lattice with $N = 7$ sites. Each site is shown as an open leg of the tensor network, and each open leg is associated with a node from the hypergraph's sequence of nodes $\mathbf{q}_0$. On the left of Fig. 2.15, I show the updated tensor network. Here the motif $\mathcal{M}$ consists of one primitive, an arity $k = 3$ cycle with the operator O_{n_0,n_1,n_2} as a mapping. Every edge $\mathbf{e}_r$ in the connectivity matrix specifies which sites the operator is applied on. For example, the first edge $\mathbf{e}_0$ shown in Fig. 2.15 specifies that O_{n_0,n_1,n_2} acts on the sites zero, two and four. More generally, we can associate any rank $2k$ tensor with a primitive $\mathcal{M}[i]$ in a motif $\mathcal{M}$, then each arity-k edge $\mathbf{e}_r$ in the connectivity matrix $E^{(i)}$ specifies the connectivity of that tensor.

https://scholar.sun.ac.za

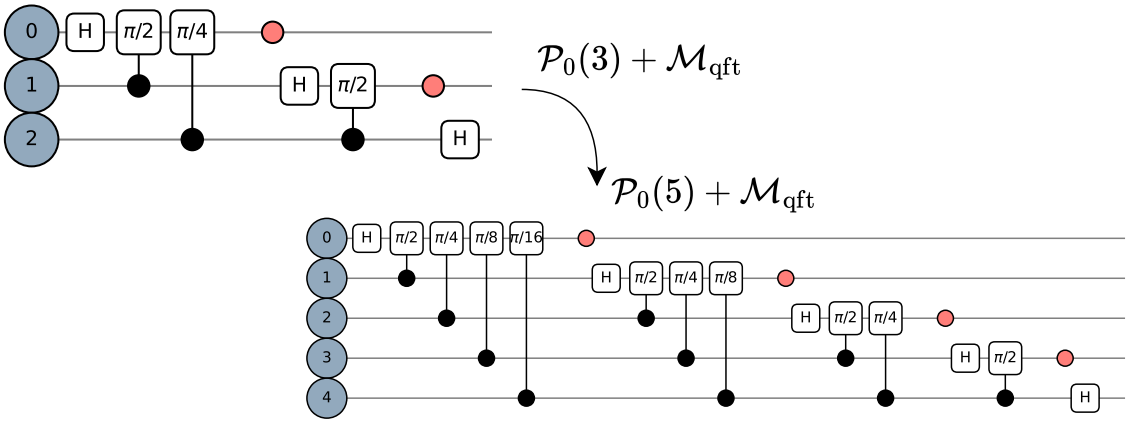


Figure 2.16: Illustration of how the motif $\mathcal{M}_{\text{qft}}$ implements the general N qubit quantum Fourier transform (QFT). The motif is instantiated on three and five qubits yielding the QFT circuit each time. This motif is shown in Code. 2.1

2.6.2 Quantum circuits

The quantum Fourier transform

```
qft = (
    Qpivot(H,"1*")
    + Qpivot(CP(ϕ), "1*")
    + Qmask("1*")
) * N
```

Code 2.1: The quantum Fourier transform on N qubits expressed as a single motif.

After Eq. (2.90), I mentioned that if the associated operators for each primitive are all unitary, then the motif can be regarded as an abstract quantum circuit. In Code. 2.1 I show such an example, here the quantum Fourier transform for N qubits is represented as the motif $\mathcal{M}_{\text{qft}}$. I am using a shorthand for the primitives, similar to how they are used in the python package [1]. For the pivot, the first argument is its mapping, which in this case is the Hadamard and controlled-phase gates. The second argument is the base pattern b^p parameter. For the mask, the only argument is its base pattern b^p, and its associated operator (mapping) is the $k = 1$ identity I. The remaining parameters take on their default values as specified in Tables 2.2 and 2.3. The Hadamard and controlled-phase gates are

https://scholar.sun.ac.za

defined as

$$\mathrm{H}\,|z,\pm\rangle \equiv \frac{1}{\sqrt{2}}\Big(|z,+\rangle \pm |z,-\rangle\Big), \tag{2.91}$$

$$\mathrm{CP}(\phi)\,|z,\sigma,\sigma'\rangle \equiv \begin{cases} e^{i\phi}\,|z,\sigma,\sigma'\rangle & \text{if } \sigma=\sigma'=-, \\ |z,\sigma,\sigma'\rangle & \text{otherwise,} \end{cases} \tag{2.92}$$

where $|z,\pm\rangle$ are the eigenstates of the Pauli-Z matrix with eigenvalues ± 1. Here, the variational parameter ϕ has a functional relationship with the edge index r for each edge $\mathbf{e}_r$ in the connectivity matrix $E^{(i)}$. This relationship is given by $\phi = 2^{-(r+1)}\pi$, which is easy to specify within the python package. In Fig. 2.16, I show an example of the $N = 3$ and $N = 5$ instantiations of the QFT motif. This example illustrates how the AHL captures the abstract structure of a quantum algorithm: the motif $\mathcal{M}_{\text{qft}}$ fully encodes the N-qubit implementation of the quantum Fourier transform. This example also shows how this structure can be expressed compactly, in just a few lines.

Hierarchical quantum circuits

Here I show how the AHL is used to express two popular quantum circuit ansatzes, the quantum convolutional neural network from Cong et al. [5] and the hierarchical quantum circuits from Grant et al. [123]. These ansatzes are used as parameterised quantum circuit models in quantum machine learning. In Fig. 2.17(c), I show the original QCNN on $N = 15$ qubits. The unitary operators U employ generalised Gell-Mann matrices parameterised based on the number of qubits k they act upon. The line, $\text{Qinit}(15) + m_2 + m_3 + m_4 + m_5$, creates the circuit on 15 qubits, applying the same architecture to N is detailed below with Alg. 1. To introduce a depth d parameter to the circuit, the last line should be modified to $(m_2 + m_3 + m_4 + m_5) \times d$. In Fig. 2.17(d), I show a 16-qubit MERA circuit. For an 8-qubit MERA circuit, the last line would be changed to $\text{Qinit}(8) + m_1^1 + m_2^1 \times 2$, and in general, $\text{Qinit}(N) + m_1^1 + m_2^1 \times \log_2(N-1)$ produces an N-qubit MERA circuit. These examples highlight that general N-qubit architectures can be captured with a few lines of code in a modular manner, and scaling up to larger systems is accomplished with minimal adjustments.

In Alg. 1, I provide the general motif that captures the QCNN described in Ref. [5] for their learning procedure. The QCNN on N qubits is characterised by a single integer n determining the reduction of system size. Starting at M_1, the permute primitive generates 2-qubit unitaries between all pairwise combinations of $n+1$ qubits. Subsequently, the second primitive stored in M_2 takes M_1 as its mapping, causing it to treat M_1 as a single $n+1$ qubit unitary. The cycle in M_2 has a stride

https://scholar.sun.ac.za

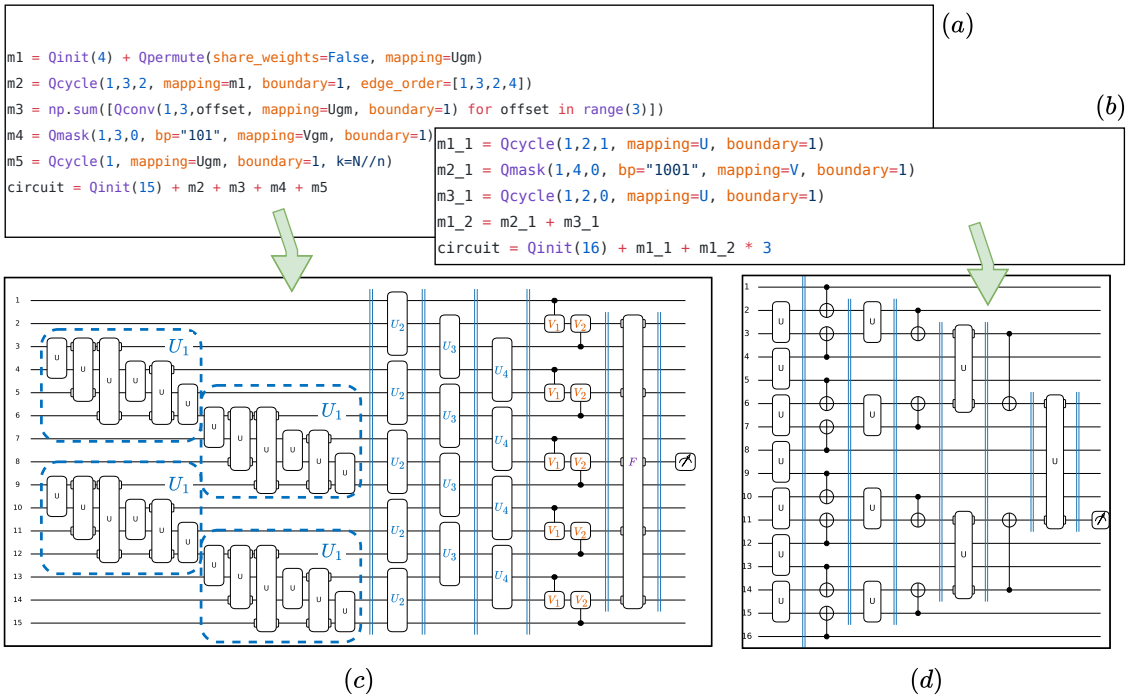


Figure 2.17: Example quantum circuits from Cong et al. [5] (c) and Grant et al. [123] (d), created using our python package.

of one, a step of n, and an offset of $n-1$. The U_1 blocks in Fig. 2.17 show the M_2 cycle on $N = 15$ qubits and with $n = 3$. In Alg. 1, this is followed by n cycles, stored in M_3, each associated with n-qubit unitaries and having an incremented offset. The variable M_4 in Alg. 1 contains a mask with arity $k = n$ and a base pattern $b^p = \texttt{!0!}$ that masks the outer $n-1$ qubits from each $(n-1)^{th}$ qubit. Finally, M_5 contains a cycle with arity equal to the number of remaining qubits. There are many choices of associating unitary operators with each of these primitives. The unitary operators from the original QCNN are based on $2^k \times 2^k$ Gell-Mann matrices, where k indicates the number of qubits the operator acts upon. Here, k corresponds to the arity of each primitive. For instance, in Alg. 1 the variable M_2 contains an arity $k = n+1$ cycle which would be associated with $2^{n+1} \times 2^{n+1}$ Gell-Mann matrices.

https://scholar.sun.ac.za

Algorithm 1 Original QCNN from [5].

Require: $N, n,$

Ensure: Motif$\rightarrow \mathcal{M}_{\text{qcnn}}$

$M_1 \leftarrow \text{Qinit}(n+1) + \text{Qpermute}(k=2)$
$M_2 \leftarrow \text{Qcycle}(h_0 = n-1, h_1 = 1, h_2 = n, \delta_b = 1, \text{mapping} = M_1)$
$M_3 \leftarrow \sum_{i=0}^{n-1} \text{Qcycle}(h_0 = i, h_1 = 1, h_2 = n, \delta_b = 1, k = n)$
$M_4 \leftarrow \text{Qmask}(b^p = \texttt{!0!}, k = n, h_2^b = 2)$
$M_5 \leftarrow \text{Qcycle}(k = \lfloor \frac{N}{n} \rfloor, \delta_b = 1)$
$M_6 \leftarrow \text{Qinit}(N) + \sum_{i=1}^{5} M_i$
$\triangleright$ Motif: repeat d (depth) times
$\mathcal{M}_{\text{qcnn}} \leftarrow M_6 \times d$

https://scholar.sun.ac.za

Chapter 3

Ground-State Structure Search

The abstract hypergraph language (AHL) presented in Chapter 2 can be used to generate ansatzes for quantum many-body states which are both analytically tractable and accurate over wide parameter regimes. Within this context, the abstract hypergraphs will be used to describe tensor networks as discussed in Section 2.6.1 of Chapter 2. I will use the terms motif, tensor network, and ansatz interchangeably. Similarly, the terms instantiated motif and individual (in the context of evolutionary algorithms) will also be used interchangeably. The next three chapters will be concerned with this topic, which is based on our work from Ref. [3]. In Ref. [3] we introduced a method which blends ideas from variational many-body physics, tensor network theory and algorithmic architecture search to design ground-state ansatzes. We showed that a compact, interpretable ansatz can be discovered and written down in closed form by a computer-driven search. Our approach leverages the AHL to construct tensor network states (TNS) via an evolutionary algorithm. Since the motifs automatically scale with system size, we can evaluate ansatz fitness for small systems, which is computationally efficient, while favouring structures that continue to perform well with increasing system size. This ensures that the ansatz captures robust features of the ground-state structure. Remarkably, we find analytically tractable ansatzes with a degree of universality, which encode correlations, capture finite-size effects, accurately predict ground-state energies, and offer a good description of critical phenomena. We demonstrate this method on the Lipkin-Meshkov-Glick model (LMG) and the quantum transverse-field Ising model (TFIM), where the same ansatz was independently generated for both. The simple structure of the ansatz allows us to obtain exact expressions for the expectation values of local observables as well as for correlation functions. In addition, it permits symmetries that are broken in the ansatz to be restored, which provides a systematic means of improving the accuracy of the ansatz.

https://scholar.sun.ac.za

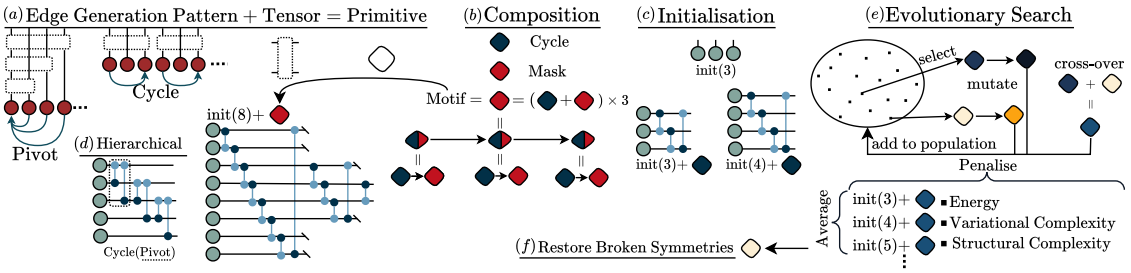


Figure 3.1: An overview of our method. The abstract hypergraph language enables ansatz generation via an evolutionary algorithm. (a) A primitive (*cycle*, *mask* or *pivot*) is an edge generation pattern associated with a tensor. (b) Composition: Sequences of primitives form motifs; sequences of motifs form higher-level motifs. (c) A primitive applied to an initialised state (specified with the *init* command) forms a tensor network. (d) A specified network, being itself a tensor, can again be associated with an edge generation pattern to form a new primitive. (e) The evolutionary algorithm mutates and crosses over motifs each generation. (f) Once the ansatz is found, broken symmetries can be restored.

3.1 Search strategy

The elements of this method are outlined in Fig. 3.1. Within the AHL the elementary building blocks are primitives and we associate a tensor with each one (a) to build tensor networks. The primitives can be composed sequentially (b) to create a sequence of primitives called a *motif*. In turn, multiple motifs can be composed to form sequences of motifs, and so on. Motifs are built without specifying a system size, and size-scaling is captured within the properties of primitives. System size is specified through the *init* command, which initialises the state with a tensor (c). When a motif is applied to an initialised state, a tensor network is formed. Since a tensor network is itself a tensor, it can again be associated with a primitive, allowing larger networks to be built hierarchically from sub-networks (d). Our evolutionary algorithm (e) follows the design of those in Neural Architecture Search [121, 124] which is based on tournament selection [125]. The optimal ansatz produced by this algorithm will generally not exhibit the same symmetries as the system Hamiltonian. However, with sufficient complexity penalties the resulting structure will be simple enough for broken symmetries to be restored (f). The algorithm starts with a randomly initialised pool of motifs each containing a single primitive. The tensors associated with each of these primitives are chosen from a fixed set and contain variational parameters. A tournament is then repeatedly held between a set fraction $\rho \in [0, 1]$ of the total population. Here ρ denotes the selection pressure, with large values facilitating exploitation. During each tournament, the winners (defined below) are used

https://scholar.sun.ac.za

to generate offspring through mutation, cross-over and other genetic operations using the AHL. The winners are typically the two fittest motifs, but there is a probability $\epsilon \in [0, 1]$ that two random motifs are declared winners instead. Large values of ϵ therefore facilitate a wider exploration of possible network structures. We evaluate the fitness $f(\psi_a)$ of motif ψ_a over system sizes $n \in \mathcal{N}$ with penalties on variational and structural complexity according to

$$f(\psi_a) \equiv \sum_{n\in\mathcal{N}} (E(\psi_a^{(n)}) + l_1 S(\psi_a^{(n)}) + l_2 V(\psi_a^{(n)}))w_n. \tag{3.1}$$

Here E, S, and V are respectively the per-site energy expectation value, the structural complexity, and variational complexity of an instantiated motif (individual) $\psi_a^{(n)}$. The parameters l_1 and l_2 control the penalties associated with these two complexities, while $w_n \in [0, 1]$ are normalised weights assigned to the various system sizes in $\mathcal{N}$. The variational complexity $V(\psi_a^{(n)})$ of an individual is the number of variational parameters it contains. The structural complexity $S(\psi_a^{(n)})$ is the sum of the ranks of the tensors that make up the individual, divided by $2n$. During the search, a main worker process maintains a queue of unevaluated motifs, of which the fitnesses have not yet been calculated, together with a pool of evaluated ones. Each tournament enqueues unique motifs while duplicates only increase the multiplicities of ones already in the pool. We define one evolutionary step as the addition of 10 unique evaluated motifs to the pool.

3.2 Genetic operations

After each tournament, the two winning motifs $\mathcal{M}_a$ and $\mathcal{M}_b$ are combined into eight motifs using various genetic operations. Some of the resulting motifs may already be in the pool, since the motif can be produced through multiple evolutionary paths. The genetic operations we consider are listed in Tab. 3.1. Let $\mathcal{M}_a$ be the motif with greater or equal fitness than $\mathcal{M}_b$, except when the winners are declared randomly. During a tournament, the `chop_and_join` operation is applied to both $\mathcal{M}_a$ and $\mathcal{M}_b$, producing two motifs. The `mutation`, `insert`, `cycle_mapping`, `mutate_step_up` and `dropout` operations are applied to motif $\mathcal{M}_a$, producing five motifs. Finally, the `mutation` operation is applied to motif $\mathcal{M}_b$, producing one motif. In total, eight motifs are produced. To favour diversity, we do not remove any motifs from the pool during evolution.

https://scholar.sun.ac.za

Operation	Input	Output	Brief description
`chop_and_join`	$\mathcal{M}, \mathcal{M}'$	$\mathcal{M}_{cj_1}, \mathcal{M}_{cj_2}$	Randomly cut each of $\mathcal{M}$ and $\mathcal{M}'$ into a head and a tail. Then swap the tails between the two, producing two new motifs.
`insert`	$\mathcal{M}$	$\mathcal{M}'$	Insert a random primitive at a random index of the input motif.
`mutate`	$\mathcal{M}$	$\mathcal{M}'$	Select a random primitive from the input motif and change one of its parameter values. This includes the possibility of changing its associated tensor.
`mutate_step_up`	$\mathcal{M}$	$\mathcal{M}'$	Create a random primitive and give it a mapping that is the input motif.
`cycle_mapping`	$\mathcal{M}$	$\mathcal{M}'$	Select a random primitive from the input motif and use its mapping as the mapping for a random cycle.
`dropout`	$\mathcal{M}$	$\mathcal{M}'$	Remove a random primitive from the input motif.

Table 3.1: Genetic operations used to generate new motifs.

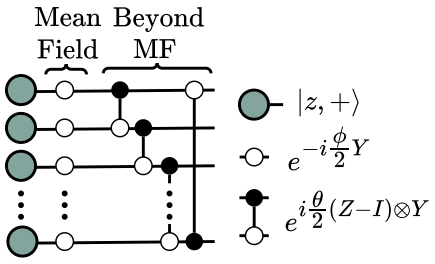


Figure 3.2: The ansatz generated by our method for the LMG and TFIM models.

3.3 The discovered ansatz

We demonstrate our method on the Lipkin-Meshkov-Glick (LMG) model [126–128] and the quantum transverse-field Ising model (TFIM) [129]. Remarkably, the search yields the same ansatz for both models, showcasing its ability to identify network structures with some degree of universality. For the LMG model it yields highly accurate results for finite systems, far surpassing that of a mean-field treatment, and which become exact in the thermodynamic limit. For the TFIM case, we obtain accurate results across all system sizes and greatly improve

https://scholar.sun.ac.za

upon the mean-field treatment in the thermodynamic limit. Only small system sizes ($\mathcal{N} = \{3, 4, 5\}$) were required and the search takes about 6 CPU hours to find the ansatz. Further details regarding the computational cost, robustness and reliability of the search are provided in Sections 3.4, 3.5 and 3.6. In Chapter 4 we derive analytic expressions for energies, local observables, correlation functions and restore the symmetries in the ansatz that the search initially broke. Then in Chapter 5 we present the predictions of the ansatz for both models. We also show that the algorithm autonomously constructs a mean-field treatment and extends it to incorporate correlations. This provided us with a simple and interpretable structure. The search yielded the following motif

$$\mathcal{M}_\psi = \mathcal{P}_{\text{mf}} + \mathcal{P}_{\text{corr}}, \tag{3.2}$$

where $\mathcal{P}_{\text{mf}}$ is an arity $k = 1$ cycle with $h_0 = 0, h_1 = 1, h_2 = 1$ with associated tensor $e^{-i\frac{\theta}{2}Y}$. This primitive implements the mean-field solution for both models. The second primitive $\mathcal{P}_{\text{corr}}$ introduces correlations beyond the mean-field level. It is an arity $k = 2$ cycle also with $h_0 = 0, h_1 = 1, h_2 = 1, \delta_b = 0$ and associated tensor $e^{i\frac{\theta}{2}(Z-I)\otimes Y}$. The ansatz is shown in Fig. 3.2. For N spins it produces the state

$$|\psi_o\rangle = \left(\prod_{k=0}^{\overleftarrow{N-1}} C^{\theta}_{k,k+1} R^{\theta}_{k+1}\right)\left(\prod_{j=0}^{\overleftarrow{N-1}} R^{\phi}_{j}\right)|z,+\rangle^{\otimes N}, \tag{3.3}$$

where ϕ and θ are variational parameters and $|z,\pm\rangle$ are the eigenstates of the Pauli-Z matrix with eigenvalues ± 1. The operator products in Eq. (3.3) are ordered with the k and j indices increasing from right to left. The two unitary operators appearing in $|\psi_o\rangle$ are

$$C^{\theta}_{ij} = e^{i\frac{\theta}{2}Z_i Y_j} \qquad \text{and} \qquad R^{\theta}_{j} = e^{-i\frac{\theta}{2}Y_j}. \tag{3.4}$$

Here (X_i, Y_i, Z_i) are the Pauli spin matrices associated with the spin at site $i \in \{0, \ldots, N-1\}$, obeying periodic boundary conditions: $i + N \equiv i$. As shown in Chapter 4, it is possible to obtain an exact representation of $|\psi_o\rangle$ as an MPS. In this form the ansatz reads

$$|\psi_o\rangle = 2^{-N/2} \sum_{\vec{\sigma}} \text{Tr}(B^{\sigma_0} A^{\sigma_1} \cdots A^{\sigma_{N-1}}) \, |y, \vec{\sigma}\rangle \,, \tag{3.5}$$

https://scholar.sun.ac.za

where $|y, \sigma_i\rangle$ are the eigenstates of the Pauli-Y_i matrix with eigenvalues $\sigma_i = \pm 1$, and

$$A^+ = \begin{bmatrix} \cos\frac{\theta}{2} & \cos\frac{\theta}{2} \\ i\sin\frac{\theta}{2} & -i\sin\frac{\theta}{2} \end{bmatrix} \begin{bmatrix} e^{-\frac{i(\theta+\phi)}{2}} & 0 \\ 0 & e^{\frac{i(\theta+\phi)}{2}} \end{bmatrix}, \tag{3.6}$$

$$B^+ = \begin{bmatrix} \cos\frac{\theta}{2} & \cos\frac{\theta}{2} \\ i\sin\frac{\theta}{2} & i\sin\frac{\theta}{2} \end{bmatrix} \begin{bmatrix} e^{-\frac{i(\theta+\phi)}{2}} & 0 \\ 0 & e^{-\frac{i(\theta-\phi)}{2}} \end{bmatrix}, \tag{3.7}$$

$$A^- = (A^+)^*, \quad B^- = (B^+)^*. \tag{3.8}$$

https://scholar.sun.ac.za

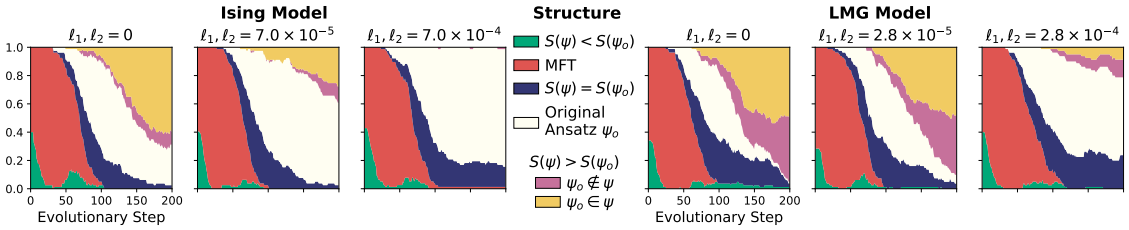


Figure 3.3: The density of particular structural classes of discovered ansatzes at different evolutionary steps. Each subplot shows results for a different complexity penalty pair (l_1, l_2)(increasing to the right). At each step (vertical slice) the lengths of intervals of a particular colour indicate the fraction of runs which yielded a fittest motif within a specific class. The classes are determined by the structural complexity $S(\psi)$ of the fittest motif ψ. White represents the original ansatz ψ_o in Eq. (3.3). Green and red denote ansatzes with structural complexity less than the original, with red being the mean-field solution. Blue indicate ansatzes with structural complexity equal to the original. Pink and yellow are more complex: yellow contains the original ansatz as a building block, while pink does not.

3.4 Robustness of the search

Here we demonstrate the robustness and reliability of the method. We ran 480 experiments sweeping over different complexity penalties for both models and report on the prominence of the original ansatz in Fig. 3.2. The results are shown in Fig. 3.3. Each subplot contains data from $79 \pm (3)$ runs where the selection pressure and exploration parameters are fixed at $\rho = 0.01$ and $\epsilon = 0.33$, and the fitness is evaluated for the set of system sizes $\mathcal{N} = \{5, 6, 7\}$. The fittest ansatz at each evolutionary step is classified into one of six classes indicated by colour. The classes are based on the structural complexity of that ansatz $S(\psi)$ relative to that of the original ansatz $S(\psi_o)$: green and red denote lower complexity, blue and white equal complexity, and pink and yellow higher complexity. Between the mean-field (red) and original ansatz (white) there are competing local minima (blue) with exactly the same structural complexity as the original, namely $S(\psi_o^{(n)}) = 3$ for all $n \in \mathcal{N}$ but with lower fitness. Most prominent in this class was the ansatz

$$|\psi_x\rangle = \left(\overleftarrow{\prod_{k=0}^{N-1}} R_k^{\phi}\right)\left(\overleftarrow{\prod_{j=0}^{N-1}} e^{-i\frac{\theta}{2}X_jY_{j+1}}\right)|z,+\rangle^{\otimes N}, \tag{3.9}$$

with $j+N \equiv j$. We see that for the largest penalties l_1 and l_2, the fittest ansatz after 200 steps was the original ansatz in 83% and 55% of runs for the Ising model and

https://scholar.sun.ac.za

LMG model respectively. As we decrease the complexity penalties more complex structures begin to emerge, shown in yellow and pink. These structures have $S(\psi) > 3$, and are distinguished by whether they contain the original ansatz as a building block (yellow) or not (pink). The prevalence of the former yellow class for lower complexity penalties shows that the original ansatz is a robust structure that functions as a building block for more complex ansatzes. Fig. 3.3 also shows that the effect of the complexity penalties is model-dependent. With decreasing complexity penalties the LMG model is dominated by complex structures more readily than the Ising model. This results from the smaller energy difference found between different network structures for the LMG model. Still, the original ansatz (white) is a prominent beyond-mean-field structure in all cases.

3.5 Choice of operator basis

The original ansatz from Eq. (3.3) is represented as a product of unitary operators, meaning it is equivalent to a quantum circuit. This is because the initial operator pool was generated from the Pauli-matrices $\sigma \in \{I, X, Y, Z\}$ in the form $e^{i\phi\sigma_i}, e^{i\theta\sigma_i\otimes\sigma_j}$ where $\theta, \phi \in \mathbb{R}$. We can change the operator pool to include non-unitary tensors to find more general tensor networks. For example, using an operator basis that contains ladder operators, i.e. $\{I, \sigma^+, \sigma^-, Z\}$ our method finds another efficient ansatz shown in Fig. 3.4. With this operator basis, the search was

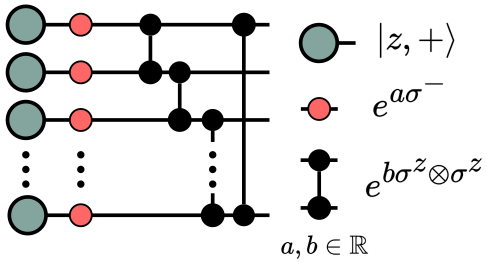


Figure 3.4: Ansatz discovered with the ladder-operator basis $\{I, \sigma^+, \sigma^-, Z\}$. The search returns a translationally invariant, two-parameter periodic MPS Eq. (3.10), and it attains the same energy expectation values as the original ansatz in Fig. 3.2.

able to obtain a translationally invariant ansatz. Numerically, this ansatz yields the same energy expectation values as the original ansatz in Fig. 3.2 for all system

https://scholar.sun.ac.za

sizes. We can express this ansatz as the following two-parameter periodic MPS

$$|\psi_l\rangle = \frac{1}{M_l} \sum_{\vec{\sigma}} \mathrm{Tr}\left(C^{\sigma_0} C^{\sigma_1} \cdots C^{\sigma_{N-1}}\right) |z, \vec{\sigma}\rangle \tag{3.10}$$

where M_l is a normalisation factor, and the matrices $C^{\pm}$ are given by

$$C^{\pm} = a^{\frac{1\mp 1}{2}} \begin{bmatrix} \cosh(b) & \pm\cosh(b) \\ \pm\sinh(b) & \sinh(b) \end{bmatrix}, \tag{3.11}$$

with $a, b \in \mathbb{R}$. Expectation values may be obtained in the same fashion as detailed in Chapter 4. The choice of operator basis will depend on the context of the problem. For example, if the goal is to obtain short-depth quantum circuits for state preparation on a specific quantum device, then using native gates for that hardware will be beneficial.

3.6 Computational cost

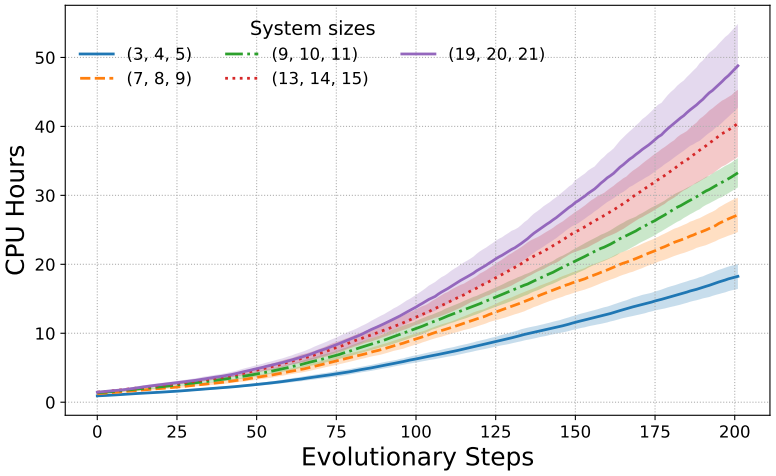


Figure 3.5: Runtime of the evolutionary search for the Ising model. We consider five system–size groups, each averaged over 15 runs (75 runs total), using complexity penalties $l_1 = l_2 = 7\times10^{-4}$ and selection/exploration parameters $\rho = 0.01$ and $\epsilon = 0.33$.

The main bottleneck in the evaluation of individuals during the evolutionary search is memory, as the bond dimension between two sites grows exponentially with the number of tensors that connect them. Increasing the complexity

https://scholar.sun.ac.za

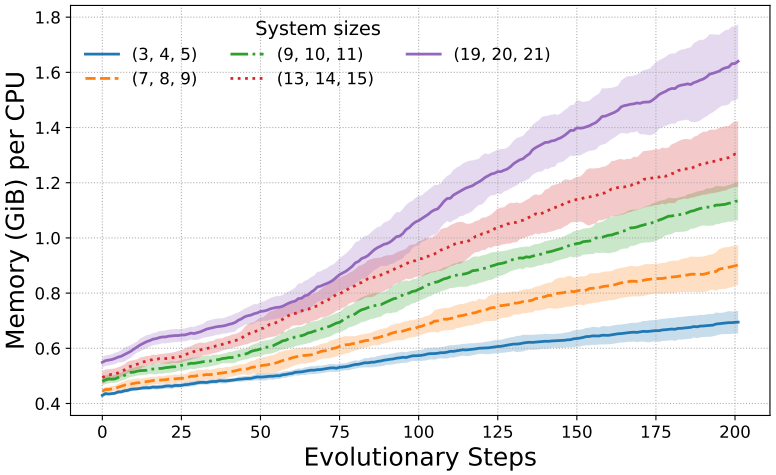


Figure 3.6: Per-CPU memory usage during evolutionary search. Fitness evaluations are run in parallel, so each CPU corresponds to one worker. Memory therefore depends on both the number of workers and the complexity penalty.

penalty limits the exploration of overly large network structures and, in turn, allows larger system sizes to be considered. Because the AHL automatically scales over system sizes, it can explore structures that, in principle, extend well beyond what we can evaluate explicitly for large systems. Figs. 3.5 and 3.6 report the computational cost of our method over 75 searches for an Ising-model ansatz, organised into five size groups and averaged over 15 runs each. We used $l_1 = l_2 = 7 \times 10^{-4}$, $\rho = 0.01$, and $\epsilon = 0.33$ for all runs. In 65 of the 75 cases (86%), the fittest individual at step 200 was the original ansatz, close to the 83% ratio observed at step 200 for system sizes $n \in \{5, 6, 7\}$ in Fig. 3.3 with the same penalty. This suggests that the converged ansatz is largely insensitive to the system sizes considered. We evaluated system sizes up to $n = 21$ spins, enabled by the complexity penalty. From Fig. 3.3, beyond-mean-field structures begin to dominate around 100 steps; in Fig. 3.5 this corresponds to 6.27 CPU-hours on average for the smallest group, $n \in \{3, 4, 5\}$, and about 13.8 CPU-hours for the largest, $n \in \{19, 20, 21\}$. Fig. 3.6 shows the per-worker memory footprint. For 100 evolutionary steps and $n \in \{3, 4, 5\}$ we required less than $0.57\,\text{GiB}$ per worker on average, with each worker occupying one CPU core. For 100 steps and $n \in \{19, 20, 21\}$ each worker required less than $1.06\,\text{GiB}$ on average.

After finding an ansatz with the search algorithm, we optimise its variational parameters for each system size. This optimisation may not always be required, for example with the Ising model the optimal angles essentially become constant once $N > 20$. Therefore, re-optimisation for larger system sizes is not required.

https://scholar.sun.ac.za

For the LMG model, the situation is different. For the ferromagnetic phase, numerical results suggest that the leading-order finite-size corrections scale like $1/N$. Using only system sizes $N \in \{50, 100, 150\}$, we can extrapolate to arbitrary system sizes without losing accuracy. For the paramagnetic phase, extrapolation to the thermodynamic limit is no longer reliable. The symmetrised ansatz produces essentially exact results in this phase, which might cause the optimal parameter values to depend more sensitively on the system size.

https://scholar.sun.ac.za

Chapter 4

Analysis of a Universal Ansatz

4.1 Matrix product state representation

Here we show how the MPS form of our ansatz in Eq. (3.5) can be derived from its original form in Eq. (3.3). Recall that the ansatz generated by our evolutionary algorithm produces, for N spins, the state

$$|\psi_o\rangle = \left(\prod_{k=0}^{\overleftarrow{N-1}} C^{\theta}_{k,k+1} R^{\theta}_{k+1}\right)\left(\prod_{j=0}^{\overleftarrow{N-1}} R^{\phi}_{j}\right)|z,+\rangle^{\otimes N}, \tag{4.1}$$

where ϕ and θ are variational parameters and $|z,\pm\rangle$ are the eigenstates of the Pauli-Z matrix with eigenvalues ± 1. The two unitary operators appearing in $|\psi_o\rangle$ are

$$C^{\theta}_{ij} = e^{i\frac{\theta}{2} Z_i Y_j} \qquad \text{and} \qquad R^{\theta}_{j} = e^{-i\frac{\theta}{2} Y_j}. \tag{4.2}$$

All site indices are treated as periodic, i.e. $i \equiv i + N$. The operator products in Eq. (4.1) are ordered with the k and j indices increasing from right to left. For example, if $N = 3$ the state reads

$$|\psi_o\rangle = C^{\theta}_{2,0} R^{\theta}_{0} C^{\theta}_{1,2} R^{\theta}_{2} C^{\theta}_{0,1} R^{\theta}_{1} R^{\phi}_{2} R^{\phi}_{1} R^{\phi}_{0} \, |z,+\rangle_0 \otimes |z,+\rangle_1 \otimes |z,+\rangle_2 . \tag{4.3}$$

It will be convenient to perform this derivation in the Y basis due to the multiple Pauli-Y operators appearing in Eq. (4.1). We therefore introduce

$$|\sigma_k\rangle \equiv |y,\sigma_k\rangle , \tag{4.4}$$

with $\sigma_k = \pm$ to denote the eigenstates, with eigenvalues ± 1, of the Pauli-Y matrix associated with the spin at site k. The initial product state from Eq. (4.1) can be

https://scholar.sun.ac.za

expressed in this basis as

$$|z,+\rangle^{\otimes N} = 2^{-N/2}\sum_{\vec{\sigma}}|\vec{\sigma}\rangle\,. \tag{4.5}$$

We will recast Eq. (4.1) in MPS form by successively applying the operator blocks $C^{\theta}_{k,k+1}R^{\theta+\phi}_{k+1}$ to the initial product state $|z,+\rangle^{\otimes N}$. Let

$$U(k_0) = \left(\prod_{k=k_0}^{\overleftarrow{N-1}} C^{\theta}_{k,k+1}R^{\theta+\phi}_{k+1}\right) \tag{4.6}$$

denote the final $N-k_0$ blocks to be applied. Since C^{θ}_{ij} commutes with R^{ϕ}_{k} when $i \neq k$, the form of $|\psi_o\rangle$ in Eq. (4.1) can be written as

$$|\psi_o\rangle = R_0^{-\phi}U(0)R_0^{\phi}\,|z,+\rangle^{\otimes N} = 2^{-N/2}R_0^{-\phi}U(0)R_0^{\phi}\sum_{\vec{\sigma}}|\vec{\sigma}\rangle\,. \tag{4.7}$$

The action of the two operators in Eq. (4.2) on the $|\sigma\rangle$ states are

$$C^{\theta}_{ij}\,|\sigma_i\sigma_j\rangle = c\,|\sigma_i\sigma_j\rangle + is\sigma_j\,|-\sigma_i\sigma_j\rangle \qquad \text{and} \qquad R^{\theta}_{j}\,|\sigma_j\rangle = e^{-i\frac{\theta}{2}\sigma_j}\,|\sigma_j\rangle\,, \tag{4.8}$$

where

$$c \equiv \cos\!\left(\frac{\theta}{2}\right) \qquad \text{and} \qquad s \equiv \sin\!\left(\frac{\theta}{2}\right). \tag{4.9}$$

Using the actions above, we begin to rewrite $|\psi_o\rangle$ by applying R_0^{ϕ} together with the first (rightmost) block of operators $C^{\theta}_{01}R_1^{\theta+\phi}$ in $U(0)$ to the initial state:

$$|\psi_o\rangle = 2^{-N/2}R_0^{-\phi}U(0)R_0^{\phi}\sum_{\vec{\sigma}}|\vec{\sigma}\rangle \tag{4.10}$$

$$= 2^{-N/2}R_0^{-\phi}U(1)\sum_{\vec{\sigma}}C^{\theta}_{01}R_1^{\theta+\phi}R_0^{\phi}\,|\vec{\sigma}\rangle \tag{4.11}$$

$$= 2^{-N/2}R_0^{-\phi}U(1)\sum_{\vec{\sigma}}e^{-i\frac{(\theta+\phi)}{2}\sigma_1}e^{-i\frac{\phi}{2}\sigma_0}\left(c\,|\sigma_0\sigma_1\rangle + is\sigma_1\,|-\sigma_0\sigma_1\rangle\right)\otimes|\vec{\sigma}'\rangle \tag{4.12}$$

$$= 2^{-N/2}R_0^{-\phi}U(1)\sum_{\vec{\sigma}}e^{-i\frac{(\theta+\phi)}{2}\sigma_1}\left(ce^{-i\frac{\phi}{2}\sigma_0} + is\sigma_1e^{i\frac{\phi}{2}\sigma_0}\right)|\vec{\sigma}\rangle\,. \tag{4.13}$$

Notice how we can write the amplitude inside the sum as an inner product of vectors as

$$e^{-i\frac{(\theta+\phi)}{2}\sigma_1}\left(ce^{-i\frac{\phi}{2}\sigma_0} + is\sigma_1e^{i\frac{\phi}{2}\sigma_0}\right) = \begin{bmatrix}e^{-i\frac{\phi}{2}\sigma_0} & e^{i\frac{\phi}{2}\sigma_0}\end{bmatrix}\begin{bmatrix}ce^{-i\frac{(\theta+\phi)}{2}\sigma_1}\\ is\sigma_1e^{-i\frac{(\theta+\phi)}{2}\sigma_1}\end{bmatrix} \tag{4.14}$$

$$\equiv \mathbf{b}(\sigma_0)^T\mathbf{a}(\sigma_1), \tag{4.15}$$

https://scholar.sun.ac.za

which also serves to define the vectors $\mathbf{a}(\sigma_i)$ and $\mathbf{b}(\sigma_i)$. After the application of the first operator block in $U(0)$, $|\psi_o\rangle$ therefore reads

$$|\psi_o\rangle = R_0^{-\phi} U(1) 2^{-N/2} \sum_{\vec{\sigma}} \mathbf{b}(\sigma_0)^T \mathbf{a}(\sigma_1) |\vec{\sigma}\rangle . \tag{4.16}$$

It follows via induction that after applying the rightmost k operator blocks from $U(0)$, the resulting form of $|\psi_o\rangle$ will be

$$|\psi_o\rangle = R_0^{-\phi} U(k) 2^{-N/2} \sum_{\vec{\sigma}} \mathbf{r}(\sigma_0, \dots, \sigma_{k-1})^T \mathbf{a}(\sigma_k) |\vec{\sigma}\rangle , \tag{4.17}$$

where $\mathbf{r}(\sigma_0, \dots \sigma_{k-1})^T$ is a row vector depending on the spin states $\sigma_0, \dots, \sigma_{k-1}$. To prove this claim and obtain an expression for $\mathbf{r}(\sigma_0, \dots, \sigma_{k-1})$, we start from Eq. (4.17) and write

$$|\psi_o\rangle = 2^{-N/2} R_0^{-\phi} U(k) \sum_{\vec{\sigma}} \mathbf{r}(\sigma_0, \dots, \sigma_{k-1})^T \mathbf{a}(\sigma_k) |\vec{\sigma}\rangle \tag{4.18}$$

$$= 2^{-N/2} R_0^{-\phi} U(k+1) \sum_{\vec{\sigma}} \mathbf{r}(\sigma_0, \dots, \sigma_{k-1})^T \mathbf{a}(\sigma_k) \left(C^{\theta}_{k,k+1} R^{\theta+\phi}_{k+1} |\vec{\sigma}\rangle \right) \tag{4.19}$$

$$= 2^{-N/2} R_0^{-\phi} U(k+1) \textstyle\sum_{\vec{\sigma}} \mathbf{r}(\sigma_0, \dots, \sigma_{k-1})^T \mathbf{a}(\sigma_k) \left(e^{-i\frac{(\theta+\phi)}{2}\sigma_{k+1}} \right) \left(c |\sigma_k \sigma_{k+1}\rangle + is\sigma_{k+1} |-\sigma_k \sigma_{k+1}\rangle \right) \otimes |\vec{\sigma}'\rangle \tag{4.20}$$

$$= 2^{-N/2} R_0^{-\phi} U(k+1) \sum_{\vec{\sigma}} \mathbf{r}(\sigma_0, \dots, \sigma_{k-1})^T \left[e^{-i\frac{(\theta+\phi)}{2}\sigma_{k+1}} (c\mathbf{a}(\sigma_k) + is\sigma_{k+1}\mathbf{a}(-\sigma_k)) \right] |\vec{\sigma}\rangle . \tag{4.21}$$

The factor in the square brackets in the final line results from the action of the k'th block $C^{\theta}_{k,k+1} R^{\theta+\phi}_{k+1}$, and can be expressed compactly as

$$e^{-i\frac{(\theta+\phi)}{2}\sigma_{k+1}} \left(c\mathbf{a}(\sigma_k) + is\sigma_{k+1}\mathbf{a}(-\sigma_k) \right) = \begin{bmatrix} \mathbf{a}(\sigma_k) & \mathbf{a}(-\sigma_k) \end{bmatrix} \begin{bmatrix} ce^{-i\frac{(\theta+\phi)}{2}\sigma_{k+1}} \\ is\sigma_{k+1} e^{-i\frac{(\theta+\phi)}{2}\sigma_{k+1}} \end{bmatrix} \tag{4.22}$$

$$\equiv A^{\sigma_k} \mathbf{a}(\sigma_{k+1}) \tag{4.23}$$

where we identified the vector $\mathbf{a}(\sigma_{k+1})$ from Eq. (4.15) and introduced the matrix

$$A^{\sigma_k} = \begin{bmatrix} \mathbf{a}(\sigma_k) & \mathbf{a}(-\sigma_k) \end{bmatrix} . \tag{4.24}$$

Explicitly, this becomes

$$A^{+} = \begin{bmatrix} c & c \\ is & -is \end{bmatrix} \begin{bmatrix} e^{-i(\theta+\phi)/2} & 0 \\ 0 & e^{i(\theta+\phi)/2} \end{bmatrix} , \tag{4.25}$$

$$A^{-} = \begin{bmatrix} c & c \\ -is & is \end{bmatrix} \begin{bmatrix} e^{i(\theta+\phi)/2} & 0 \\ 0 & e^{-i(\theta+\phi)/2} \end{bmatrix} . \tag{4.26}$$

https://scholar.sun.ac.za

These are the matrices A^+ and A^- from Eq. (3.6). Taken together, we have shown that

$$|\psi_o\rangle = 2^{-N/2} R_0^{-\phi} U(k) \sum_{\vec{\sigma}} \mathbf{r}(\sigma_0, \dots, \sigma_{k-1})^T \mathbf{a}(\sigma_k) |\vec{\sigma}\rangle \tag{4.27}$$

$$= 2^{-N/2} R_0^{-\phi} U(k+1) \sum_{\vec{\sigma}} \mathbf{r}(\sigma_0, \dots, \sigma_{k-1})^T A^{\sigma_k} \mathbf{a}(\sigma_{k+1}) |\vec{\sigma}\rangle \tag{4.28}$$

$$\equiv 2^{-N/2} R_0^{-\phi} U(k+1) \sum_{\vec{\sigma}} \mathbf{r}(\sigma_0, \dots, \sigma_k)^T \mathbf{a}(\sigma_{k+1}) |\vec{\sigma}\rangle \,. \tag{4.29}$$

This establishes the validity of the general form of $|\psi_o\rangle$ in Eq. (4.17) and yields the recurrence relation

$$\mathbf{r}(\sigma_0, \dots, \sigma_k)^T = \mathbf{r}(\sigma_0, \dots, \sigma_{k-1})^T A^{\sigma_k} \qquad \text{with} \qquad \mathbf{r}(\sigma_0)^T = \mathbf{b}(\sigma_0)^T, \tag{4.30}$$

which can be solved to produce

$$\mathbf{r}(\sigma_0, \dots, \sigma_k)^T = \mathbf{b}(\sigma_0)^T A^{\sigma_1} A^{\sigma_2} \cdots A^{\sigma_k}. \tag{4.31}$$

Choosing $k = N-1$ in Eq. (4.29) and using $U(N) = I$ and $\sigma_N = \sigma_0$ yields

$$|\psi_o\rangle = 2^{-N/2} \sum_{\vec{\sigma}} \mathbf{b}(\sigma_0)^T A^{\sigma_1} A^{\sigma_2} \cdots A^{\sigma_{N-1}} (e^{i\frac{\phi}{2}\sigma_0} \mathbf{a}(\sigma_0)) |\vec{\sigma}\rangle \,. \tag{4.32}$$

Finally, we express the expansion coefficients as traces, which leads to

$$|\psi_o\rangle = 2^{-N/2} \sum_{\vec{\sigma}} \mathrm{Tr}\Big(B^{\sigma_0} A^{\sigma_1} A^{\sigma_2} \cdots A^{\sigma_{N-1}} \Big) |\vec{\sigma}\rangle \,, \tag{4.33}$$

where

$$B^{\sigma_0} \equiv e^{i\frac{\phi}{2}} \mathbf{a}(\sigma_0) \mathbf{b}(\sigma_0)^T. \tag{4.34}$$

Written out, we have

$$B^+ = \begin{bmatrix} c & c \\ is & is \end{bmatrix} \begin{bmatrix} e^{-i(\theta+\phi)/2} & 0 \\ 0 & e^{-i(\theta-\phi)/2} \end{bmatrix}, \tag{4.35}$$

$$B^- = \begin{bmatrix} c & c \\ -is & -is \end{bmatrix} \begin{bmatrix} e^{i(\theta+\phi)/2} & 0 \\ 0 & e^{i(\theta-\phi)/2} \end{bmatrix}. \tag{4.36}$$

These are the matrices B^+ and B^- from Eq. (3.7).

https://scholar.sun.ac.za

4.2 Trace formulas for expectation values

We first demonstrate how the expectation values of one- and two-site observables with respect to a matrix product state of the form

$$|\psi_t\rangle = \frac{1}{M}\sum_{\vec{\sigma}} \mathrm{Tr}(A^{\sigma_0}A^{\sigma_1}\cdots A^{\sigma_{N-1}})\,|\vec{\sigma}\,\rangle \tag{4.37}$$

can be expressed as traces of transfer matrix products. Here $|\vec{\sigma}\,\rangle = |\sigma_0, \sigma_1, \ldots, \sigma_{N-1}\rangle$ with $\sigma_i \in \{+,-\}$ is a product state for N spin sites, $A^{\pm}$ are matrices, and M^{-1} is a normalisation factor. Let O be a single-site observable, acting on the state $|\sigma\rangle$ of the site as

$$O\,|\sigma\rangle = \langle +|O|\sigma\rangle\,|+\rangle + \langle -|O|\sigma\rangle\,|-\rangle\,. \tag{4.38}$$

If O_i denotes the observable O for site i, then its action on $|\psi_t\rangle$ is

$$O_i\,|\psi_t\rangle = \frac{1}{M}\sum_{\vec{\sigma}} \mathrm{Tr}(A^{\sigma_0}A^{\sigma_1}\cdots A^{\sigma_{N-1}})\left[\langle +|O|\sigma_i\rangle\,|+\rangle_i + \langle -|O|\sigma_i\rangle\,|-\rangle_i\right]\otimes|\vec{\sigma}\,'\rangle\,, \tag{4.39}$$

where $|\vec{\sigma}\,'\rangle$ excludes the state of site i. Taking the inner product with $|\psi_t\rangle$ then produces

$$\langle\psi_t|O_i|\psi_t\rangle = \tfrac{1}{M^2}\textstyle\sum_{\vec{\sigma}} \mathrm{Tr}\left(A^{\sigma_0}\cdots A^{\sigma_{i-1}}\left[\langle +|O|\sigma_i\rangle^* A^+ + \langle -|O|\sigma_i\rangle^* A^-\right]A^{\sigma_{i+1}}\cdots A^{\sigma_{N-1}}\right)^* \mathrm{Tr}(A^{\sigma_0}A^{\sigma_1}\cdots A^{\sigma_{N-1}}) \tag{4.40}$$

From here we use the fact that for matrices Q and R it holds that $\mathrm{Tr}(Q\otimes R) = \mathrm{Tr}(Q)\,\mathrm{Tr}(R)$ to combine the product of the two traces in the sum above, and then $(Q_1Q_2)\otimes(R_1R_2) = (Q_1\otimes R_1)(Q_2\otimes R_2)$ to group matrices associated with the same site together in Kronecker products. The sum over $\vec{\sigma}$ now factorises within the trace, and can be performed to produce

$$\langle\psi_t|O_i|\psi_t\rangle = \frac{1}{M^2}\,\mathrm{Tr}(T^iT_oT^{N-i-1}), \tag{4.41}$$

with the two transfer matrices given by

$$T = A^{+*}\otimes A^+ + A^{-*}\otimes A^- \qquad \text{and} \qquad T_o = \sum_{\sigma,\sigma'=\pm}\langle\sigma'|O|\sigma\rangle\,A^{\sigma'*}\otimes A^{\sigma}. \tag{4.42}$$

This result readily generalises to arbitrary multi-site observables. For example,

$$\langle\psi_t|O_iO_{i+r}|\psi_t\rangle = \frac{1}{M^2}\,\mathrm{Tr}(T^iT_oT^{r-1}T_oT^{N-i-r-1}). \tag{4.43}$$

https://scholar.sun.ac.za

4.3 Transfer matrices - definitions and diagonalisation

We will use the following compact notation for trigonometric functions of the two variational angles θ and ϕ that appear in our ansatz:

$$\begin{aligned}
&c \equiv \cos(\tfrac{\theta}{2}) && s \equiv \sin(\tfrac{\theta}{2}) && C \equiv \cos(\theta) && S \equiv \sin(\theta)\\
&d \equiv \cos(\tfrac{\theta+\phi}{2}) && t \equiv \sin(\tfrac{\theta+\phi}{2}) && D \equiv \cos(\theta+\phi) && T \equiv \sin(\theta+\phi)\\
&e \equiv \cos(\tfrac{\theta-\phi}{2}) && u \equiv \sin(\tfrac{\theta-\phi}{2}) && E \equiv \cos(\theta-\phi) && U \equiv \sin(\theta-\phi)\\
&f \equiv \cos(\tfrac{\phi}{2}) && v \equiv \sin(\tfrac{\phi}{2}) && F \equiv \cos(\phi) && V \equiv \sin(\phi)
\end{aligned} \tag{4.44}$$

For calculating expectation values it will be convenient to switch to the Z basis and rewrite the MPS form of the ansatz in Eq. (4.33) as a linear combination of Z_i eigenstates. The simple connection between the eigenstates of Y_i and Z_i leads to the form

$$|\psi_o\rangle = \sum_{\vec{\sigma}} \mathrm{Tr}(B_z^{\sigma_0} A_z^{\sigma_1} A_z^{\sigma_2} \cdots A_z^{\sigma_{N-1}}) \, |\vec{\sigma}\,\rangle \,, \tag{4.45}$$

where $|\sigma\rangle \equiv |z,\sigma\rangle$, $|\vec{\sigma}\,\rangle = |\sigma_0, \sigma_1, \sigma_2, ...\rangle$ satisfies $Z_i \, |\vec{\sigma}\,\rangle = \sigma_i \, |\vec{\sigma}\,\rangle$ with $\sigma_i \in \{+,-\}$. Here $A_z^{\pm}$ and $B_z^{\pm}$ are given in terms of the matrices $A^{\pm}$ and $B^{\pm}$ from Eqs. (4.25), (4.26), (4.35) and (4.36) by

$$\begin{bmatrix} A_z^+ \\ A_z^- \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ i & -i \end{bmatrix} \begin{bmatrix} A^+ \\ A^- \end{bmatrix}, \qquad \begin{bmatrix} B_z^+ \\ B_z^- \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ i & -i \end{bmatrix} \begin{bmatrix} B^+ \\ B^- \end{bmatrix}. \tag{4.46}$$

Explicitly, we have

$$A_z^+ = \begin{bmatrix} cd & cd \\ st & st \end{bmatrix}, \quad A_z^- = \begin{bmatrix} ct & -ct \\ -sd & sd \end{bmatrix}, \quad B_z^+ = \begin{bmatrix} cd & ce \\ st & su \end{bmatrix}, \quad B_z^- = \begin{bmatrix} ct & cu \\ -sd & -se \end{bmatrix}. \tag{4.47}$$

We also define the transfer matrix

$$T_a = A_z^{+*} \otimes A_z^+ + A_z^{-*} \otimes A_z^-, \tag{4.48}$$

which is associated with the sites at positions $i > 0$. Here A_z^{+*} and A_z^{-*} are the complex conjugates of A_z^+ and A_z^-. Similarly, the transfer matrix for the first site at $i = 0$ reads

$$T_b = B_z^{+*} \otimes B_z^+ + B_z^{-*} \otimes B_z^-. \tag{4.49}$$

https://scholar.sun.ac.za

The transfer matrix associated with the observable O_i for site $i > 0$ is defined as

$$T_{ao} \equiv \sum_{\sigma,\sigma'=\pm} A_z^{\sigma *} \otimes A_z^{\sigma'} \langle \sigma | O | \sigma' \rangle, \tag{4.50}$$

while for site $i = 0$ this becomes

$$T_{bo} \equiv \sum_{\sigma,\sigma'=\pm} B_z^{\sigma *} \otimes B_z^{\sigma'} \langle \sigma | O | \sigma' \rangle. \tag{4.51}$$

Our calculations will involve arbitrary powers of T_a and T_{ax}, with the latter being the transfer matrix associated with the observable X_i at a site $i > 0$. It will therefore be useful to obtain diagonalised forms of these two matrices. From Eq. (4.47) it is clear that the $A_z^{\pm}$ mattrices are at most of rank one, and since $\text{rank}(M \otimes N) = \text{rank}(M)\,\text{rank}(N)$ and $\text{rank}(M + N) \leq \text{rank}(M) + \text{rank}(N)$ it follows from Eq. (4.48) that T_a is at most of rank two. The two right and two left eigenvectors of T_a with non-zero eigenvalues are found to be

$$|r_0\rangle = (c^2, 0, 0, s^2)^T, \qquad |r_1\rangle = \left(\frac{c^2 D(C + ST - 1)}{ST(ST - 1)}, \frac{1}{2}, \frac{1}{2}, \frac{s^2 D(C - ST + 1)}{ST(ST - 1)} \right)^T \tag{4.52}$$

and

$$|l_0\rangle = \left(1, \frac{CD}{1 - ST}, \frac{CD}{1 - ST}, 1\right)^T, \qquad |l_1\rangle = (0, 1, 1, 0)^T. \tag{4.53}$$

These satisfy the eigenvalue equations

$$T_a |r_0\rangle = |r_0\rangle, \qquad T_a |r_1\rangle = ST |r_1\rangle, \qquad \langle l_0| T_a = \langle l_0|, \qquad \langle l_1| T_a = ST \langle l_1|, \tag{4.54}$$

and are normalised such that $\langle l_i | r_j \rangle = \delta_{ij}$ for $i, j \in \{0, 1\}$. This allows T_a, and powers thereof, to be expressed in the diagonal forms

$$T_a = |r_0\rangle\langle l_0| + ST\, |r_1\rangle\langle l_1| \quad \text{and} \quad T_a^N = |r_0\rangle\langle l_0| + (ST)^N\, |r_1\rangle\langle l_1|. \tag{4.55}$$

The same procedure can be applied to the transfer matrix T_{ax}, of which the explicit form follows from Eq. (4.50) as

$$T_{ax} \equiv A_z^{+*} \otimes A_z^{-} + A_z^{-*} \otimes A_z^{+}. \tag{4.56}$$

Again T_{ax} is at most of rank two, and the eigenvectors with non-zero eigenvalues are

$$|r_0'\rangle = (0, -1, 1, 0)^T, \qquad |r_1'\rangle = \left(-c^2, \frac{DS}{2T}, \frac{DS}{2T}, s^2\right)^T \tag{4.57}$$

and

$$|l_0'\rangle = \left(0, -\frac{1}{2}, -\frac{1}{2}, 0\right)^T, \qquad |l_1'\rangle = (-1, 0, 0, 1)^T. \tag{4.58}$$

https://scholar.sun.ac.za

These obey the eigenvalue equations

$$T_{ax}\left|r_0'\right\rangle = S\left|r_0'\right\rangle, \qquad T_{ax}\left|r_1'\right\rangle = T\left|r_1'\right\rangle, \qquad \left\langle l_0'\right| T_{ax} = S\left\langle l_0'\right|, \qquad \left\langle l_1'\right| T_{ax} = T\left\langle l_1'\right|, \tag{4.59}$$

and are normalised such that $\langle l_i'|r_j'\rangle = \delta_{ij}$ for $i,j \in \{0,1\}$. This allows for the diagonal representations

$$T_{ax} = S\left|r_0'\right\rangle\!\left\langle l_0'\right| + T\left|r_1'\right\rangle\!\left\langle l_1'\right| \quad \text{and} \quad T_{ax}^N = S^N\left|r_0'\right\rangle\!\left\langle l_0'\right| + T^N\left|r_1'\right\rangle\!\left\langle l_1'\right|. \tag{4.60}$$

4.4 Expectation value calculations for the translationally invariant ansatz

The translationally invariant ansatz is obtained by replacing $B^{\pm}$ by $A^{\pm}$ in the original MPS ansatz (4.33). This particular modification is justified in Chapter 5; it marginally improves results for small systems, while still yielding the same ground-state energy as the original ansatz for both models in the thermodynamic limit. The modification yields

$$|\tilde{\psi}_t\rangle \equiv \sum_{\vec{\sigma}} \mathrm{Tr}(A_z^{\sigma_0}\cdots A_z^{\sigma_{N-1}})\,|\vec{\sigma}\,\rangle, \tag{4.61}$$

where the tilde indicates that the state is unnormalised. The loss of normalisation is due to the fact that this state can no longer be expressed as a unitary operator acting on a product state, as was the case for the original ansatz in Eq. (4.1). To normalise the state in Eq. (4.61) we first compute, using Eq. (4.41) with $O = I$,

$$M_t^2 \equiv \langle\tilde{\psi}_t|\tilde{\psi}_t\rangle = \sum_{\vec{\sigma}} \mathrm{Tr}(A_z^{\sigma_0}\cdots A_z^{\sigma_{N-1}})^* \,\mathrm{Tr}(A_z^{\sigma_0}\cdots A_z^{\sigma_{N-1}}) = \mathrm{Tr}(T_a^N). \tag{4.62}$$

Using the diagonal form of T_a in Eq. (4.55) we obtain

$$M_t^2 = 1 + (ST)^N, \tag{4.63}$$

which we use to define the normalised translationally invariant state as

$$|\psi_t\rangle = \frac{1}{M_t}\,|\tilde{\psi}_t\rangle. \tag{4.64}$$

The expectation value of the observable X at site i with respect to $|\psi_t\rangle$ now follows from Eq. (4.41) as

$$\langle X_i\rangle = \frac{1}{M_t^2}\,\mathrm{Tr}(T_a^i T_{ax} T_a^{N-i-1}). \tag{4.65}$$

https://scholar.sun.ac.za

The explicit forms of T_a, T_{ax}, and their powers in Eqs. (4.55) and (4.60) allow this and similar traces to be evaluated easily. The resulting algebra, while somewhat lengthy in cases, is straightforward to perform using software packages such as *Mathematica* or *SymPy*. We find that

$$\langle X_i \rangle = \frac{1}{M_t^2}\left[\frac{C^2(S-T)}{ST-1} + (ST)^N \frac{D^2(S-T)}{T^2(ST-1)}\right]. \tag{4.66}$$

We can apply the same procedure to obtain a closed-form expression for the correlation function $\langle \psi_t | Z_i Z_{i+r} | \psi_t \rangle$, which, from Eq. (4.41) is given by the trace

$$\langle \psi_t | Z_i Z_{i+r} | \psi_t \rangle = \frac{1}{M_t^2} \mathrm{Tr}(T_{az} T_a^{r-1} T_{az} T_a^{N-r-1}). \tag{4.67}$$

The relevant transfer matrix follows from Eq. (4.50) as

$$T_{az} \equiv A_z^{+*} \otimes A_z^{+} - A_z^{-*} \otimes A_z^{-}. \tag{4.68}$$

Combining this with the diagonal form of T_a from Eq. (4.55) yields

$$\langle \psi_t | Z_i Z_{i+r} | \psi_t \rangle = \frac{1}{M_t^2}\left[f(r) + (ST)^N f(-r)\right], \tag{4.69}$$

where

$$f(r) = \frac{C^2 D^2 + (S-T)^2 (ST)^r}{(ST-1)^2}. \tag{4.70}$$

4.5 Expectation value calculations for the translationally and parity symmetric ansatz

We can restore the parity symmetry that the state $|\tilde{\psi}_t\rangle$ from Eq. (4.64) breaks by projecting it onto the positive parity subspace. This yields the unnormalised state

$$|\tilde{\psi}_p\rangle \equiv \frac{1}{\sqrt{2}}\Big(|\tilde{\psi}_t\rangle + P|\tilde{\psi}_t\rangle\Big), \tag{4.71}$$

where $P \equiv \prod_{i=0}^{N-1} X_i$ is the parity operator that flips all spins via the Pauli X operator. The required normalisation factor for $|\tilde{\psi}_p\rangle$ is found to be

$$M_p^2 \equiv \langle \tilde{\psi}_p | \tilde{\psi}_p \rangle = \langle \tilde{\psi}_t | \tilde{\psi}_t \rangle + \langle \tilde{\psi}_t | P | \tilde{\psi}_t \rangle = M_t^2 + \mathrm{Tr}(T_{ax}^N), \tag{4.72}$$

https://scholar.sun.ac.za

where the extension of Eq. (4.43) to the N-site operator P was used to express the second term as a trace. Using the normalisation factor of $|\psi_t\rangle$ in Eq. (4.63) and the diagonal form of T_{ax} in Eq. (4.56) we find that

$$M_p^2 = 1 + (ST)^N + T^N + S^N. \tag{4.73}$$

The normalised parity and translationally symmetric ansatz is now given by

$$|\psi_p\rangle = \frac{1}{M_p}|\tilde{\psi}_p\rangle\,. \tag{4.74}$$

To calculate the expectation value of the observable X_i with respect to this state, we first write

$$\langle\psi_p|X_i|\psi_p\rangle = \frac{1}{M_p^2}\Big(\langle\tilde{\psi}_t|X_i|\tilde{\psi}_t\rangle + \langle\tilde{\psi}_t|X_iP|\tilde{\psi}_t\rangle\Big). \tag{4.75}$$

As in the previous section, we can translate these expectation values with respect to $|\tilde{\psi}_t\rangle$ into traces involving the relevant transfer matrices. Notice that X_i in the second term undoes the action of P on the ith site, and so by extending Eq. (4.43), Eq. (4.75) becomes

$$\langle\psi_p|X_i|\psi_p\rangle = \frac{1}{M_p^2}\left[\mathrm{Tr}(T_a^{N-1}T_{ax}) + \mathrm{Tr}(T_{ax}^{N-1}T_a)\right]. \tag{4.76}$$

The diagonal forms of T_a and T_{ax} in Eqs. (4.55) and (4.60) now lead to

$$\langle\psi_p|X_i|\psi_p\rangle = \frac{1}{M_p^2}\left[\frac{C^2(S-T)}{ST-1} + \frac{(ST)^N D^2(S-T)}{T^2(ST-1)} + T^{N-2}(T-S)(ST+1)\right]. \tag{4.77}$$

The same procedure yields the correlation function

$$\langle\psi_p|Z_iZ_{i+r}|\psi_p\rangle = \frac{1}{M_p^2}\left[f(r) + (ST)^N f(-r)\right] + \frac{(ST)^r}{M_p^2}\left(S^{N-2r} + T^{N-2r}\right), \tag{4.78}$$

with $f(r)$ as in Eq. (4.70).

4.6 Expectation value calculations for the original ansatz

The expression $\langle\psi_t|O_i|\psi_t\rangle = \frac{1}{M^2}\mathrm{Tr}(T^iT_OT^{N-i-1})$ in Eq. (4.41) for the expectation value of a single-site operator with respect to a translationally invariant MPS

https://scholar.sun.ac.za

state can be easily adapted to the original ansatz $|\psi_o\rangle$ in Eq. (4.33). For the expectation value of X_i this results in

$$\langle\psi_o|X_i|\psi_o\rangle = \begin{cases} \mathrm{Tr}(T_b T_a^{i-1} T_x T_a^{N-i-1}) & \text{when } i \neq 0, \\ \mathrm{Tr}(T_{bx} T_a^{N-1}) & \text{when } i = 0. \end{cases} \tag{4.79}$$

The transfer matrix T_b is given in Eq. (4.49), while T_{bx} follows from Eq. (4.51) as

$$T_{bx} = B_z^{+*} \otimes B_z^{-} + B_z^{-*} \otimes B_z^{+}. \tag{4.80}$$

For $i \neq 0$ we find

$$\langle\psi_o|X_i|\psi_o\rangle = \frac{C^2(S-T)}{ST-1} - (ST)^{i-1}\Big(\frac{C^2D^2S}{ST-1} + SCDF\Big). \tag{4.81}$$

For $i = 0$ this becomes

$$\langle\psi_o|X_0|\psi_o\rangle = C(c^2T + s^2U) + \frac{CSD}{ST-1}(c^2D + s^2E) - CS(ST)^{N-1} \tag{4.82}$$

$$- (ST)^{N-1}\left[\frac{c^2D^2(C + ST - 1) + s^2DE(C - ST + 1)}{T(ST-1)}\right]. \tag{4.83}$$

4.7 Thermodynamic limit of the average magnetisation per site

The expectation values of X_i with respect to the three main ansatzes were found in Eqs. (4.66), (4.77) and (4.81) to be

$$\langle\psi_t|X_i|\psi_t\rangle = \frac{1}{M_t^2}\left[\frac{C^2(S-T)}{ST-1} + (ST)^N\frac{D^2(S-T)}{T^2(ST-1)}\right], \tag{4.84}$$

$$\langle\psi_p|X_i|\psi_p\rangle = \frac{1}{M_p^2}\left[\frac{C^2(S-T)}{ST-1} + T^N\frac{(D^2S^N + 1 - S^2T^2)(S-T)}{T^2(ST-1)}\right], \tag{4.85}$$

$$\langle\psi_o|X_{i\neq 0}|\psi_o\rangle = \frac{C^2(S-T)}{ST-1} - (ST)^{i-1}\Big(\frac{C^2D^2S}{ST-1} + SCDF\Big). \tag{4.86}$$

We see that both the translational and parity symmetric ansatzes converge to the same expression in the thermodynamic limit:

$$\lim_{N\to\infty}\langle\psi_t|X_i|\psi_t\rangle = \lim_{N\to\infty}\langle\psi_p|X_i|\psi_p\rangle = \frac{C^2(S-T)}{ST-1}. \tag{4.87}$$

https://scholar.sun.ac.za

In particular, this implies that the average magnetisation per spin in the direction of the transverse field, i.e. $\langle \sigma_x \rangle$ with $\sigma_x = \frac{1}{2}\sum_{i=0}^{N-1} X_i$, will also be given by the same expression in the thermodynamic limit for both these ansatzes. This term appears in the Hamiltonian for both the Ising and LMG models. If we consider the same quantity with respect to the original ansatz we find that it too converges to this expression:

$$\lim_{N\to\infty} \frac{1}{N} \langle \psi_o | \sigma_x | \psi_o \rangle = \lim_{N\to\infty} \Big(\frac{1}{2N} \langle \psi_o | X_0 | \psi_o \rangle + \frac{1}{2N} \sum_{i=1}^{N-1} \langle \psi_o | X_i | \psi_o \rangle \Big), \tag{4.88}$$

$$= \frac{C^2(S-T)}{2(ST-1)} - \lim_{N\to\infty} \frac{K_0}{2N} \sum_{i=1}^{N-1} (ST)^{i-1}, \tag{4.89}$$

$$= \frac{C^2(S-T)}{2(ST-1)} - \lim_{N\to\infty} \frac{K_0}{2N} \frac{1-(ST)^{N-1}}{1-ST}, \tag{4.90}$$

$$= \frac{C^2(S-T)}{2(ST-1)}. \tag{4.91}$$

Here $K_0 = \frac{C^2 D^2 S}{ST-1} + SCDF$ which does not depend on N.

4.8 Growth of bond dimension

Generally speaking for any MPS, the application of the operator C_{ij}^{θ} will increase the bond dimension between sites i and j by a factor of two. This is because C_{ij}^{θ} adds a new virtual bond between the sites i and j which we can explicitly capture as a contraction between two two-dimensional vectors of operators,

$$C_{ij}^{\theta} = \begin{bmatrix} I_i & Z_i \end{bmatrix} \begin{bmatrix} cI_j \\ isY_j \end{bmatrix} = cI_iI_j + isZ_iY_j. \tag{4.92}$$

Here it is understood that matrix multiplication is performed during the contraction process. Since every site in our ansatz shares one control and one target from all the C_{ij}^{θ} operators in Eq. (4.1), every site has two virtual indices each of dimension two. Therefore our state can be written as a matrix product state (MPS) with bond dimension $D = 2$. To explicitly see how C_{ij} introduces virtual bonds, consider an arbitrary product state $|\rho\rangle$,

$$|\rho\rangle = \sum_{\vec{\sigma}} \Big(\prod_{k=0}^{N-1} p_k(\sigma_k) \Big) \, |\vec{\sigma}\rangle \tag{4.93}$$

https://scholar.sun.ac.za

where $p_k(\sigma_k) \in \mathbb{C}$ are order-0 tensors. Applying C_{ij}^{θ} on $|\rho\rangle$ turns the scalars p_i and p_j into the vectors $\mathbf{p_i}$ and $\mathbf{p_j}$ as follows:

$$C_{ij}^{\theta}|\rho\rangle = \sum_{\vec{\sigma}}\Big(\prod_{k=0}^{N-1} p_k(\sigma_k)\Big)\Big([I_i \quad Z_i]\begin{bmatrix} cI_j \\ is\,Y_j\end{bmatrix}\Big)|\sigma_i\sigma_j\rangle\otimes|\vec{\sigma}'\rangle\,, \tag{4.94}$$

$$= \sum_{\vec{\sigma}}\Big(\prod_{k=0}^{N-1} p_k(\sigma_k)\Big)\Big([|\sigma_i\rangle \quad |-\sigma_i\rangle]\begin{bmatrix} c\,|\sigma_j\rangle \\ is\,\sigma_j\,|\sigma_j\rangle\end{bmatrix}\Big)\otimes|\vec{\sigma}'\rangle\,, \tag{4.95}$$

$$= \sum_{\vec{\sigma}}\Big(\prod_{k\neq i,j} p_k(\sigma_k)\Big)\Big([p_i(\sigma_i)\,|\sigma_i\rangle \quad p_i(\sigma_i)\,|-\sigma_i\rangle]\begin{bmatrix} c\,p_j(\sigma_j)\,|\sigma_j\rangle \\ is\,\sigma_j p_j(\sigma_j)\,|\sigma_j\rangle\end{bmatrix}\Big)\otimes|\vec{\sigma}'\rangle\,, \tag{4.96}$$

$$= \sum_{\vec{\sigma}}\Big(\prod_{k\neq i,j} p_k(\sigma_k)\Big)\Big([p_i(\sigma_i) \quad p_i(-\sigma_i)]\begin{bmatrix} c\,p_j(\sigma_j) \\ is\,\sigma_j p_j(\sigma_j)\end{bmatrix}\Big)|\vec{\sigma}\rangle\,, \tag{4.97}$$

$$\equiv \sum_{\vec{\sigma}}\Big(\prod_{k\neq i,j} p_k(\sigma_k)\Big)\mathbf{p}_i(\sigma_i)^{\dagger}\mathbf{p}_j(\sigma_j)\,|\vec{\sigma}\rangle\,. \tag{4.98}$$

Here we obtained Eq. (4.97) by regrouping terms that share the same σ_i in the total sum. Similarly, applying one C_{ij}^{θ} after the other leads to

$$C_{jl}^{\theta}C_{ij}^{\theta}|\rho\rangle = \sum_{\vec{\sigma}}\Big(\prod_{k\neq i,j} p_k(\sigma_k)\Big)\mathbf{p}_i(\sigma_i)^{\dagger}\mathbf{p}_j(\sigma_j)\Big([I_j \quad Z_j]\begin{bmatrix} cI_l \\ is\,Y_l\end{bmatrix}\Big)|\sigma_i\sigma_l\rangle\otimes|\vec{\sigma}'\rangle\,, \tag{4.99}$$

$$= \textstyle\sum_{\vec{\sigma}}\Big(\prod_{k\neq i,j} p_k(\sigma_k)\Big)\mathbf{p}_i(\sigma_i)^{\dagger}\Big([\mathbf{p}_j(\sigma_j) \quad 0]\otimes|\sigma_j\rangle + [0 \quad \mathbf{p}_j(\sigma_j)]\otimes|-\sigma_j\rangle\Big)\begin{bmatrix} c\,p_l(\sigma_l) \\ is\,\sigma_l p_l(\sigma_l)\end{bmatrix}\otimes|\sigma_l,\vec{\sigma}'\rangle\,, \tag{4.100}$$

$$= \sum_{\vec{\sigma}}\Big(\prod_{k\neq i,j,l} p_k(\sigma_k)\Big)\mathbf{p}_i(\sigma_i)^{\dagger}\Big([\mathbf{p}_j(\sigma_j) \quad \mathbf{p}_j(-\sigma_j)]\begin{bmatrix} c\,p_l(\sigma_l) \\ is\,\sigma_l p_l(\sigma_l)\end{bmatrix}\Big)|\vec{\sigma}\rangle\,, \tag{4.101}$$

$$\equiv \sum_{\vec{\sigma}}\Big(\prod_{k\neq i,j,l} p_k(\sigma_k)\Big)\mathbf{p}_i(\sigma_i)^{\dagger}P_j(\sigma_j)\mathbf{p}_l(\sigma_l)\,|\vec{\sigma}\rangle\,. \tag{4.102}$$

Here C_{ij}^{θ} introduced a virtual bond between sites i and j and C_{jl}^{θ} introduced a virtual bond between sites j and l. Resulting in site j having two virtual indices, each of dimension two.

https://scholar.sun.ac.za

Chapter 5

Predictions for Two Prototypical Spin Models

The ansatz is again shown in Fig. 5.1. Recall that for N spins it produces the state

$$|\psi_o\rangle = \left(\prod_{k=0}^{\overleftarrow{N-1}} C^{\theta}_{k,k+1} R^{\theta}_{k+1}\right)\left(\prod_{j=0}^{\overleftarrow{N-1}} R^{\phi}_{j}\right)|z,+\rangle^{\otimes N}, \tag{5.1}$$

where ϕ and θ are variational parameters and $|z,\pm\rangle$ are the eigenstates of the Pauli-Z matrix with eigenvalues ± 1. The operator products in Eq. (5.1) are ordered with the k and j indices increasing from right to left. The two unitary operators appearing in $|\psi_o\rangle$ are

$$C^{\theta}_{ij} = e^{i\frac{\theta}{2}Z_i Y_j} \qquad \text{and} \qquad R^{\theta}_{j} = e^{-i\frac{\theta}{2}Y_j}. \tag{5.2}$$

Here (X_i, Y_i, Z_i) are the Pauli spin matrices associated with the spin at site $i \in \{0, \ldots, N-1\}$, obeying periodic boundary conditions: $i + N \equiv i$. In Section 4.1 we derived an exact expression of $|\psi_o\rangle$ as an MPS. In this form the ansatz reads

$$|\psi_o\rangle = 2^{-N/2} \sum_{\vec{\sigma}} \mathrm{Tr}(B^{\sigma_0} A^{\sigma_1} \cdots A^{\sigma_{N-1}})\,|y,\vec{\sigma}\rangle\,, \tag{5.3}$$

where $|y,\sigma_i\rangle$ are the eigenstates of the Pauli-Y_i matrix with eigenvalues $\sigma_i = \pm 1$, and

$$A^{+} = \begin{bmatrix} \cos\frac{\theta}{2} & \cos\frac{\theta}{2} \\ i\sin\frac{\theta}{2} & -i\sin\frac{\theta}{2} \end{bmatrix} \begin{bmatrix} e^{-\frac{i(\theta+\phi)}{2}} & 0 \\ 0 & e^{\frac{i(\theta+\phi)}{2}} \end{bmatrix}, \tag{5.4}$$

$$B^{+} = \begin{bmatrix} \cos\frac{\theta}{2} & \cos\frac{\theta}{2} \\ i\sin\frac{\theta}{2} & i\sin\frac{\theta}{2} \end{bmatrix} \begin{bmatrix} e^{-\frac{i(\theta+\phi)}{2}} & 0 \\ 0 & e^{-\frac{i(\theta-\phi)}{2}} \end{bmatrix}, \tag{5.5}$$

$$A^{-} = (A^{+})^{*}, \quad B^{-} = (B^{+})^{*}. \tag{5.6}$$

https://scholar.sun.ac.za

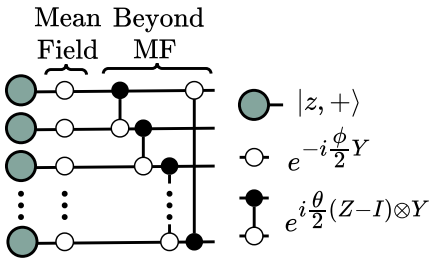


Figure 5.1: The ansatz generated by our method for the LMG and TFIM models.

Both the TFIM and LMG models exhibit translational invariance. However, this property is not shared by $|\psi_o\rangle$ due to the lone $B^{\pm}$ matrix appearing in Eq. (5.3). It seems that the complexity penalty on motifs during the search prevents the generation of an explicitly translationally invariant state. At least, this is true when the operator pool consists of Unitaries. This is consistent with the idea that symmetry-breaking ansatzes require lower structural complexity for similar ground-state energy convergence as symmetry-preserving ansatzes in VQE [130, 131]. We will restore this symmetry through a minimal modification of the original ansatz $|\psi_o\rangle$ by replacing the B^{σ_0} with A^{σ_0} in Eq. (5.3). This modification marginally improves results for small systems, while still yielding the same ground-state energy as the original ansatz for both models in the thermodynamic limit. This results in the translationally invariant ansatz

$$|\psi_t\rangle = \frac{1}{M}\sum_{\vec{\sigma}} \mathrm{Tr}(A^{\sigma_0}\cdots A^{\sigma_{N-1}})\,|y,\vec{\sigma}\rangle\,, \tag{5.7}$$

where M is a normalisation factor. The structure of both $|\psi_o\rangle$ and $|\psi_t\rangle$ in Eqs. (5.3) and (5.7) allows us to obtain closed-form expressions for expectation values using a transfer matrix approach. See Section 4.4. For $|\psi_t\rangle$ we find

$$\langle X_i\rangle = \frac{1}{M^2}\left[\frac{c^2(s-t)}{st-1} + \frac{d^2(s-t)}{t^2(st-1)}(st)^N\right], \tag{5.8}$$

$$\langle Z_i\rangle = \frac{1}{M^2}\left[\frac{cd((st)^N-1)}{st-1}\right], \tag{5.9}$$

$$\langle Z_i Z_{i+r}\rangle = \frac{1}{M^2}\left[f(r) + (st)^N f(-r)\right], \tag{5.10}$$

where

$$f(r) = \frac{c^2d^2 + (s-t)^2(st)^r}{(st-1)^2}, \tag{5.11}$$

$$M^2 = 1 + (st)^N, \quad |st| \neq 1 \tag{5.12}$$

https://scholar.sun.ac.za

and

$$c = \cos(\theta), \quad s = \sin(\theta), \tag{5.13}$$
$$d = \cos(\theta + \phi), \quad t = \sin(\theta + \phi). \tag{5.14}$$

Note that this notation differs from the one used in Chapter 4.

5.1 The Lipkin-Meshkov-Glick model

The LMG Hamiltonian for N spin-$\frac{1}{2}$ particles reads

$$H = -\frac{J}{4N}\sum_{i<j} Z_i Z_j - \frac{h}{2}\sum_{i=0}^{N-1} X_i, \tag{5.15}$$

where J and h set the strengths of the spin-spin interaction and external field respectively. The all-to-all nature of the spin interaction results in the system's mean-field description becoming exact for certain predictions in the thermodynamic limit. We first show that our ansatz shares this property. Fig. 5.1 shows that our approach contains the mean-field result as a special case. Specifically, the first layer of R^ϕ rotations generates a product state amounting to a mean-field ansatz. When $\theta \neq 0$, the second layer of $C^\theta R^\theta$ rotations then introduces correlations beyond the mean-field level. To proceed, we calculate the energy per spin in the thermodynamic limit with respect to $|\psi_t\rangle$ using Eqs. (5.8) and (5.10). See Section 5.1.1 for details. Setting $J = 1$, this yields

$$\lim_{N\to\infty} \frac{\langle H\rangle}{N} = -\frac{c^2 d^2}{8(st-1)^2} - \frac{h(s-t)c^2}{2(st-1)}, \tag{5.16}$$

which is a function of θ and ϕ via Eqs. (5.13) and (5.14). Minimising this expression with respect to these angles produces

$$\sin(\phi) = \begin{cases} 2h & |2h| \leq 1 \\ \mathrm{sgn}(h) & \text{otherwise} \end{cases} \quad \text{and} \quad \theta = 0, \tag{5.17}$$

as shown in Section 5.1.2. The vanishing of θ implies that our ansatz reduces to a product state generated by the first layer of R^ϕ rotations. Inserting this into Eq. (5.9) for $\langle Z_i\rangle$ yields the spontaneous magnetisation

$$\lim_{N\to\infty} \frac{1}{2N}\sum_i \langle Z_i\rangle = \begin{cases} \pm\frac{1}{2}\sqrt{1-4h^2} & |2h| \leq 1 \\ 0 & \text{otherwise} \end{cases}, \tag{5.18}$$

https://scholar.sun.ac.za

from which we identify the critical value of h as $h_c = 1/2$. This field strength marks the transition between the paramagnetic ($|h| > h_c$) and ferromagnetic ($|h| < h_c$) phases. For the energy per spin we find

$$\lim_{N\to\infty} \frac{\langle H\rangle}{N} = \begin{cases} -\frac{1}{2}(h^2 + \frac{1}{4}) & |h| \leq h_c \\ -\frac{|h|}{2} & |h| > h_c \end{cases}. \tag{5.19}$$

Both Eqs. (5.18) and (5.19) are exact results for the thermodynamic limit.

For finite systems, the optimal value of θ is non-zero, and the layer of $C^\theta R^\theta$ rotations in Eq. (5.1) will introduce correlations between the spins. This brings about a major improvement in accuracy compared to the product state mean-field ansatz. We further this improvement by restoring in our ansatz the symmetries present in the LMG Hamiltonian (5.15). Specifically, H exhibits permutation symmetry under the exchange of any two spins, and also parity symmetry under a π-rotation about the x-axis, which sends (X_i, Y_i, Z_i) to $(X_i, -Y_i, -Z_i)$. We enforce these symmetries on the ansatz $|\psi_t\rangle$ by projecting it onto the relevant symmetry subspaces. As shown in Section 5.1.3, this yields a state $|\psi_s\rangle$ within the $(2S+1)$-dimensional subspace corresponding to the maximum magnitude $S = N/2$ of the total spin. The analytic expression for $|\psi_s\rangle$, parameterised by θ and ϕ, now serves as a refined version of the original ansatz. We use this symmetrised ansatz to estimate the ground-state energy as well as the RMS magnetisation

$$M_{\text{rms}} = \frac{1}{2N}\sqrt{\left\langle \left(\textstyle\sum_i Z_i\right)^2 \right\rangle}. \tag{5.20}$$

Fig. 5.2 shows the result of this calculation of M_{rms} for different numbers of spins N. Remarkably, there is no visible difference between our ansatz-based result and the exact value of the magnetisation. This suggests that the symmetrised ansatz $|\psi_s\rangle$ captures finite-size effects very accurately. Fig. 5.3 shows the relative error in the ground-state energy, $\epsilon_{\text{rel}} = |E_{\text{pred}} - E_{\text{exact}}|/|E_{\text{exact}}|$, for different field strengths h, plotted on a logarithmic scale as N increases. For our symmetrised ansatz, this error is at most of order 10^{-6} for $h = h_c = 1/2$ and about $N = 25$, and tends to zero as N increases. The result of using the mean-field product state ansatz (with $\theta = 0$) is also shown. While this too becomes exact in the thermodynamic limit, it fares much worse than the symmetrised ansatz for finite system sizes.

Both the translationally invariant ansatz $|\psi_t\rangle$ and its symmetrised counterpart $|\psi_s\rangle$ produce exact results for M_{rms} and the ground-state energy within the thermodynamic limit. However, these quantities probe limited features of the two states, and it turns out that $|\psi_t\rangle$ and $|\psi_s\rangle$ have fundamentally different characters, even in this limit. Specifically, we found that $|\psi_t\rangle$ reduces to a mean-field

https://scholar.sun.ac.za

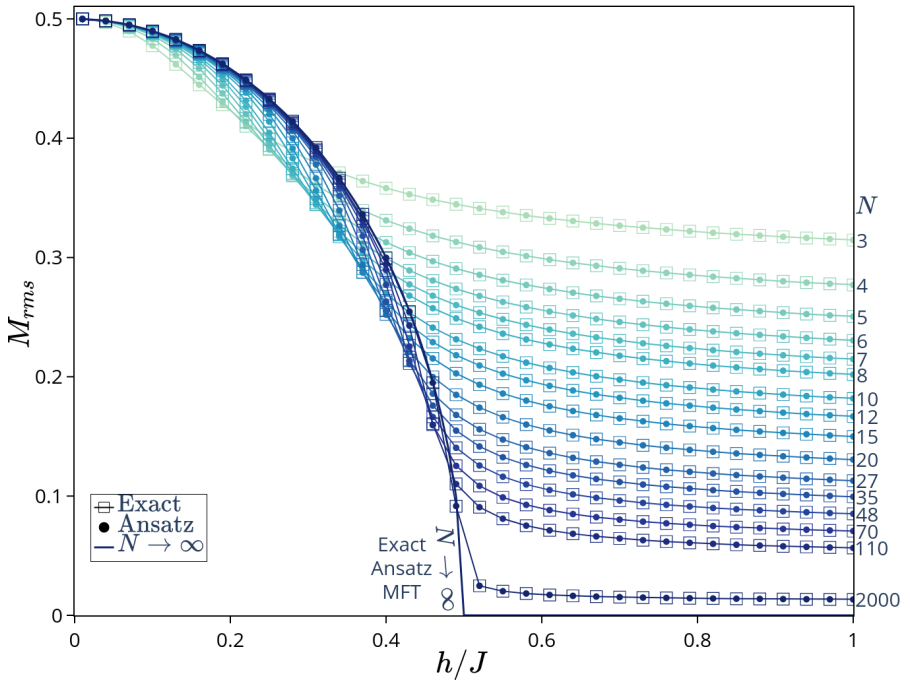


Figure 5.2: RMS magnetisation of Eq. (5.20) vs h/J for the LMG model. Exact results compared to the symmetrised ansatz.

product state as $N \to \infty$ due to the optimal value of θ vanishing. In contrast, optimising $|\psi_s\rangle$ yields a non-zero θ, even in the $N \to \infty$ limit, thereby retaining the entanglement from the $C^\theta R^\theta$ rotations. This aligns with Refs. [132–134] where it is shown that the *exact* ground state in the paramagnetic phase always contains non-trivial entanglement and does not reduce to a product state as $N \to \infty$. These observations underscore that the symmetrisation step can fundamentally alter the correlations present in the ansatz, and that optimising the variational parameters before versus after this step can yield very different results.

5.1.1 The energy per spin in the thermodynamic limit

Both $s = \sin(\theta)$ and $t = \cos(\theta + \phi)$ are sines of angles, and so st cannot exceed one in magnitude. Starting from the full expression for the expectation value of the LMG Hamiltonian and assuming that $|st| < 1$ then yields in the $N \to \infty$ limit the result

$$\lim_{N\to\infty} \langle H\rangle/N = \frac{h\,(s^2-1)\,(s-t)}{2st-2} - \frac{J(s^2-1)\,(t^2-1)}{8(st-1)^2}, \tag{5.21}$$

https://scholar.sun.ac.za

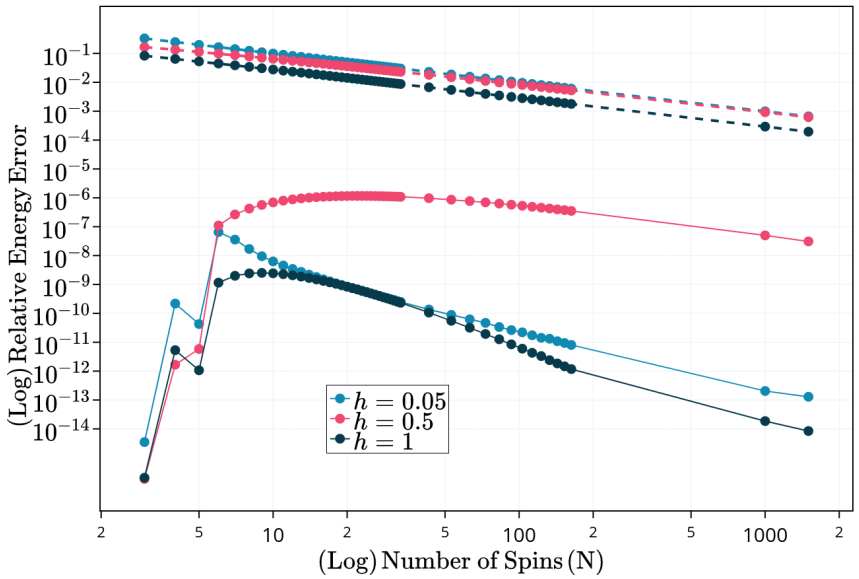


Figure 5.3: Relative ground-state energy error vs N for the LMG model at different field strengths h. Compares symmetrised ansatz (solid) and MFT (dashed).

The case for which $|st| = 1$ occurs when $(s,t) = (1,1)$, $(s,t) = (-1,-1)$, $(s,t) = (1,-1)$, or $(s,t) = (-1,1)$. Focussing on even N, we find for the first two of these that $\lim_{N\to\infty}\langle H\rangle/N = -J/8$, and for the second two $\lim_{N\to\infty}\langle H\rangle/N = 0$. It is only precisely at $h = 0$ where the first of these energies match the energy obtained from the optimal angles in Eq. (5.18). For all other values of h, using values of the angles for which $|st| = 1$ therefore does not yield the minimum energy. We therefore assume that $|st| < 1$.

5.1.2 Energy minimisation

For the LMG model the derivatives of the energy expectation value with respect to the two angles, written in terms of $s = \sin(\theta)$ and $t = \sin(\theta + \phi)$ are:

$$\partial_\theta\langle H\rangle/N = \frac{\sqrt{1-s^2}\left(2h(st-1)\left(-\left(s^2+1\right)t^2+2\left(s^2+1\right)st-3s^2+1\right)+\left(t^2-1\right)(s-t)\right)}{4(st-1)^3}, \tag{5.22}$$

$$\partial_\phi\langle H\rangle/N = -\frac{\left(s^2-1\right)\sqrt{1-t^2}\left(2h\left(s^2-1\right)(st-1)+s-t\right)}{4(st-1)^3}. \tag{5.23}$$

Setting both of these derivatives to zero yields the following solutions:

https://scholar.sun.ac.za

1. $s = \sin(\theta) = \pm 1$ with $t = \sin(\theta + \phi)$ arbitrary. This yields $\langle H \rangle / N = 0$.
2. $s = \sin(\theta) = 0$ and $t = \sin(\theta + \phi) = \pm 1$. This yields $\langle H \rangle / N = \mp h/2$.
3. $s = \sin(\theta) = 0$ and $t = \sin(\theta + \phi) = 2h$. This yields $\langle H \rangle / N = -(1 + 4h^2)/8$.

The stationary point that yields the minimum value of $\langle H \rangle / N$ is therefore given by

$$\sin(\theta + \phi) = \begin{cases} 2h & |2h| \leq 1 \\ \operatorname{sgn}(h) & \text{otherwise} \end{cases} \quad \text{and} \quad \sin(\theta) = 0. \tag{5.24}$$

This yields $\theta = 0, \pi$. Selecting $\theta = 0$ leads to one solution for ϕ when $|2h| \geq 1$ and two solutions when $|2h| < 1$. The latter two solutions correspond to the two symmetry-broken ground states in the ferromagnetic phase. Selecting $\theta = \pi$ leads to the same solution(s) for ϕ, just shifted by π. However, this yields, up to a phase, the same ansatz as when $\theta = 0$. (Although the two solutions for ϕ get exchanged.) We therefore only consider the $\theta = 0$ solution.

5.1.3 Symmetry projection

Here we provide details on how the permutation and parity symmetries of the LMG Hamiltonian is restored in the translationally invariant ansatz from Eq. (5.7). This requires projecting the ansatz onto the appropriate symmetry subspace. We first define the permutation operator $\mathbb{P}$ through its action on a spin basis state $|\vec{\sigma}\rangle = |\sigma_0, \sigma_1, \ldots, \sigma_{N-1}\rangle$ as

$$\mathbb{P} |\vec{\sigma}\rangle = \frac{1}{N!} \sum_{p \in \sigma_N} |\sigma_{p(0)}, \sigma_{p(1)}, \ldots, \sigma_{p(N-1)}\rangle \tag{5.25}$$

where the sum runs over all permutations of $\{0, 1, \ldots, N-1\}$. Let $n_-(\vec{\sigma})$ denote the number of $-$ states appearing in $\vec{\sigma}$. We define the normalised state

$$|n\rangle = \binom{N}{n}^{-1/2} \sum_{\vec{\sigma}'} \delta_{n_-(\vec{\sigma}'),n} |\vec{\sigma}'\rangle , \tag{5.26}$$

which is an equal superposition of the product states with exactly n spins in the $-$ state. This allows us to rewrite Eq. (5.25) as

$$\mathbb{P} |\vec{\sigma}\rangle = \binom{N}{n}^{-1/2} |n = n_-(\vec{\sigma})\rangle . \tag{5.27}$$

https://scholar.sun.ac.za

Applying $\mathbb{P}$ to the ansatz $|\psi_t\rangle = M^{-1}\sum_{\vec{\sigma}} \mathrm{Tr}(A_z^{\sigma_0}\cdots A_z^{\sigma_{N-1}})\,|\vec{\sigma}\,\rangle$ produces

$$\mathbb{P}\,|\psi_t\rangle = \frac{1}{M}\sum_{n=0}^{N}\binom{N}{n}^{-1/2}\mathrm{Tr}\left[\sum_{\vec{\sigma}}\delta_{n_-(\vec{\sigma}),n}A_z^{\sigma_0}\cdots A_z^{\sigma_{N-1}}\right]|n\rangle\,, \tag{5.28}$$

where the trace contains a sum of all products of n and $(N-n)$ of the A_z^- and A_z^+ matrices, respectively. Here we use the ansatz $|\psi_t\rangle$ in the Z basis where the $A_z^\pm$ matrices are given by Eq. (4.61) from Section 4.3. The normalisation factor M is the one from Eq. (5.12). To calculate this trace we utilise a generating function approach. Specifically, we note that the trace appearing in Eq. (5.28) will be the coefficient of the x^n term in the polynomial

$$\mathrm{Tr}((A_z^+ + xA_z^-)^N) = \lambda_+(x)^N + \lambda_-(x)^N, \tag{5.29}$$

where $\lambda_\pm(x)$ are the two eigenvalues of $A_z^+ + xA_z^-$. These can be calculated as

$$\lambda_\pm(x) = \frac{1}{2}\left((a+xb) \pm \sqrt{(a+xb)^2 - 4xg}\right) \tag{5.30}$$

where

$$a = \cos\left(\frac{\phi}{2}\right), \quad b = \sin\left(\theta+\frac{\phi}{2}\right), \quad g = \sin(\theta). \tag{5.31}$$

We can expand 5.29 using the binomial theorem, which results in

$$\lambda_+^N + \lambda_-^N = \frac{1}{2^{N-1}}\sum_{i=0}^{\lfloor N/2\rfloor}\binom{N}{2i}\left[(a+xb)^2 - 4xg\right]^i(a+xb)^{N-2i}. \tag{5.32}$$

The goal now is to rewrite this sum in order to group like powers of x together. Expanding the power of the square bracket in the sum yields

$$\lambda_+^N + \lambda_-^N = \frac{1}{2^{N-1}}\sum_{i=0}^{\lfloor N/2\rfloor}\binom{N}{2i}\sum_{j=0}^{i}\binom{i}{j}(-4g)^j x^j (a+xb)^{2(i-j)}(a+xb)^{N-2i} \tag{5.33}$$

$$= \frac{1}{2^{N-1}}\sum_{i=0}^{\lfloor N/2\rfloor}\binom{N}{2i}\sum_{j=0}^{i}\binom{i}{j}(-4g)^j x^j (a+xb)^{N-2j}. \tag{5.34}$$

Next we expand the $N-2j$ power of $(a+xb)$ to get

$$\lambda_+^N + \lambda_-^N = \frac{1}{2^{N-1}}\sum_{i=0}^{\lfloor N/2\rfloor}\binom{N}{2i}\sum_{j=0}^{i}\binom{i}{j}(-4g)^j x^j \sum_{k=0}^{N-2j}\binom{N-2j}{k}x^k b^k a^{N-2j-k} \tag{5.35}$$

$$= \frac{1}{2^{N-1}}\sum_{i=0}^{\lfloor N/2\rfloor}\sum_{j=0}^{i}\sum_{k=0}^{N-2j}\binom{N}{2i}\binom{i}{j}\binom{N-2j}{k}a^{N-2j-k}b^k(-4g)^j x^{k+j}. \tag{5.36}$$

https://scholar.sun.ac.za

Finally, we replace the sum over $k \in \{0, \dots, N-2j\}$ with one over $n = k + j \in \{j, \dots, N-j\}$. This produces

$$\lambda_+^N + \lambda_-^N = \frac{1}{2^{N-1}} \sum_{i=0}^{\lfloor N/2 \rfloor} \sum_{j=0}^{i} \sum_{n=j}^{N-j} \binom{N}{2i} \binom{i}{j} \binom{N-2j}{n-j} a^{N-j-n} b^{n-j} (-4g)^j x^n. \tag{5.37}$$

To obtain an expression for the coefficient of x^n we need to change the order of the summations. Here we use the fact that

$$\binom{m}{k} = 0 \quad \text{whenever } m < k \text{ or } k < 0. \tag{5.38}$$

From Eq. (5.38) we see that $\binom{N-2j}{n-j} = 0$ if $n < j$ so that

$$\sum_{n=j}^{N-j} \binom{N-2j}{n-j} = \sum_{n=0}^{N-j} \binom{N-2j}{n-j}, \tag{5.39}$$

where we chose $n = 0 \leq j$ as the lower limit of the summation. Similarly, the coefficient $\binom{N-2j}{n-j}$ is zero for $n - j > N - 2j$, which is equivalent to $n > N - j$ so that

$$\sum_{n=0}^{N-j} \binom{N-2j}{n-j} = \sum_{n=0}^{N} \binom{N-2j}{n-j}, \tag{5.40}$$

where we chose $n = N \geq N - j$ as the upper limit of the summation. Finally, we will show that

$$\sum_{j=0}^{i} \binom{i}{j} \binom{N-2j}{n-j} = \sum_{j=0}^{n} \binom{i}{j} \binom{N-2j}{n-j}, \tag{5.41}$$

where both $j = i$ or $j = n$ are valid upper limits for the summation. To see this, consider the case $i < n$, then for any $j = n > i$ we have $\binom{i}{j} = 0$, making Eq. (5.41) true. Next, consider the case $n < i$, then for any $j = i > n$ we have $\binom{N-2j}{n-j} = 0$ also making Eq. (5.41) true. Finally, Eq. (5.41) is obviously true when $i = n$. With these three observations we can rewrite Eq. (5.37) as

$$\lambda_+^N + \lambda_-^N = \frac{1}{2^{N-1}} \sum_{n=0}^{N} x^n a^{N-n} b^n \sum_{i=0}^{\lfloor N/2 \rfloor} \sum_{j=0}^{n} \binom{N}{2i} \binom{i}{j} \binom{N-2j}{n-j} (-1)^j \left(\frac{4g}{ab}\right)^j \tag{5.42}$$

https://scholar.sun.ac.za

If we let

$$T(N,j) \equiv \sum_{i=0}^{\lfloor N/2 \rfloor} \binom{N}{2i} \binom{i}{j}, \tag{5.43}$$

then Eq. (5.42) can be written as

$$\lambda_+^N + \lambda_-^N = \sum_{n=0}^{N} x^n \frac{a^{N-n}b^n}{2^{N-1}} \sum_{j=0}^{n} (-1)^j T(N,j) \binom{N-2j}{n-j} \left(\frac{4g}{ab}\right)^j, \tag{5.44}$$

$$= \sum_{n=0}^{N} x^n S(N,n), \tag{5.45}$$

where

$$S(N,n) \equiv \frac{a^{N-n}b^n}{2^{N-1}} \sum_{j=0}^{n} (-1)^j T(N,j) \binom{N-2j}{n-j} \left(\frac{4g}{ab}\right)^j. \tag{5.46}$$

This results provides an explicit form for the amplitudes of the permutation symmetric projected ansatz

$$\mathbb{P}\left|\psi_t\right\rangle = \frac{1}{M} \sum_{n=0}^{N} \binom{N}{n}^{-1/2} S(N,n) \left|n\right\rangle. \tag{5.47}$$

The parity transformation swaps $+$ and $-$ spin states, or equivalently, maps $|n\rangle$ to $|N-n\rangle$. This observation allows us to restore the parity symmetry in $\mathbb{P}\left|\psi_t\right\rangle$ above. The resulting (unnormalised) permutation and parity invariant ansatz reads

$$\left|\tilde{\psi}_s\right\rangle = \frac{1}{M} \sum_{n=0}^{N} \binom{N}{n}^{-\frac{1}{2}} P(N,n) \left|n\right\rangle \tag{5.48}$$

where

$$P(N,n) \equiv \frac{1}{2}(S(N,n) + S(N,N-n)). \tag{5.49}$$

Interestingly, $T(N,j)$ in Eq. (5.43) turns out to be the Riordan array [135] which obeys the recurrence relation

$$T(N,j) = 2T(N-1,j) + T(N-2,j-1). \tag{5.50}$$

https://scholar.sun.ac.za

Using this result alongside the binomial additive identity $\binom{N}{k} = \binom{N-1}{k} + \binom{N-1}{k-1}$ we can derive a recurrence relation for the $S(N,n)$ amplitudes as

$$S(N,n) = \frac{a^{N-n}b^n}{2^{N-1}} \sum_{j=0}^{n} (-1)^j T(N,j) \binom{N-2j}{n-j} \left(\frac{4g}{ab}\right)^j \tag{5.51}$$

$$= \frac{a^{N-n}b^n}{2^{N-1}} \sum_{j=0}^{n} \Big(2T(N-1,j) + T(N-2,j-1)\Big) \left[\binom{N-1-2j}{n-j} + \binom{N-1-2j}{n-1-j}\right] (-1)^j \left(\frac{4g}{ab}\right)^j \tag{5.52}$$

$$= aS(N-1,n) + bS(N-1,n-1) + \frac{a^{N-n}b^n}{2^{N-1}} \sum_{j=0}^{n} T(N-2,j-1)\binom{N-2j}{n-j}(-1)^j \left(\frac{4g}{ab}\right)^j \tag{5.53}$$

$$= aS(N-1,n) + bS(N-1,n-1) - sS(N-2,n-1) \tag{5.54}$$

where $S(0,0) = 2, S(1,0) = a, S(1,1) = b$ and $S(N,n) = 0$ if $N < 0$ or $n < 0$ generates the triangle.

5.2 The quantum transverse-field Ising model

The Hamiltonian for the TFIM with N spin-$\frac{1}{2}$ particles on a periodic chain is

$$H = -\frac{J}{4} \sum_{i=0}^{N-1} Z_i Z_{i+1} - \frac{h}{2} \sum_{i=0}^{N-1} X_i, \tag{5.55}$$

with J and h again the interaction and external field strengths. We set $J = 1$ as before. For the LMG model it was seen that the mean-field ansatz with $\theta = 0$ in Eq. (5.1) was sufficient for correctly predicting the critical value of the external field strength and for calculating the order parameter M_{rms} in the thermodynamic limit. For the TFIM the situation is quite different. Here, even in the thermodynamic limit, the $C^\theta R^\theta$ rotations play a crucial role in introducing correlations between spins, and are essential for shifting the estimate for the critical field strength closer to its true value. Using the ansatz in Eq. (5.7) together with Eqs. (5.8) - (5.10) we find the energy per spin in the thermodynamic limit to be

$$\lim_{N\to\infty} \frac{\langle H \rangle}{N} = -\frac{s-t}{4(st-1)} \left((s-t) + 2hc^2\right) - \frac{1}{4}. \tag{5.56}$$

which is a function of θ and ϕ via Eqs. (5.13) and (5.14). By minimising this expression with respect to θ and ϕ we identify a critical field strength of

$$h_c = \frac{1+\sqrt{2}}{4} \approx 0.604, \tag{5.57}$$

https://scholar.sun.ac.za

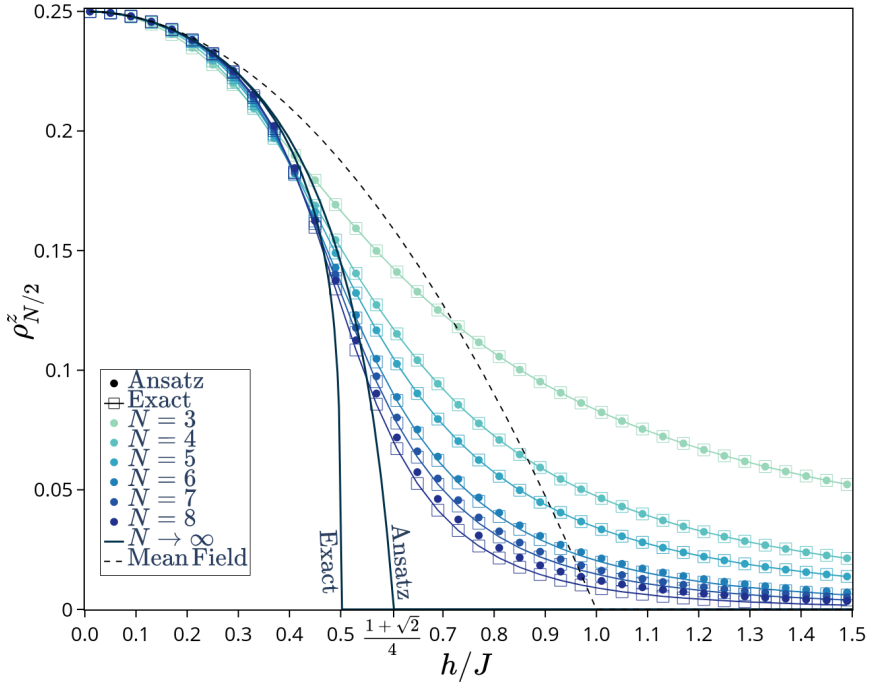


Figure 5.4: The long-range correlation function $\rho^z_{N/2}$ versus h/J for the TFIM. Results are shown for the symmetrised ansatz (finite N, $N \to \infty$), together with exact values and the mean-field prediction.

above which the magnetisation $\langle Z_i \rangle$ in Eq. (5.9) vanishes. See Section 5.2.1 for details of this calculation. This estimate for h_c is indeed closer to the exact value $h_c^{\text{ex}} = 0.5$ when compared to the mean-field result of $h_c^{\text{mf}} = 1$, which would follow from setting $\theta = 0$ and only varying ϕ. While the TFIM Hamiltonian lacks the permutation symmetry of the LMG model, it retains the parity symmetry. We again restore this symmetry by projecting the ansatz in Eq. (5.7) onto the positive symmetry subspace to produce a modified ansatz $|\psi_p\rangle$. See Section 4.5. Using $|\psi_p\rangle$ we calculate the long-range correlation function

$$\rho^z_{N/2} = \frac{1}{4} \langle \psi_p | Z_0 Z_{N/2} | \psi_p \rangle \,, \tag{5.58}$$

which serves as an order parameter for characterising the model's two phases. Fig. 5.4 shows the result this calculation for various system sizes. While our ansatz-based result matches the exact one closely for small N, it begins to deviate from it as N increases. This is to be expected due to the error in the ansatz's prediction of the critical field strength. The mean-field result, with its prediction of $h_c^{\text{mf}} = 1$, is also shown. Fig. 5.5 shows the relative error in the ground-state

https://scholar.sun.ac.za

energy, calculated using $|\psi_p\rangle$, as a function of N for different field strengths h. Again, for small N we essentially match the exact solution, but from $N = 6$ the relative error grows up to about 3×10^{-3}. The mean-field prediction is also shown, and we see that our ansatz improves upon it for all system sizes.

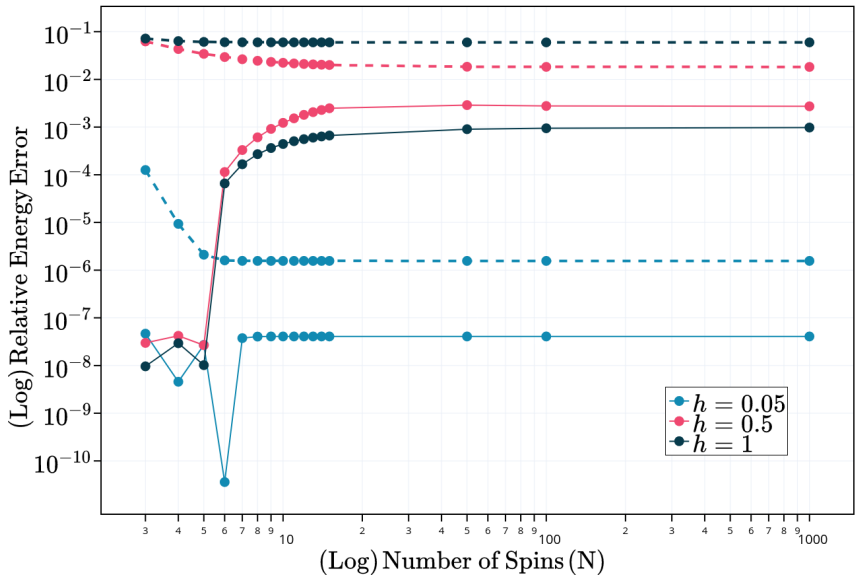


Figure 5.5: The relative error in the ground-state energy for the TFIM as a function of number of spins N for different field strengths h. Results obtained using the $|\psi_p\rangle$ ansatz (solid lines) are shown with those of mean-field theory (dashed lines).

5.2.1 Energy minimisation

For the Ising model the energy expectation value in the thermodynamic limit is given in Eq. (5.56). Its derivatives with respect to the two angles, written again in terms of $s = \sin(\theta)$ and $t = \sin(\theta + \phi)$ are:

$$\partial_\theta \langle H\rangle/N = \frac{\sqrt{1-s^2}\left(h\left(-2\left(s^2+1\right)t^2 + 4\left(s^2+1\right)st - 6s^2 + 2\right) - \left(s^2+2\right)t + 2s + t^3\right)}{4(st-1)^2}, \tag{5.59}$$

$$\partial_\phi \langle H\rangle/N = \frac{\sqrt{1-t^2}\left((s-t)(s(s+t)-2) - 2h\left(s^2-1\right)^2\right)}{4(st-1)^2}. \tag{5.60}$$

https://scholar.sun.ac.za

Setting these to zero leads to the following solutions for the stationary points:

1. For $h \in \mathbb{R}$: $(s,t) = (1,1)$ and $(-1,-1)$ yielding $\langle H\rangle/N = -1/4$
2. For $h \in \mathbb{R}$: $(s,t) = (1,-1)$ and $(-1,1)$ yielding $\langle H\rangle/N = 1/4$
3. For $|h| \geq 1/4$: $(s,t) = (1/(4h),1)$ yielding $\langle H\rangle/N = -h/2 - 1/(32h)$
4. For $|h| \geq 1/4$: $(s,t) = (1/(4h),-1)$ yielding $\langle H\rangle/N = h/2 + 1/(32h)$

The remaining solution satisfies

$$h(1+s^2)^2 - 2S = 0 \quad \text{and} \quad t = s(4/(1+s^2) - 1). \tag{5.61}$$

Since $|t| \leq 1$, the second equation places the restriction $|s| \leq \sqrt{2}-1$ on s. In turn, this limits, via the first equation, the range of h values for which this solution exists to $|h| \leq h_c = (\sqrt{2}+1)/4$. The first equation is a quartic polynomial in s, and yields four solutions. Only one of these is real and satisfies $|s| \leq \sqrt{2} - 1$. We denote this solution by $s^*(h)$ and the corresponding solution for t by $t^*(h)$. Comparing the values of $\langle H\rangle/N$ at these stationary points allows us to identify the values of s and t that minimise H:

$$s = \begin{cases} 1/(4h) & |h| > h_c \\ s^*(h) & |h| \leq h_c \end{cases} \quad \text{and} \quad t = \begin{cases} \text{sgn}(h) & |h| > h_c \\ t^*(h) & |h| \leq h_c \end{cases} \tag{5.62}$$

When $|h| \geq h_c$ we therefore have $|t| = 1$, which implies that $d = 0$, since $t^2 + d^2 = 1$, where $d = \cos(\theta + \phi)$. This causes the expectation value $\langle Z_i\rangle$ in Eq. (5.9) to vanish.

https://scholar.sun.ac.za

Chapter 6

Beyond Ground States

6.1 Quantum phase recognition

Using the AHL, we search for a quantum circuit ansatz to perform quantum phase recognition (QPR). We follow a similar search strategy as in Chapter 3 but with a different fitness function. Only two genetic operations were considered. First, a mutation which involves replacing a primitive within a motif with a randomly generated one. Second, crossover, which consists of combining two motifs end-to-end, if possible, or interweaving them otherwise. To facilitate comparison, we consider the same task and setup from Ref. [5], which introduces a quantum circuit ansatz called the quantum convolutional neural network (QCNN). The QCNN has also been implemented on a superconducting quantum processor [136]. The objective is to recognise a $\mathbb{Z}_2 \times \mathbb{Z}_2$ symmetry-protected topological (SPT) phase for a ground state that belongs to a family of cluster-Ising Hamiltonians [137]:

$$H = -J \sum_{i=1}^{N-2} Z_i X_{i+1} Z_{i+2} - h_1 \sum_{i=1}^{N} X_i - h_2 \sum_{i=1}^{N-1} X_i X_{i+1}. \tag{6.1}$$

Here, X_i, Z_i are Pauli operators acting on the spin at site i, and the SPT phase contains an $S = 1$ Haldane chain [138]. The ground state can belong to an SPT, paramagnetic or antiferromagnetic phase depending on the values of h_1, h_2, and J. Our goal is to identify an ansatz capable of distinguishing between SPT and other phases by measuring a single qubit. Following the approach in [5], we consider a system of $N = 15$ spins and train a circuit on 40 equally spaced points along $h_2 = 0$, where the ground state is known to be in the SPT phase when $h_1 \leq 1$. We also evaluate the circuit using the same sample complexity from

https://scholar.sun.ac.za

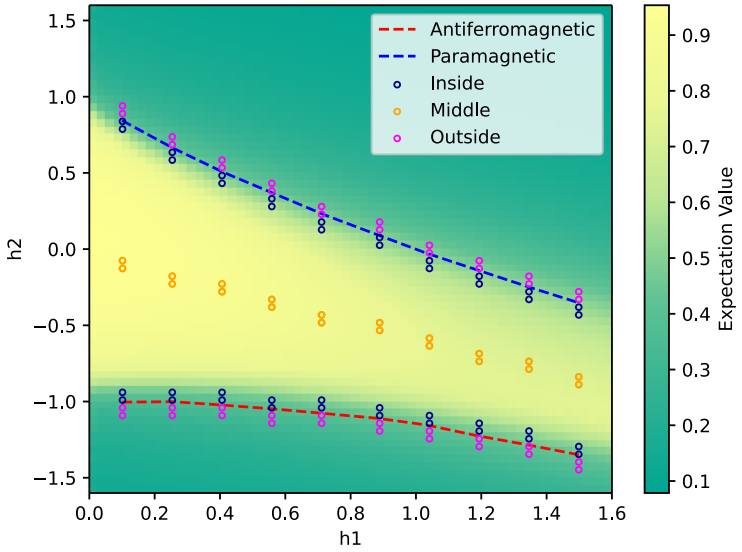


Figure 6.1: Expectation values for the circuit found via evolutionary search for a system of $N = 15$ spins. Points represent a test set of 64×64 ground states for various h_1 and h_2 values of the Hamiltonian, $J = 1$. The same colour scale as in Ref. [5] is used to facilitate comparison

Ref. [5]:

$$M_{\text{min}} = \frac{1.96^2}{(\arcsin\sqrt{p} - \sqrt{\arcsin p_0})} \tag{6.2}$$

where p represents the probability of measuring a non-zero expectation value and $p_0 = 0.5$. Equation (6.2) calculates the minimum number of measurements required to be 95% confident that $p \neq 0.5$, with p being the expectation value of the circuit U encoded with the ground state $|\psi_g\rangle$ transformed into a probability: $p = (\langle\psi_g| U |\psi_g\rangle + 1)/2$. Therefore, a well-performing QCNN will yield low values of M_{min} near the phase boundary for points within the SPT phase. We define the fitness of an architecture as a linear combination of the sample complexity values $M_{\text{in}}, M_{\text{middle}}$ for points in the SPT phase, and the mean squared error MSE_{out} for points outside the boundary. Fig. 6.1 illustrates the points considered for M_{in}, M_{middle} and MSE_{out}. During search, we assigned the majority of the weight to M_{in} as the goal is to develop a model that confidently identifies SPT phases near the boundary. To prevent a model from classifying all points as SPT, MSE_{out} is included, while M_{middle} ensures overall good performance. Finally, during search, we added a regularisation term for the number of parameters to find well-performing architectures with low computational complexity.

https://scholar.sun.ac.za

Metric	Reference	Found
Number of parameters	1308	**11**
Sample Complexity (Inside)	61.523	**36.079**
Sample Complexity (Middle)	**10.992**	13.253
MSE (Outside)	**0.164**	0.167

Table 6.1: Different performance metrics (lower is better) for the 15-qubit QCNN from [5] and the architecture found via evolutionary search. Sample complexity represents the expected number of measurements required to be 95% confident that the ground state is in the SPT phase (non-zero expectation value). Metrics are calculated on a set of points in the test set, where inside refers to SPT points near the phase boundary, outside to non-SPT points near the phase boundary and middle to points in between, as shown in Fig. 6.1.

Table 6.1 and Fig. 6.1 show the performance of the best ansatz found during search. The search algorithm identified an ansatz with only 11 parameters, in contrast to the 1308 parameters of the QCNN. For points in the SPT phase near the boundary, the sample complexity of the discovered ansatz ($M_{\text{in}} = 36.079$) is lower than that of the reference (61.523), resulting in 25 fewer measurements required on average. Although the reference ansatz exhibits slightly better sample complexity for points in the middle of the phase boundary ($M_{\text{middle}} = 10.992$) compared to the discovered ansatz ($M_{\text{middle}} = 13.253$), and a marginally lower MSE for points outside the phase boundary ($\text{MSE}_{\text{out}} = 0.164$ compared to $\text{MSE}_{\text{out}} = 0.167$), the improvements in M_{in} and the number of parameters are substantial and more advantageous. The discovered ansatz is shown in Fig. 6.2, and the phase diagram it generates is shown in Fig. 6.1. The search was conducted on a system equipped with two Intel Xeon E5-2640 processors (2.0 GHz) and 128 GB of RAM, and it took approximately 2 hours to discover the final ansatz (over 831 generations). We attribute this success to a well-defined search space, with the AHL aiming to simplify the process of creating such spaces.

6.2 Quantum algorithm synthesis

Using the AHL, we showed in Ref. [4] that the general N-qubit realisations of three well-known quantum algorithms could be rediscovered using a computer-driven search. These algorithms are the quantum Fourier transform (QFT), the Deutsch–Jozsa algorithm, and Grover's search algorithm. Amy Rouillard is the lead author of Ref. [4]. Both Amy and I conceived the idea. Amy implemented the search strategy, performed the numerical experiments, and analysed the results. Amy also prepared the manuscript with editorial assistance from me. The

https://scholar.sun.ac.za

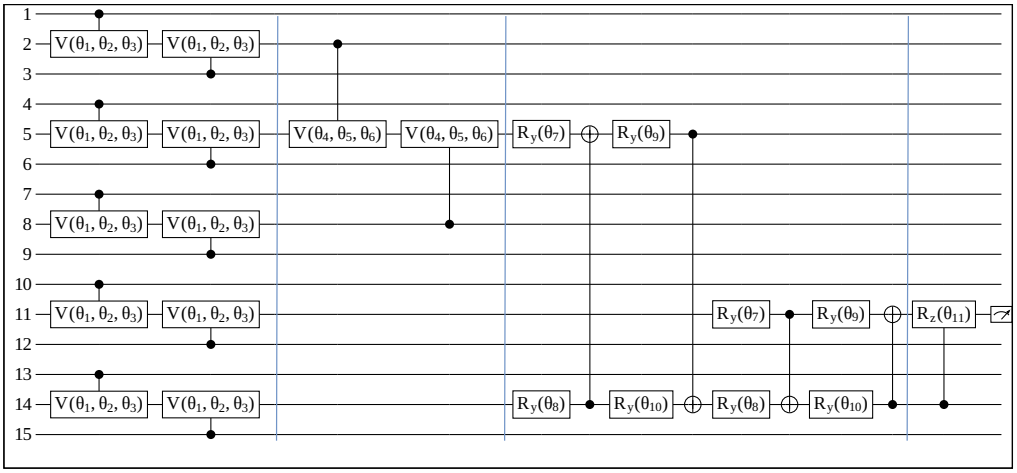


Figure 6.2: Circuit found by the evolutionary search after 831 generations. It begins with a mask primitive with base pattern $b^p = \texttt{101}$ and a two-qubit Gell-Mann unitary as its mapping. This pattern also appears in the original QCNN , as can be seen in Fig. 2.17(c), but here all V gates share weights. A second mask with different weights further coarse-grains the circuit, followed by a cycle (mapping from Fig. 6.8(c)) acting on the four remaining qubits. The circuit ends with a single CRZ from a mask before measurement.

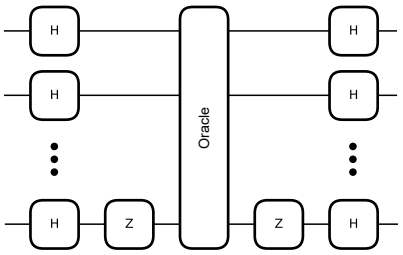
search strategy utilises the AHL, developed by me, alongside an evolutionary algorithm to find candidate algorithms. The evolutionary algorithm in Ref. [4] and the one I used in Chapter 3 are both based on those used in Neural Architecture Search [121]. Amy also contributed to the development of the AHL to make it more suitable for this task, particularly through the design of the pivot primitive and its implementation in the python package [1].

6.2.1 Results

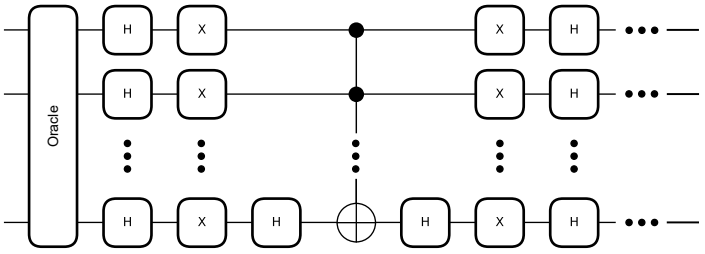
The task during evolutionary search is to find a motif $\mathcal{M}_{\text{alg}}$ that implements the desired N-qubit algorithm. This desired algorithm is specified through example input-output pairs for different circuit sizes. The details for setting up the training examples and the fitness functions can be found in Ref. [4], here I summarise the results and performance of the search. Fig. 6.3 shows, for each algorithm, the motif found via evolutionary search on the left and its corresponding N-qubit circuit representation on the right. We note the first result, in each case, the common circuit representation from literature [139] was automatically found. The motif descriptions on the left of Fig. 6.3 also illustrate how these well-known algorithms are compactly captured in the AHL. The compactness of each motif fur-

https://scholar.sun.ac.za


```
dj = (
Qcycle(H)
+ Qpivot(Z,"*1")
+ Oracle
+ Qpivot(Z,"*1")
+ Qcycle(H)
) × r
```




```
grover = (
Oracle
+ Qcycle(H)
+ Qcycle(X)
+ Qpivot(H,"*1")
+ Qpivot(MCX,"*1")
+ Qpivot(H,"*1")
+ Qcycle(X)
+ Qcycle(H)
) × r
```




```
qft = (
Qpivot(H,"1*")
+ Qpivot(CP(ϕ),"1*")
+ Qmask("1*")
) × r
```

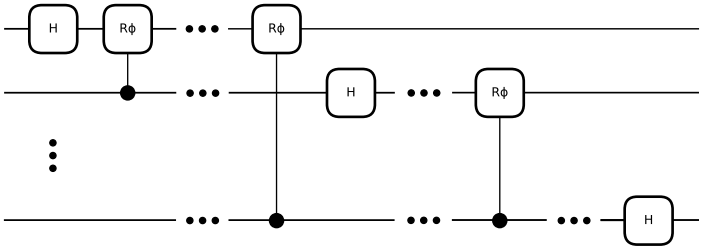


Figure 6.3: Two equivalent N-qubit representations of three learnt algorithms: Deutsch–Jozsa (top), Grover's search (middle), and the quantum Fourier transform (bottom). Left: The motifs built by the evolutionary algorithm. Right: circuit representations that match common presentations in the literature. The parameter r controls the number of repetitions of the structure, with the circuit size inferred from training. This figure is from Ref. [4].

ther enables us to more easily interpret aspects of each algorithm. For example, the symmetric structure of the diffusion term in Grover's search is immediately identifiable.

Fig. 6.4 on the left shows the maximum fitness in the population for every generation in a successful Grover's search experiment. On the right is a heat map of the fitness for the top 200 individuals, decreasing in fitness toward the right, for each generation in the same experiment. The left figure shows how the fitness

https://scholar.sun.ac.za

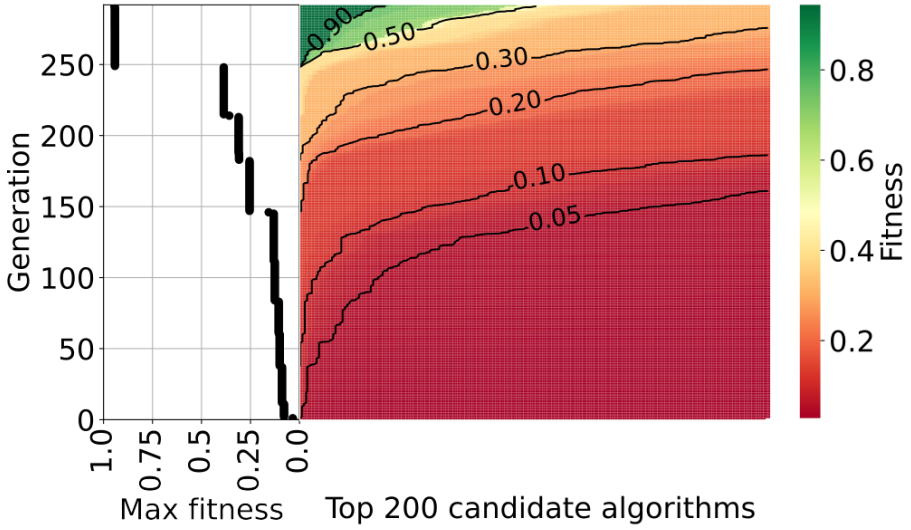


Figure 6.4: The evolution of the fitness scores over 292 generations of a successful experiment in which Grover's search was rediscovered. The maximum fitness of the population at each generation of the evolutionary search (left) increases over time with periods of plateau. The heat map and contour plot (right) show the fitness of the best performing 200 candidate algorithms sorted by fitness, decreasing from left to right. Left and right share the same y-axis. This figure is from Ref. [4].

increases in discrete jumps with increasing generations. Grover's search algorithm is found at generation 292 which corresponds to a fitness of 1.0. The best fitness in the pool of individuals steadily increases about every 40 generations up until generation 250 where there is a large jump to a fitness of 0.9. This behaviour is expected for the evolutionary algorithm used here and the one used in Chapter 3. On the right of Fig. 6.4 the heat map shows the distribution of fitness for the top 200 individuals with increasing generations. Contour lines of equal fitness is also shown. We can see that after about 160 generations all top 200 individuals had a fitness greater than 0.05 and about 20 generations later all of them had a fitness greater than 0.1. The flatter a contour line the faster individuals with a higher fitness entered the top 200 pool. The 0.5 and 0.9 contour lines meet at the fittest individual around generation 250. This is because of the large jump in fitness, skipping over 0.5, around that generation. We can understand the regions surrounding these two contour lines by looking at Fig. 6.5. Shown here is the path of genetic operations that led to Grover's search algorithm as a directed graph. Here nodes are coloured by the same colour scale as the fitness heat map of Fig. 6.4 and edges indicate the type of genetic operation. In particular, Fig. 6.5 (c) corresponds to the large jump in fitness observed in Fig. 6.4.

https://scholar.sun.ac.za

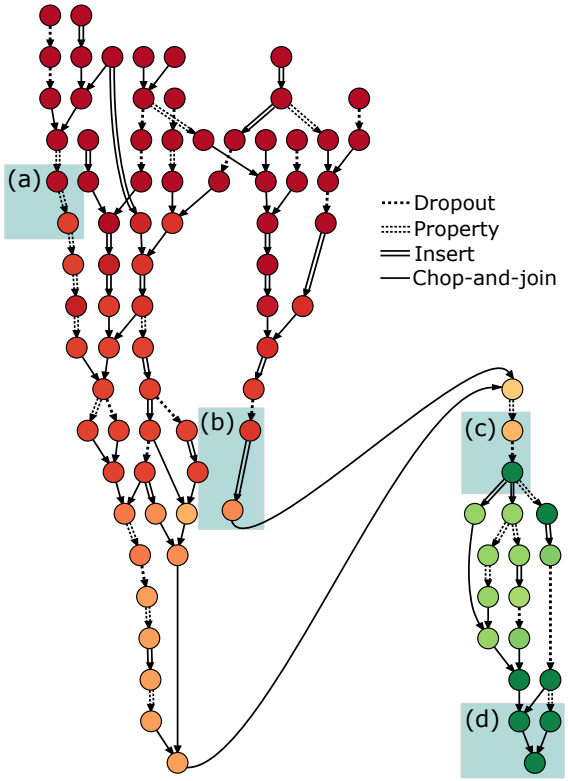


Figure 6.5: A directed graph of a subset of the population that evolved into an instance of Grover's search algorithm. Each node represents a candidate algorithm from the same population shown in Fig. 6.4. The colour of each node corresponds to the fitness, matching the scale in Fig. 6.4. Each mutation type is represented by a different edge. This figure is from Ref. [4].

We see that the individual with fitness less than 0.5 mutated into an individual with a fitness greater than 0.9. After this mutation, we see that the high fitness individual had offspring with lower fitness, between 0.5 and 0.9. Therefore in the heat map of Fig. 6.4, some of the indiduals below the 0.9 and above the 0.5 contours came from an individual with a fitness greater than 0.9.

Finally, in Fig. 6.6 we compare the total number of evaluated individuals for each learning task with different search space configurations. These configurations are further detailed in Ref. [4]. In brief, for motifs consisting of N_p primitives the smallest space consists of 13^{N_p} possible motifs and the huge space of 646^{N_p} motifs. We define the size of the search space by setting N_p equal to the minimum number of motifs, that we know of, to implement the algorithm. This is equivalent to the worst case number of evaluations that a brute force search, with

https://scholar.sun.ac.za

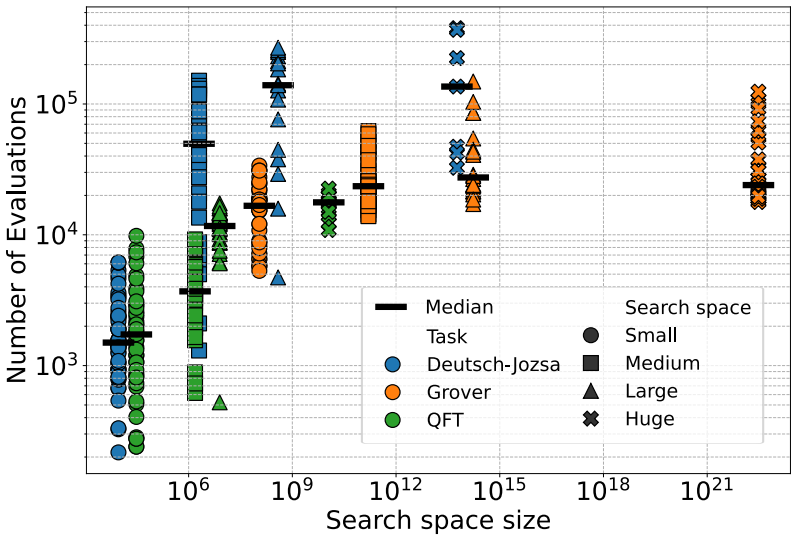


Figure 6.6: The number of evaluations required to successfully design three different quantum algorithms. For each task, four search spaces of increasing size were tested. The median number of evaluations is indicated with a horizontal bar. This figure is from Ref. [4].

motifs, would need to perform. In all cases, the number of evaluations is a small fraction of the total search space size. This indicates the efficiency of the search algorithm. In particular, we see that for Grover's search algorithm we required a median number of evaluations on the order of 10^4, whereas a worst case brute force requires 10^{22} evaluations. It is also worth noting that our definition for search space size assumes the use of the AHL. This already brings about a major reduction in the search space compared to a gate sequence description.

6.3 Quantum machine learning

Machine learning using parameterised quantum circuits (PQCs) is a possible application for quantum computing [140–143]. Here, a quantum computer executes the circuit, and a classical computer optimises its parameters. Two key factors must be considered when using PQCs for machine learning: the method of data encoding (feature map) [144] and the choice of a quantum circuit [145]. The typical approach is to keep the circuit architecture (gate layout) fixed and to optimise continuous parameters such as rotation angles. Optimising architecture, often called variable-structure ansatz in the literature, is generally not the focus because of its computational complexity [141]. However, the architecture of a circuit can improve its expressive power and the effectiveness of initialisa-

https://scholar.sun.ac.za

tion techniques [146]. Therefore, we look towards NAS to optimise architecture in a quantum circuit setting. This approach, sometimes referred to as quantum architecture search (QAS) [54, 56], has shown promising results for the variational quantum eigensolver (VQE) [35, 38, 147, 148], the quantum approximate optimisation algorithm (QAOA) [53, 149] and general architecture search [54–56, 150, 151]. However, these approaches are often task-specific or impose additional constraints, such as circuit topology or allowable gates, to make them computationally feasible. To the best of the authors' knowledge, the use of a domain-specific language (such as the AHL) to reduce the search space size and capture abstract circuit structures has not yet been explored. In Section 6.3.1, we show how the AHL combined with random search can be used to find quantum circuit structures that outperform manually designed ones. This section also highlights the importance of the structure of a circuit in quantum machine learning. Section 6.3.1 is based on the work we presented in Ref. [2]. Then in Section 6.3.2 we compare the performance of a QCNN designed with the AHL to a quantum kernel-assisted support vector machine (QSVM) on a pulsar classification task. We presented the work from Section 6.3.2 in Ref. [152].

6.3.1 Music genre classification

Here we use the AHL to illustrate the importance of the choice of circuit architecture for parameterised quantum circuit models. Consider the machine learning pipeline for classifying musical genres from audio signals, seen in Fig. 6.7. In our early attempts to improve model performance on this task we found that altering the circuit architecture had the greatest impact. By altering architecture we mean the following: given a quantum circuit that consists of n unitary gates, an altered architecture uses the same n gates, rearranged differently on the circuit. The types of rearrangements may be changing which qubits the gates act upon, altering the order of gate occurrences, or adjusting larger architectural motifs, such as masking specific qubits while leaving others available for subsequent gates and so on. Specifically, this pipeline starts with a 30-second recording of a song (a), and we transform it in two ways. The first is tabular form (b), derived from standard digital signal processing statistics of the audio signal. The second is image form (c), constructed using a Mel frequency spectrogram. Both datasets are benchmarked separately, each with its own preprocessing and encoding. For the tabular data, we test Principal Component Analysis (PCA) and tree-based feature selection before encoding it in a quantum state using either qubit, IQP, or amplitude encoding. Once encoded, we choose two-qubit unitaries U_m and V_m as the mappings for the cycle and mask primitives, as shown in (d). Of all the components in this pipeline, altering the architecture had the greatest impact on

https://scholar.sun.ac.za

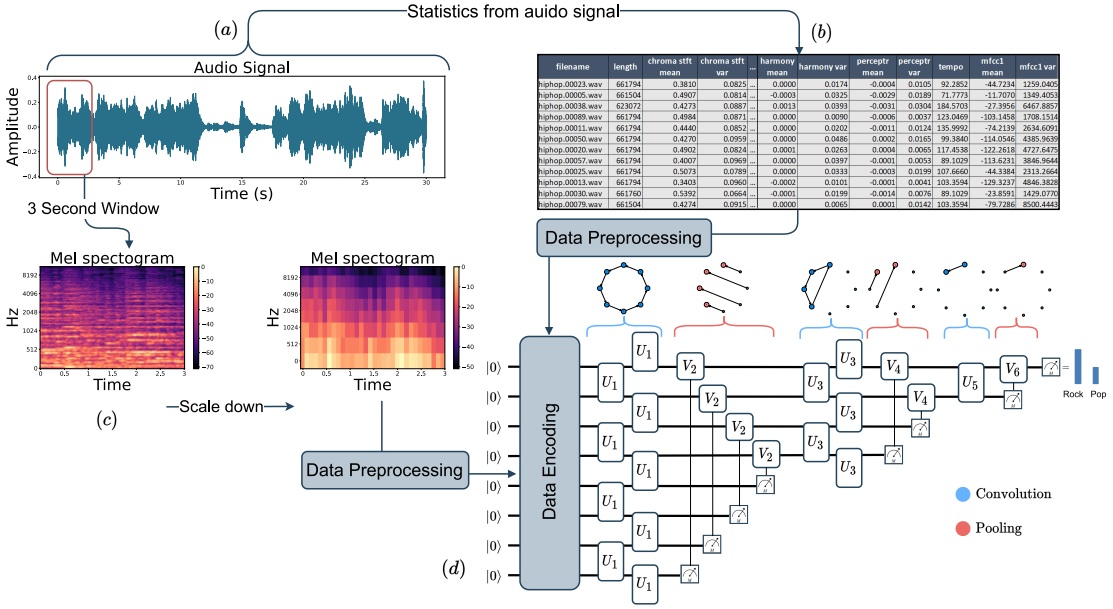


Figure 6.7: The machine learning pipeline we implemented for music genre classification. Given an audio signal of a song (a), we generate two representations of data: tabular (b) and image (c). Each form has data preprocessing applied before being encoded into a quantum state (d). The circuit shown in (d) favours Principal Component Analysis (PCA) because qubits are pooled from bottom to top and principal components are encoded from top to bottom. This architecture is an instance of the reverse binary tree family that we generated with the AHL.

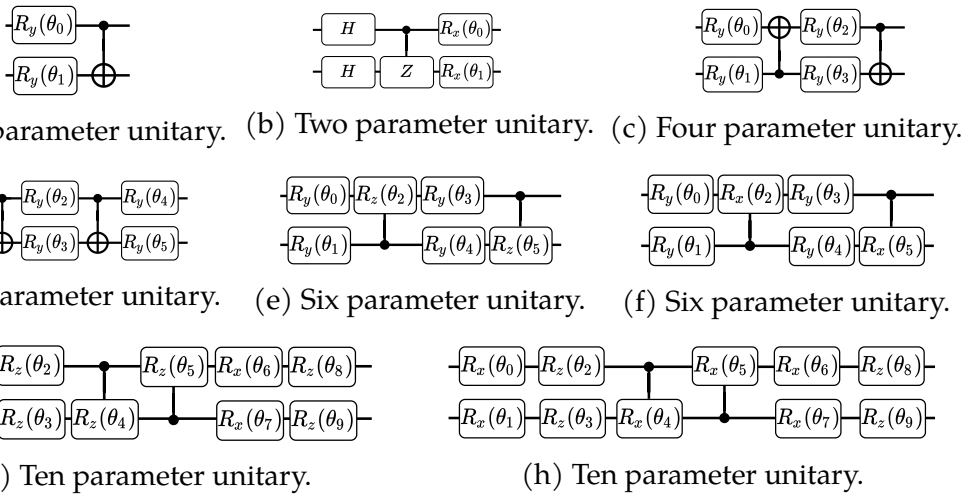
model performance.

We illustrate the impact of architecture on model performance by comparing the fixed architecture from the experiments of Hur et al. [153] to other architectures in the same family while keeping all other components of the pipeline the same. The architecture from Ref. [153] is shown in Fig. 6.9. This architecture is an instance of the motif

$$\mathcal{M}_r = (\mathcal{P}_c + \mathcal{P}_m) \times \log_2(N), \tag{6.3}$$

where N denotes the number of qubits. The parameter values for the cycle and mask in this motif that reproduces the specific architecture from Ref. [153] are shown in Tab. 6.2 and 6.3. The cycle also has an edge order of $(1, 3, 5, 7, 2, 4, 6, 8)$. The architecture family we compare this to is the same motif $\mathcal{M}_r$ from Eq. (6.3) but with different parameter values of h_1, b^p and h_1^c. In Tab. 6.2 and 6.3 these parameters are highlighted in bold and each alteration is specified as the 3-tuple (h_1, b^p, h_1^c). The resulting search space is a family of reverse binary tree architec-

https://scholar.sun.ac.za



(a) Two parameter unitary. (b) Two parameter unitary. (c) Four parameter unitary.

(d) Six parameter unitary. (e) Six parameter unitary. (f) Six parameter unitary.

(g) Ten parameter unitary. (h) Ten parameter unitary.

Figure 6.8: The different mappings used for the cycle primitive across all experiments. These are the same set of mappings used in the benchmarks of Ref. [153]. It is based on previous studies that explore the expressibility and entangling capability of parameterised circuits [154], hierarchical quantum classifiers [123], and extensions to the VQE [155].

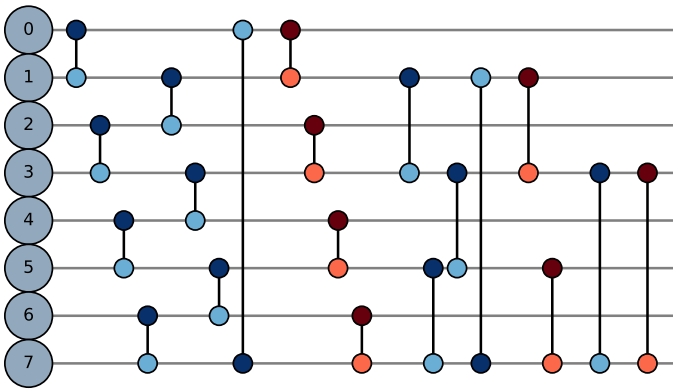
tures, which is further detailed in Section 6.3.1. To evaluate architecture performance, we use the country vs rock genre pair, which proved to be one of the most difficult classification tasks from the 45 possibilities. We compare eight unitary mappings with different levels of complexity, shown in Fig. 6.8. Here the mapping refers to the two-qubit unitary used for the cycle primitive. For example, the model in Fig. 6.7 (d) is described by the alteration $(1, *!, 0)$ and specifying the mapping from 6.8a corresponds to U_1, U_2 and U_3 being the gates from Fig. 6.8a.

Cycle: $\mathcal{P}_c$					
Parameter	k	δ_b	h_0	$\mathbf{h_1}$	h_2
Value	2	0	0	**1**	1

Table 6.2: The parameter values of the cycle that determine the architecture in Ref. [153]

Table 6.4 shows the results of these comparisons. The reference architecture is as described above and the alteration found via random search. Each value in the architecture column represents the average model accuracy and standard deviation from 30 separate trained instances on the same held-out test set. We note

https://scholar.sun.ac.za



$$\mathcal{G}_h = \mathcal{P}_0(8) + (\mathcal{P}_c + \mathcal{P}_m) \times \mathbf{3}$$

Figure 6.9: The architecture from Hur et al. [153] shown as a hypergraph made with our AHL.

Mask: $\mathcal{P}_m$											
Parameter	k	δ_b	$\mathbf{b^p}$	b^e	h_0^a	h_1^a	h_2^a	h_0^b	h_1^b	h_2^b	$\mathbf{h_1^c}$
Value	2	0	10	10	0	1	1	0	1	1	**0**

Table 6.3: The parameter values of the mask that determine the architecture in Ref. [153]

the first important result, through random search we improved the performance for all mappings considered, in one case, by 18.05%. The second important result is that altering architecture can improve the model's performance without increasing its complexity. For instance, the best-performing model for the reference architecture is with the mapping from Fig. 6.8g, which has an average accuracy of 73.24%. However, this mapping causes the model to have $10 \times 3 = 30$ parameters. In contrast, by altering the architecture with the simplest mapping in Fig. 6.8a, the model outperformed the best reference model with an average accuracy of 75.14% while only having $3 \times 2 = 6$ parameters. Another interesting result is for the mapping from Fig. 6.8c, the reference architecture could only obtain an average accuracy of 52.69% indicating its inability to find any kind of local minimum during training, suggesting a barren plateau. But, the altered architecture was able to find a local minimum and improve the average accuracy by 18.05%. In Tab. 6.4 there is an observed decrease in performance improvement for larger parameter ansatzes. This is most likely a consequence of our chosen random search duration. We only conducted random search for approximately

https://scholar.sun.ac.za

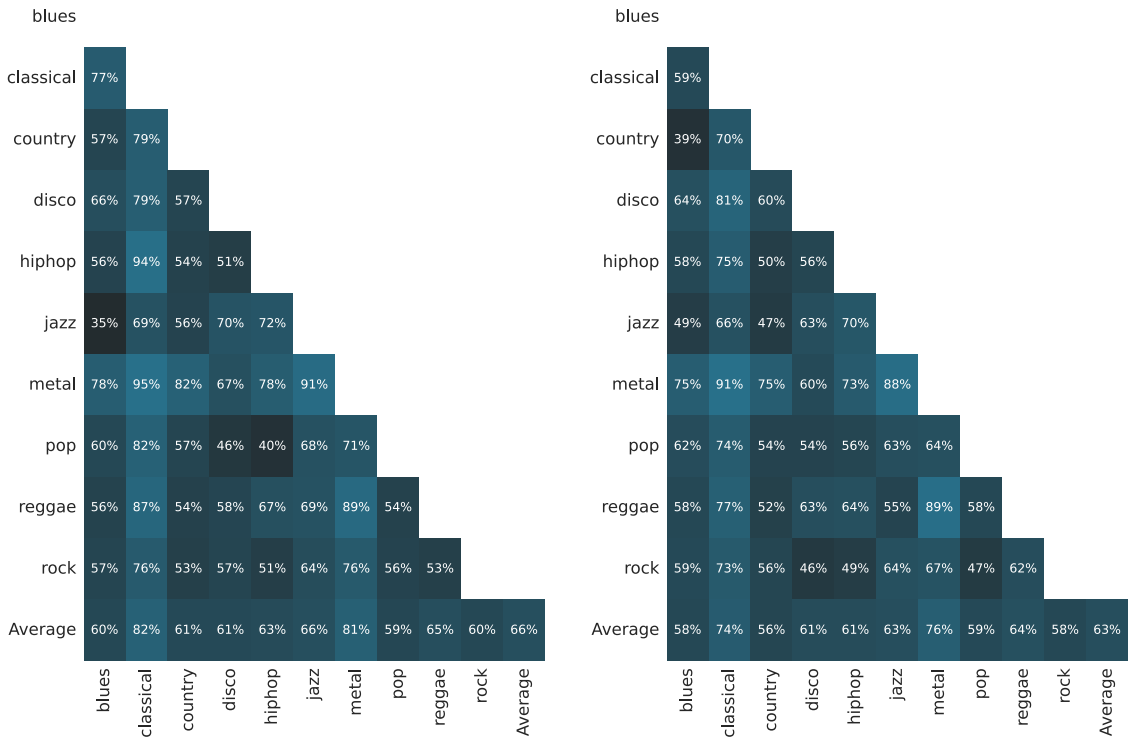
Architecture vs Mapping					
Mapping, # Params		Architecture Reference	Architecture New alteration	Δ	Alteration (h_1, b^p, h_1^c)
6.8a,	6	65.37 ± 2.8	**75.14** ± 1.7	$+9.77$	$(6, !*, 2)$
6.8b,	6	56.34 ± 3.2	**70.46** ± 1.0	$+14.12$	$(1, 10, 3)$
6.8c,	12	52.69 ± 3.8	**70.74** ± 1.3	$+18.05$	$(1, 10, 0)$
6.8d,	18	67.13 ± 1.5	**77.87** ± 2.4	$+9.87$	$(1, !*!, 2)$
6.8e,	18	67.87 ± 2.5	**73.61** ± 1.8	$+5.74$	$(6, !*, 0)$
6.8f,	18	69.21 ± 2.6	**74.80** ± 2.8	$+5.59$	$(1, !*, 3)$
6.8g,	30	73.24 ± 2.9	**79.47** ± 2.2	$+6.23$	$(2, !*, 1)$
6.8h,	30	69.35 ± 4.1	**71.71** ± 3.7	$+2.36$	$(2, !*, 1)$

Table 6.4: The average accuracy and standard deviation of the country vs rock genre pair on a held-out test set after 30 separately trained instances. All architectures come from the family of reverse binary trees, generated by the motif $\mathcal{M}_r$ in Eq. (6.4). The "reference" architecture is the one used in the experiments of Ref. [153] and the "alteration" was found through random search within the same family. The mappings refer to the associated gates used for the cycle primitive. These come from Ref. [153], which is based on previous studies that benchmarked PQCs [123, 154, 155].

2 hours on an i7-1165G7 processor for each ansatz. Consequently, for higher parameter ansatzes, which correspond to longer training times, the search space was less explored. Therefore the observed improvements are all lower bounds for the potential performance increase from altering architecture. We anticipate that significantly better architectures may still exist within the space.

The mapping from Fig. 6.8a only contains 6 variational parameters leading to small training times. Because of this, we were able to fully explore the search space of 168 architectures. In Section 6.3.1 we discuss how the size of the family can easily be increased or decreased. Table 6.5 shows the performance of each architecture in this space. Each value represents the average accuracy of five trained instances on the country vs rock genre pair. The overall accuracy of the whole space is 63.11%, indicating that the reference architecture from table 6.4 was close to the mean performance. The best-performing architecture in this space is $(h_1, b^p, h_1^c) = (6, !*, 2)$, with an average accuracy of 75.93%. This is the alteration from Table 6.4 discovered through random search. It seems that the combination of $b^p = !*$ and $h_1 = 6$ performs particularly well for this task, with an average accuracy of 72.52%. In general, it appears that the cycle stride h_1 and mask base pattern b^p have the most significant impact on performance. It is also

https://scholar.sun.ac.za



(a) $b^p = *!$ mask base pattern results

(b) $b^p = 10$ mask base pattern results

Figure 6.10: Difference in performance between two mask base patterns on low-resolution image data. Accuracies for all genre pairs are shown.

worth noting that cycle strides of $h_1 = 3, 4, 5$ performed poorly compared to the other values. The range of performance in this space goes from a minimum of 43.75% to a maximum of 75.93%, demonstrating the potential impact of architectural choices on model performance.

Finally, we compared the performance of two different architectures on the image data across all genres. This time, we used the mapping from Fig. 6.8g to compare the mask base patterns $b^p = *!$ and $b^p = 10$. The results are shown in Figs. 6.10a and 6.10b. The image data is a low-resolution ($8 \times 32 = 256 = 2^8$ pixels) spectrogram of the audio signal. We did not expect high accuracy from this data, but were interested in the variation of performance for different architectures. Both Figs. 6.10a and 6.10b show the difficulty of some genre pairs. Interestingly, the $b^p = *!$ mask base pattern outperformed the pattern $b^p = 10$ on almost all genre pairs. If we focus on the genre pairs that the models were able to classify, we see that $b^p = *!$ had 14 models that achieved an accuracy above 75%, compared to

https://scholar.sun.ac.za

the 5 of $b^p = \texttt{10}$. We also note that the image data had no PCA or tree-based feature selection applied to it, and the $b^p = \texttt{*!}$ filter was still favoured. A similar result was obtained with ansatz 6.8a. This shows that the choice of architecture impacts performance even on low-resolution data.

Search Space

The family of architectures we considered for random search all have the structure of reverse binary trees. This is primarily due to the base patterns we consider for the mask. Recall that the motif describing this family of architectures is given by

$$\mathcal{M}_r = (\mathcal{P}_c + \mathcal{P}_m) \times \log_2(N), \tag{6.4}$$

where N denotes the number of qubits. The parameter values that determine the cycle and mask are given by Tabs. 6.2 and 6.3. Three of these parameters are varied, the cycle stride h_1, the mask base pattern b^p and the mask stride h_1^c. The values for both stride parameters we consider are: $h_1 \in 1, \dots, N$ and $h_1^c \in 0, \dots, N$. We only consider base patterns that mask half the available qubits. The options we considered are given by $b^p \in \{\texttt{01}, \texttt{10}, \texttt{*!}, \texttt{!*}, \texttt{*!*}, \texttt{!*!}\}$. For any $N = 2^m$ with $m > 0$ these base patterns lead to a quantum circuit with a reverse binary tree structure. This is because the cycle step is $h_2 = 1$, meaning each qubit is connected to another qubit, and the mask will halve the available qubits for the subsequent primitive. This is repeated m times, until one qubit remains. These possible combinations of N, h_1, h_1^c, b^p represent the search space (family size). The complexity of the model (in terms of the number of unitary operations) then scales linearly with the number of qubits N. Specifically, N qubits result in $3N-2$ unitary operations [1].

In Fig. 6.11, we show how the three parameters influence the architecture instance of the reverse binary tree family. Fig. 6.11 (a) shows the repeated use of $b^p = \texttt{*!}$ for the mask. This particular pattern is useful for data preprocessing techniques such as principal component analysis (PCA), since PCA introduces an order of importance to the features used in the model. Typically, the first principal component (which explains the most variance) is encoded on the first qubit, the second principal component on the second qubit and so on. Therefore, it makes sense to mask the last qubits and leave the first qubits in the model for as long as possible.

[1] This is because of the geometric series: $N(\frac{1}{2^0} + \frac{1}{2^1} + \cdots + \frac{1}{2^{\log_2 N-1}}) + N(\frac{1}{2^1} + \frac{1}{2^2} + \cdots + \frac{1}{2^{\log_2 N-1}})$. Where the first sum is for cycle mappings and the second for mask.

https://scholar.sun.ac.za

Performance across architecture search space								
	Cycle stride, h_1							
b^p, h_1^c	1	2	3	4	5	6	7	**Avg**
"10"	**67.01**	63.63	60.76	64.93	59.98	63.1	59.49	62.81
0	65.97	58.68	56.25	66.67	62.85	59.72	63.43	61.88
1	66.32	66.32	63.54	60.07	61.46	71.88	54.17	63.73
2	66.67	60.76	60.07	68.06	54.17	58.8	63.89	61.81
3	69.1	68.75	63.19	64.93	61.46	60.19	56.48	63.84
"*!*"	66.41	**71.96**	58.25	54.25	69.27	68.15	60.53	64.18
0	65.28	72.22	60.07	49.65	70.49	68.4	60.65	63.94
1	67.01	71.18	58.68	55.9	66.32	68.4	60.19	64.09
2	68.4	71.53	58.33	51.74	71.88	68.98	58.8	64.26
3	64.93	72.92	55.9	59.72	68.4	66.67	62.5	64.42
"!*"	62.85	61.63	59.38	59.03	51.56	**72.52**	72.45	62.22
0	66.67	67.01	56.94	61.46	52.08	71.18	73.61	63.79
1	59.03	62.15	52.78	57.99	52.08	71.18	73.61	60.8
2	63.19	63.19	63.19	60.76	51.74	**75.93**	71.76	63.51
3	62.5	54.17	64.58	55.9	50.35	72.69	70.83	60.79
"01"	61.11	**68.75**	63.37	62.76	64.67	60.52	57.99	62.96
0	60.76	71.88	63.19	58.33	63.54	59.38	57.87	62.29
1	63.54	67.36	64.58	63.54	64.24	62.5	59.26	63.73
2	60.42	70.14	64.58	65.97	69.1	58.8	56.94	64.16
3	59.72	65.62	61.11	63.19	61.81	61.11	57.87	61.65
"!*!"	60.68	65.8	65.54	57.12	62.15	59.83	**67.13**	62.51
0	67.36	59.72	71.88	54.17	67.01	60.07	70.37	64.15
1	53.47	69.79	62.15	56.25	61.11	58.33	70.83	61.49
2	57.99	70.83	60.07	61.11	59.03	59.26	66.67	62.07
3	63.89	62.85	68.06	56.94	61.46	61.57	60.65	62.29
"*!"	**70.05**	65.63	64.41	53.65	68.66	63.69	60.65	63.94
0	70.14	63.54	64.58	50	68.4	61.11	62.96	62.96
1	69.79	67.71	64.58	69.1	68.06	67.01	57.87	66.62
2	70.14	62.15	63.89	43.75	68.75	62.04	61.57	61.75
3	70.14	69.1	64.58	51.74	69.44	64.35	60.19	64.37
Avg	64.68	66.23	61.95	58.62	62.72	64.69	63.04	63.11

Table 6.5: Country vs Rock average accuracy within the reverse binary tree search space, all with the mapping from Fig. 6.8a. The cycle stride h_1 is shown on the horizontal axis and the combinations of mask base pattern b^p and stride h_1^c on the vertical. The best mask base pattern and stride combinations are presented in bold along with the overall best architecture $(h_1, b^p, h_1^c) = (6, !*, 2)$.

https://scholar.sun.ac.za

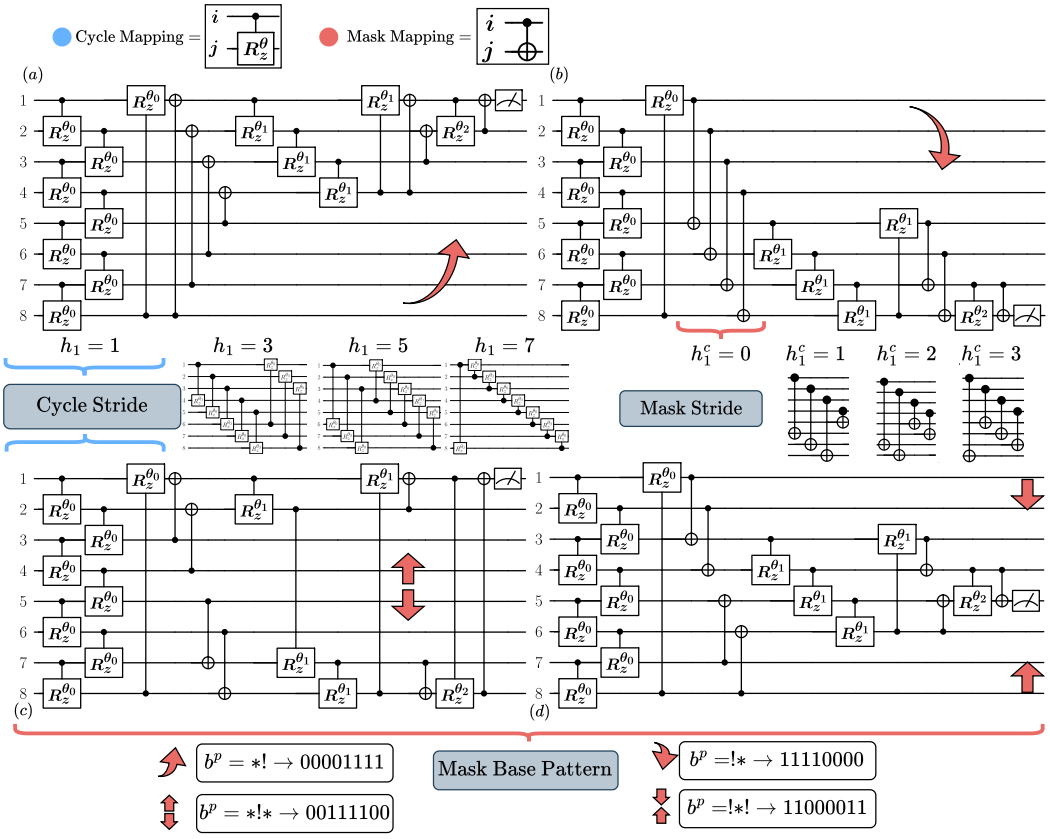


Figure 6.11: Examples of how the three hyperparameters influence the circuit architectures created by the motif from Eq. (6.4). These parameters are the cycle stride h_1, mask stride h_1^c and mask base pattern b^p. Controlled-R_z^θ gates are used for the cycle's mapping and CNOTs for the mask's mapping. The cycle stride h_1 determines how its associated gates are distributed across the circuit. The mask stride h_1^c determines how its associated gates are distributed across the circuit. For a given mask primitive, a portion of available qubits gets masked via controlled unitary operations and h_1^c dictates which controls are paired with which targets. The mask base pattern b^p specifies which qubits to mask. For example, in (d) the outside qubits are always masked, resulting in the middle qubit making it to the end of the circuit.

https://scholar.sun.ac.za

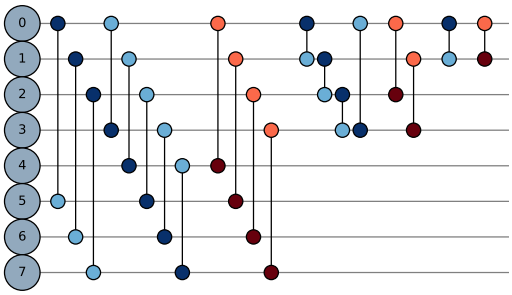


Figure 6.12: The QCNN ansatz used for pulsar classification. This particular architecture is an instance of the family of reverse binary trees discussed in Section 6.3.1. The base pattern for the mask is b^p = *!, which hides the bottom half of the circuit at each layer. The associated mapping for the cycle is shown in Fig. 6.8(a) and a CNOT was used for the mask.

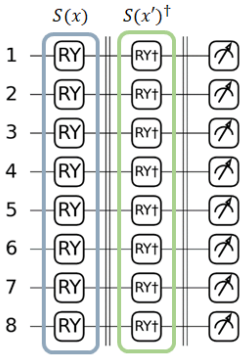


Figure 6.13: The QSVM circuit. We apply angle embedding (rotational encoding) with Y-rotation gates, followed by applying the complex conjugate transpose of this embedding using features from a separate feature vector as parameters. Measurement is done in the computational basis after a projector matrix was applied to help calculate the inner product without a SWAP test. This figure is from Ref. [152].

6.3.2 Pulsar classification

In Ref. [152], we benchmarked a particular QCNN ansatz from the family of reverse binary trees discussed in Section 6.3.1 against a quantum kernel-assisted support vector machine (QSVMs). We showed that both quantum machine learning models can accurately distinguish between pulsars and non-pulsars on the High Time Resolution Universe Survey (HTRU 2) dataset [156]. Donovan Slabbert is the lead author of Ref. [152]. Donovan performed the numerical experiments and analysis, illustrated the results, and wrote the manuscript. I championed the QCNN part of the project, and designed the circuit ansatz using the AHL.

The particular QCNN used in the study is described by $(h_1, b^p, h_1^c) = (5, *!, 0)$, using the convention discussed in Section 6.3.1. I show the hypergraph corresponding to the circuit in Fig. 6.12. The mapping used for the cycle is shown in Fig. 6.8(a). The mapping for the mask is a CNOT. The dataset consists of eight features, and we encoded each using angle embedding. While we did not exten-

https://scholar.sun.ac.za

sively explore the reverse binary tree space, we varied the cycle stride h_1 parameter. We found a spike in model performance for a stride value of five. Upon further investigation, we found a correlation between the first and fifth features that was previously unknown to us. This highlights one of the advantages of using the AHL in this setting, where comparing the performance across different structural parameters of the model led to qualitative insights into the data.

The details regarding the QSVM implementation can be found in Ref. [152], which is based on Ref. [157]. Fig. 6.13 shows the circuit used; it consists of two layers of Pauli-Y rotations, each based on a different feature vector $\mathbf{x}$. The circuit is used to compute the following kernel

$$\kappa(\mathbf{x}, \mathbf{x}') \equiv \left| \langle z, + |^{\otimes n} \left(\prod_{k=1}^{n} e^{-i(\frac{x_k - x_k'}{2})Y_k} \right) |z, +\rangle^{\otimes n} \right|^2, \tag{6.5}$$

$$= \prod_{k=1}^{n} \cos^2\left(\frac{x_k - x_k'}{2}\right). \tag{6.6}$$

Here $x_k \in \mathbf{x}$ denotes the value of the kth feature for the data point $\mathbf{x}$, and in our study we had $n = 8$ features. Eq. (6.6) is used as the kernel for a classical support vector machine. This particular kernel, consisting of the inner product of two product states, can always be efficiently computed classically. It has been shown that some quantum kernels are hard to compute classically [158], however, whether any such kernels are practically useful remains an open question [157].

The results are summarised in Fig. 6.14. All three models achieve high performance. Of the 400 test samples, the QSVM misclassified 20 data points, the QCNN 11 and the batched QCNN 12. The batched QCNN is explained in [152]. Of these misclassifications, most were false positives for the QSVM, and most were false negatives for the QCNN. Within this context, a false negative is more costly than a false positive, since we do not want the model to miss a potential pulsar. The QSVM does have three fewer false negatives than the QCNN on average, but this is too small a difference to make any conclusive statement. We can conclude that both models are effective at classifying this dataset.

https://scholar.sun.ac.za

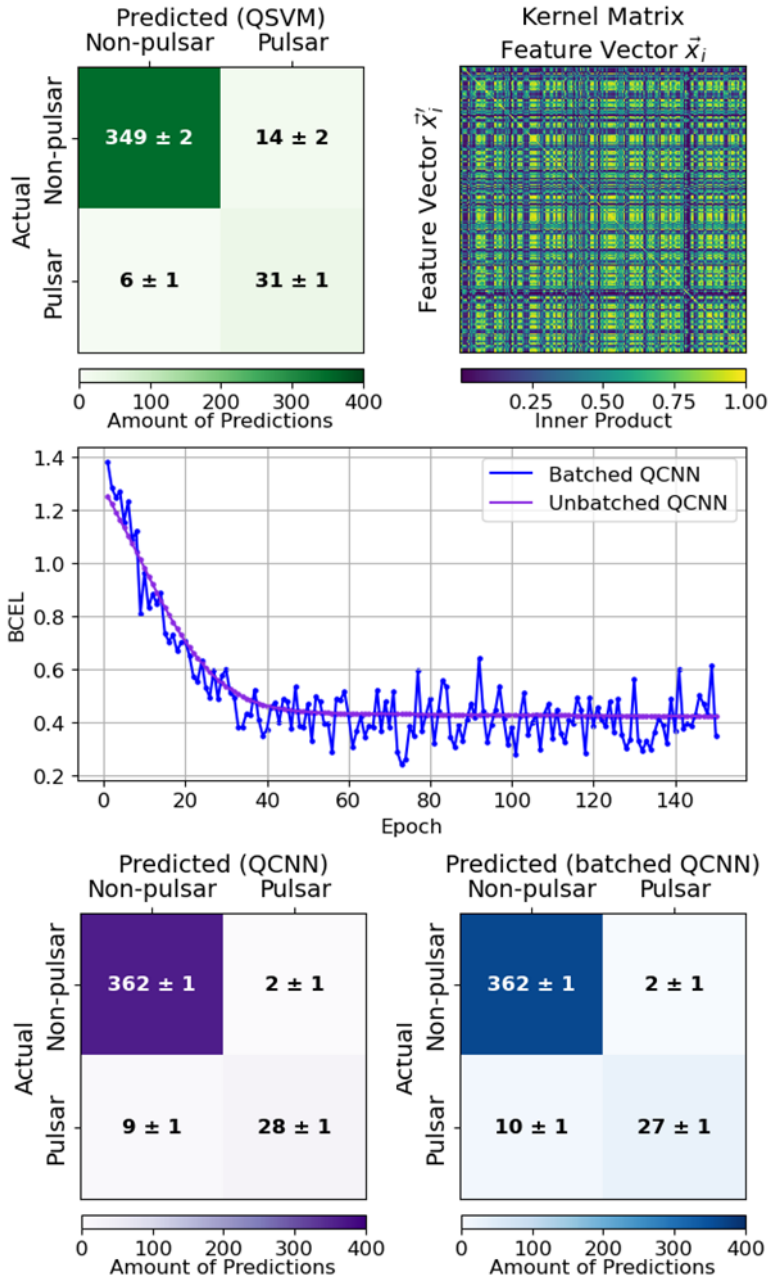


Figure 6.14: Confusion matrices for the QSVM (Green), QCNN (Purple) and batched QCNN (Blue). The results are averaged over six runs, and the standard deviation is provided. This figure is from Ref. [152].

https://scholar.sun.ac.za

Conclusion

In order to understand quantum many-body systems, it is necessary to find compressed representations of the relevant wavefunctions. These representations (ansatzes) should capture the essential physical features of the systems of interest. It is challenging to design an ansatz that provides both qualitative and quantitative insights into these systems. Consequently, constructing ansatzes that permit an analytic treatment while yielding accurate results over a wide range of system parameters is highly desirable. In this thesis, I presented a general approach for striking a balance between these qualitative and quantitative extremes. The approach centres around the abstract hypergraph language, which I detailed in Chapter 2. The key properties of the AHL are its modularity, size-scaling, compactness, the capture of simple and hierarchical (correlation) patterns and hierarchical composition. With the AHL, we are able to automate certain aspects of ansatz construction and work towards a systematic theory of ansatz design.

In Chapters 3-5, we utilised the AHL along with evolutionary search to construct ground-state ansatzes that are both analytically tractable and quantitatively accurate across a wide range of system parameters. This approach can be applied to any physical system that is amenable to a variational treatment in terms of tensor network states. For example, finding time evolution approximations would entail a train-by-example strategy as was used for quantum algorithm synthesis in Chapter 6. The core of our approach lies in the interplay between the abstract hypergraph language and the fitness criteria. The former enables fitness evaluation on small system sizes, which is computationally efficient, and allows the capturing of system-size scaling. The latter favours ansatzes with low variational and structural complexity while preserving accuracy. This results in expressive ansatzes which tend to break the underlying model's symmetries, but due to their simple structure, these symmetries can be restored analytically. Remarkably, by applying our method to both the LMG and TFIM models, the algorithm autonomously constructs a mean-field treatment and extends it to incorporate correlations. This provided us with a simple and interpretable structure. For the LMG model, it yields highly accurate results for finite systems, far surpassing

https://scholar.sun.ac.za

that of the mean-field treatment, and which become exact in the thermodynamic limit. For the TFIM case, we obtain accurate results across all system sizes and greatly improve upon the mean-field treatment in the thermodynamic limit.

In Section 6.1, using the AHL, we found an effective ansatz for quantum phase recognition. Notably, we improved upon the performance of a popular ansatz [5], while also reducing the variational parameter count from 1308 to 11. A promising direction for future work would entail using the native gate set of a specific quantum device to obtain a QPR circuit ansatz. This should be possible, as we showed in Section 3.4 that our search strategy is robust and independent of the underlying operator basis. Then, an appropriate many-body input state can be prepared in the lab and fed through this circuit to classify the phase. In Section 6.2, we showed how the general N-qubit realisations of three well-known quantum algorithms were automatically generated by considering circuits of up to five qubits only. These algorithms are the quantum Fourier transform (QFT), the Deutsch–Jozsa algorithm, and Grover's search algorithm. This result illustrates the importance of using a compact and size-scalable representation. Finally, Section 6.3 showed the significance of altering circuit architecture in the context of quantum machine learning and presented a pathway to enhance model performance without increasing its complexity.

https://scholar.sun.ac.za

Bibliography

[1]M. Lourens, *Hierarqcal*, https://github.com/matt-lourens/hierarqcal, 2024.

[2]M. Lourens et al., "Hierarchical quantum circuit representations for neural architecture search", npj Quantum Information **9**, 79 (2023) 10.1038/s41534-023-00747-z.

[3]M. Lourens, I. Sinayskiy, J. N. Kriel and F. Petruccione, *Generating Generalised Ground-State Ansatzes from Few-Body Examples*, version 3, (26th Sept. 2025) 10.1103/5vc5-9f4d, pre-published.

[4]A. Rouillard, M. Lourens and F. Petruccione, *Automated Quantum Algorithm Synthesis*, (12th Mar. 2025) 10.48550/arXiv.2503.08449, pre-published.

[5]I. Cong, S. Choi and M. D. Lukin, "Quantum convolutional neural networks", Nature Physics **15**, 1273–1278 (2019) 10.1038/s41567-019-0648-8.

[6]P. Weiss, "L'hypothèse du champ moléculaire et la propriété ferromagnétique", Journal de Physique Théorique et Appliquée **6**, 661–690 (1907) 10.1051/jphystap:019070060066100.

[7]H. A. Bethe and W. L. Bragg, "Statistical theory of superlattices", Proceedings of the Royal Society of London. Series A - Mathematical and Physical Sciences **150**, 552–575 (1997) 10.1098/rspa.1935.0122.

[8]L. Onsager, "Electric Moments of Molecules in Liquids", Journal of the American Chemical Society **58**, 1486–1493 (1936) 10.1021/ja01299a050.

[9]R. Kikuchi, "A Theory of Cooperative Phenomena", Physical Review **81**, 988–1003 (1951) 10.1103/PhysRev.81.988.

[10]T. Oguchi, "A Theory of Antiferromagnetism, II", Progress of Theoretical Physics **13**, 148–159 (1955) 10.1143/PTP.13.148.

[11]D. Yamamoto, "Correlated cluster mean-field theory for spin systems", Physical Review B **79**, 144427 (2009) 10.1103/PhysRevB.79.144427.

[12]G. M. Wysin and J. Kaplan, "Correlated molecular-field theory for Ising models", Physical Review E **61**, 6399–6403 (2000) 10.1103/PhysRevE.61.6399.

https://scholar.sun.ac.za

[13]G. M. Wysin, "Onsager reaction-field theory for magnetic models on diamond and hcp lattices", Physical Review B **62**, 3251–3258 (2000) 10.1103/PhysRevB.62.3251.

[14]A. Du, Y. Q. Yü and H. J. Liu, "Expanded Bethe–Peierls approximation for the Blume–Capel model", Physica A: Statistical Mechanics and its Applications **320**, 387–397 (2003) 10.1016/S0378-4371(02)01653-9.

[15]K. K. Zhuravlev, "Molecular-field theory method for evaluating critical points of the Ising model", Physical Review E **72**, 056104 (2005) 10.1103/PhysRevE.72.056104.

[16]I. Etxebarria, L. Elcoro and J. M. Perez-Mato, "Generalized boundary conditions for periodic lattice systems: Application to the two-dimensional Ising model on a square lattice", Physical Review E **70**, 066133 (2004) 10.1103/PhysRevE.70.066133.

[17]D. C. Mattis, "Molecular-field theory with correlations", Physical Review B **19**, 4737–4740 (1979) 10.1103/PhysRevB.19.4737.

[18]J. C. Slater, "The Theory of Complex Spectra", Physical Review **34**, 1293–1322 (1929) 10.1103/PhysRev.34.1293.

[19]P. A. M. Dirac, *The principles of quantum mechanics*, International Series of Monographs on Physics (The Clarendon Press, Oxford, 1930), x, 257, [1] p.

[20]D. R. Hartree, "The Wave Mechanics of an Atom with a Non-Coulomb Central Field. Part I. Theory and Methods", Mathematical Proceedings of the Cambridge Philosophical Society **24**, 89–110 (1928) 10.1017/S0305004100011919.

[21]J. C. Slater, "Note on Hartree's Method", Physical Review **35**, 210–211 (1930) 10.1103/PhysRev.35.210.2.

[22]V. Fock, "Näherungsmethode zur lösung des quantenmechanischen mehrkörperproblems", Zeitschrift für Physik **61**, 126–148 (1930) 10.1007/BF01340294.

[23]P. Echenique and J. L. Alonso, "A mathematical and computational review of Hartree–Fock SCF methods in quantum chemistry", Molecular Physics **105**, 3057–3098 (2007) 10.1080/00268970701757875.

[24]A. Szabo and N. S. Ostlund, *Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory* (Courier Corporation, 8th June 2012), 484 pp.

[25]GOOGLE AI QUANTUM AND COLLABORATORS et al., "Hartree-Fock on a superconducting qubit quantum computer", Science **369**, 1084–1089 (2020) 10.1126/science.abb9811.

[26]A. Peruzzo et al., "A variational eigenvalue solver on a photonic quantum processor", Nature Communications **5**, 4213 (2014) 10.1038/ncomms5213.

https://scholar.sun.ac.za

[27]A. Kandala et al., "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets", Nature **549**, 242–246 (2017) 10.1038/nature23879.

[28]I. G. Ryabinkin, T.-C. Yen, S. N. Genin and A. F. Izmaylov, "Qubit Coupled Cluster Method: A Systematic Approach to Quantum Chemistry on a Quantum Computer", Journal of Chemical Theory and Computation **14**, 6317–6326 (2018) 10.1021/acs.jctc.8b00932.

[29]K. Kowalski, "Properties of coupled-cluster equations originating in excitation sub-algebras", The Journal of Chemical Physics **148**, 094104 (2018) 10.1063/1.5010693.

[30]K. Kowalski, "Dimensionality reduction of the many-body problem using coupled-cluster subsystem flow equations: Classical and quantum computing perspective", Physical Review A **104**, 032804 (2021) 10.1103/PhysRevA.104.032804.

[31]N. P. Bauman et al., "Downfolding of many-body Hamiltonians using active-space models: Extension of the sub-system embedding sub-algebras approach to unitary coupled cluster formalisms", The Journal of Chemical Physics **151**, 014107 (2019) 10.1063/1.5094643.

[32]J. Lee, W. J. Huggins, M. Head-Gordon and K. B. Whaley, "Generalized Unitary Coupled Cluster Wave functions for Quantum Computation", Journal of Chemical Theory and Computation **15**, 311–324 (2019) 10.1021/acs.jctc.8b01004.

[33]Y. Matsuzawa and Y. Kurashige, "Jastrow-type Decomposition in Quantum Chemistry for Low-Depth Quantum Circuits", Journal of Chemical Theory and Computation **16**, 944–952 (2020) 10.1021/acs.jctc.9b00963.

[34]J. S. Kottmann, P. Schleich, T. Tamayo-Mendoza and A. Aspuru-Guzik, "Reducing Qubit Requirements while Maintaining Numerical Precision for the Variational Quantum Eigensolver: A Basis-Set-Free Approach", The Journal of Physical Chemistry Letters **12**, 663–673 (2021) 10.1021/acs.jpclett.0c03410.

[35]H. R. Grimsley, S. E. Economou, E. Barnes and N. J. Mayhall, "An adaptive variational algorithm for exact molecular simulations on a quantum computer", Nature Communications **10**, 3007 (2019) 10.1038/s41467-019-10988-2.

[36]I. G. Ryabinkin, R. A. Lang, S. N. Genin and A. F. Izmaylov, "Iterative Qubit Coupled Cluster Approach with Efficient Screening of Generators", Journal of Chemical Theory and Computation **16**, 1055–1063 (2020) 10.1021/acs.jctc.9b01084.

[37]R. Xia and S. Kais, "Qubit coupled cluster singles and doubles variational quantum eigensolver ansatz for electronic structure calculations", Quantum Science and Technology **6**, 015001 (2021) 10.1088/2058-9565/abbc74.

https://scholar.sun.ac.za

[38]H. L. Tang et al., "Qubit-ADAPT-VQE: An Adaptive Algorithm for Constructing Hardware-Efficient Ans\"atze on a Quantum Processor", PRX Quantum **2**, 020310 (2021) 10.1103/PRXQuantum.2.020310.

[39]N. H. Stair and F. A. Evangelista, "Simulating Many-Body Systems with a Projective Quantum Eigensolver", PRX Quantum **2**, 030301 (2021) 10.1103/PRXQuantum.2.030301.

[40]A. Anand et al., "A quantum computing view on unitary coupled cluster theory", Chemical Society Reviews **51**, 1659–1684 (2022) 10.1039/D1CS00932J.

[41]D. Wecker, M. B. Hastings and M. Troyer, "Progress towards practical quantum variational algorithms", Physical Review A **92**, 042303 (2015) 10.1103/PhysRevA.92.042303.

[42]W. W. Ho and T. H. Hsieh, "Efficient variational simulation of non-trivial quantum states", SciPost Physics **6**, 029 (2019) 10.21468/SciPostPhys.6.3.029.

[43]C. Kokail et al., "Self-Verifying Variational Quantum Simulation of the Lattice Schwinger Model", Nature **569**, 355–360 (2019) 10.1038/s41586-019-1177-4.

[44]R. Wiersema et al., "Exploring Entanglement and Optimization within the Hamiltonian Variational Ansatz", PRX Quantum **1**, 020319 (2020) 10.1103/PRXQuantum.1.020319.

[45]C. Cade, L. Mineh, A. Montanaro and S. Stanisic, "Strategies for solving the Fermi-Hubbard model on near-term quantum computers", Physical Review B **102**, 235122 (2020) 10.1103/PhysRevB.102.235122.

[46]J. S. Van Dyke et al., "Scaling adaptive quantum simulation algorithms via operator pool tiling", Physical Review Research **6**, L012030 (2024) 10.1103/PhysRevResearch.6.L012030.

[47]R. C. Farrell, M. Illa, A. N. Ciavarella and M. J. Savage, "Scalable Circuits for Preparing Ground States on Digital Quantum Computers: The Schwinger Model Vacuum on 100 Qubits", PRX Quantum **5**, 020315 (2024) 10.1103/PRXQuantum.5.020315.

[48]R. C. Farrell, M. Illa, A. N. Ciavarella and M. J. Savage, "Quantum simulations of hadron dynamics in the Schwinger model using 112 qubits", Physical Review D **109**, 114510 (2024) 10.1103/PhysRevD.109.114510.

[49]E. Gustafson et al., "Surrogate-constructed scalable-circuits adaptive variational quantum eigensolver in the Schwinger model", Physical Review Applied **23**, 064002 (2025) 10.1103/PhysRevApplied.23.064002.

[50]T. Angelides et al., "First-order phase transition of the Schwinger model with a quantum computer", npj Quantum Information **11**, 6 (2025) 10.1038/s41534-024-00950-6.

https://scholar.sun.ac.za

[51]G. Knizia and G. K.-L. Chan, "Density Matrix Embedding: A Simple Alternative to Dynamical Mean-Field Theory", Physical Review Letters **109**, 186404 (2012) 10.1103/PhysRevLett.109.186404.

[52]L. Yang, W. Hu and L. Li, *Scalable variational Monte Carlo with graph neural ansatz*, (25th Nov. 2020) 10.48550/arXiv.2011.12453, pre-published.

[53]L. Li et al., "Quantum optimization with a novel Gibbs objective function and ansatz architecture search", Physical Review Research **2**, 023074 (2020) 10.1103/PhysRevResearch.2.023074.

[54]S.-X. Zhang, C.-Y. Hsieh, S. Zhang and H. Yao, "Neural predictor based quantum architecture search", Machine Learning: Science and Technology **2**, 045027 (2021) 10.1088/2632-2153/ac28dd.

[55]Y. Du et al., "Quantum circuit architecture search for variational quantum algorithms", npj Quantum Information **8**, 62 (2022) 10.1038/s41534-022-00570-y.

[56]S.-X. Zhang, C.-Y. Hsieh, S. Zhang and H. Yao, "Differentiable quantum architecture search", Quantum Science and Technology **7**, 045023 (2022) 10.1088/2058-9565/ac87cd.

[57]A. Zhang and S. Zhao, "Evolutionary-based searching method for quantum circuit architecture", Quantum Information Processing **22**, 283 (2023) 10.1007/s11128-023-04033-x.

[58]Z. He et al., "Quantum Architecture Search with Meta-Learning", Advanced Quantum Technologies **5**, 2100134 (2022) 10.1002/qute.202100134.

[59]Z. He et al., "Training-Free Quantum Architecture Search", Proceedings of the AAAI Conference on Artificial Intelligence **38**, 12430–12438 (2024) 10.1609/aaai.v38i11.29135.

[60]Y. J. Patel et al., "Curriculum reinforcement learning for quantum architecture search under hardware errors", in The twelfth international conference on learning representations (2024).

[61]H. Situ, Z. He, S. Zheng and L. Li, "Distributed quantum architecture search", Physical Review A **110**, 022403 (2024) 10.1103/PhysRevA.110.022403.

[62]Z. He et al., "Self-supervised representation learning for Bayesian quantum architecture search", Physical Review A **111**, 032403 (2025) 10.1103/PhysRevA.111.032403.

[63]V. Lipardi, D. Dibenedetto, G. Stamoulis and M. H. Winands, "Quantum Circuit Design using a Progressive Widening Enhanced Monte Carlo Tree Search", Advanced Quantum Technologies **8**, e2500093 (2025) 10.1002/qute.202500093.

https://scholar.sun.ac.za

[64]S. Gulwani, "Dimensions in program synthesis", in Proceedings of the 12th international ACM SIGPLAN symposium on Principles and practice of declarative programming, PPDP '10 (26th July 2010), pp. 13–24, 10.1145/1836089.1836091.

[65]A. Desai et al., "Program synthesis using natural language", in Proceedings of the 38th International Conference on Software Engineering (14th May 2016), pp. 345–356, 10.1145/2884781.2884786.

[66]S. Anupam, M. Bowers, O. C. Reyes and A. Solar-Lezama, "MathDSL: A Domain-Specific Language for Concise Mathematical Solutions Via Program Synthesis", in (10th Oct. 2024).

[67]J. S. Kottmann et al., "TEQUILA: a platform for rapid development of quantum algorithms", Quantum Science and Technology **6**, 024009 (2021) 10.1088/2058-9565/abe567.

[68]I. Cirac, D. Perez-Garcia, N. Schuch and F. Verstraete, "Matrix Product States and Projected Entangled Pair States: Concepts, Symmetries, and Theorems", Reviews of Modern Physics **93**, 045003 (2021) 10.1103/RevModPhys.93.045003.

[69]M. Fannes, B. Nachtergaele and R. F. Werner, "Finitely correlated states on quantum spin chains", Communications in Mathematical Physics **144**, 443–490 (1992) 10.1007/BF02099178.

[70]D. Perez-Garcia, F. Verstraete, M. Wolf and J. Cirac, "Matrix product state representations", Quantum Information and Computation **7**, 401–430 (2007) 10.26421/QIC7.5-6-1.

[71]U. Schollwöck, "The density-matrix renormalization group in the age of matrix product states", Annals of Physics, January 2011 Special Issue **326**, 96–192 (2011) 10.1016/j.aop.2010.09.012.

[72]M. B. Hastings, "An area law for one-dimensional quantum systems", Journal of Statistical Mechanics: Theory and Experiment **2007**, P08024 (2007) 10.1088/1742-5468/2007/08/P08024.

[73]F. Verstraete and J. I. Cirac, "Matrix product states represent ground states faithfully", Physical Review B **73**, 094423 (2006) 10.1103/PhysRevB.73.094423.

[74]R. Orús, "A practical introduction to tensor networks: Matrix product states and projected entangled pair states", Annals of Physics **349**, 117–158 (2014) 10.1016/j.aop.2014.06.013.

[75]S. R. White, "Density matrix formulation for quantum renormalization groups", Physical Review Letters **69**, 2863–2866 (1992) 10.1103/PhysRevLett.69.2863.

https://scholar.sun.ac.za

[76]S. R. White, "Density-matrix algorithms for quantum renormalization groups", Physical Review B **48**, 10345–10356 (1993) 10.1103/PhysRevB.48.10345.

[77]S. Östlund and S. Rommer, "Thermodynamic Limit of Density Matrix Renormalization", Physical Review Letters **75**, 3537–3540 (1995) 10.1103/PhysRevLett.75.3537.

[78]J. Dukelsky, M. A. Martín-Delgado, T. Nishino and G. Sierra, "Equivalence of the variational matrix product method and the density matrix renormalization group applied to spin chains", Europhysics Letters **43**, 457 (1998) 10.1209/epl/i1998-00381-x.

[79]J. Eisert, M. Cramer and M. B. Plenio, "Colloquium: Area laws for the entanglement entropy", Reviews of Modern Physics **82**, 277–306 (2010) 10.1103/RevModPhys.82.277.

[80]F. Verstraete, V. Murg and J. Cirac, "Matrix product states, projected entangled pair states, and variational renormalization group methods for quantum spin systems", Advances in Physics **57**, 143–224 (2008) 10.1080/14789940801912366.

[81]F. Verstraete and J. I. Cirac, *Renormalization algorithms for Quantum-Many Body Systems in two and higher dimensions*, (2nd July 2004) 10.48550/arXiv.cond-mat/0407066, pre-published.

[82]V. Murg, F. Verstraete and J. I. Cirac, "Variational study of hard-core bosons in a two-dimensional optical lattice using projected entangled pair states", Physical Review A **75**, 033605 (2007) 10.1103/PhysRevA.75.033605.

[83]V. Murg, F. Verstraete and J. I. Cirac, "Exploring frustrated spin systems using projected entangled pair states", Physical Review B **79**, 195119 (2009) 10.1103/PhysRevB.79.195119.

[84]G. Vidal, "Class of Quantum Many-Body States That Can Be Efficiently Simulated", Physical Review Letters **101**, 110501 (2008) 10.1103/PhysRevLett.101.110501.

[85]G. Evenbly and G. Vidal, "Tensor Network States and Geometry", Journal of Statistical Physics **145**, 891–918 (2011) 10.1007/s10955-011-0237-4.

[86]G. Evenbly and G. Vidal, *Quantum Criticality with the Multi-scale Entanglement Renormalization Ansatz*, (31st Oct. 2013) 10.48550/arXiv.1109.5334, pre-published.

[87]G. Evenbly and G. Vidal, "Class of Highly Entangled Many-Body States that can be Efficiently Simulated", Physical Review Letters **112**, 240502 (2014) 10.1103/PhysRevLett.112.240502.

[88]G. Vidal, "Entanglement Renormalization", Physical Review Letters **99**, 220405 (2007) 10.1103/PhysRevLett.99.220405.

https://scholar.sun.ac.za

[89]Y.-Y. Shi, L.-M. Duan and G. Vidal, "Classical simulation of quantum many-body systems with a tree tensor network", Physical Review A **74**, 022320 (2006) 10.1103/PhysRevA.74.022320.

[90]V. Murg, F. Verstraete, Ö. Legeza and R. M. Noack, "Simulating strongly correlated quantum systems with tree tensor networks", Physical Review B **82**, 205105 (2010) 10.1103/PhysRevB.82.205105.

[91]M. Gerster et al., "Unconstrained tree tensor network: An adaptive gauge picture for enhanced performance", Physical Review B **90**, 125154 (2014) 10.1103/PhysRevB.90.125154.

[92]M. Gerster et al., "Fractional quantum Hall effect in the interacting Hofstadter model via tensor networks", Physical Review B **96**, 195123 (2017) 10.1103/PhysRevB.96.195123.

[93]L. Kohn et al., "Superfluid-to-Mott transition in a Bose-Hubbard ring: Persistent currents and defect formation", Physical Review A **101**, 023617 (2020) 10.1103/PhysRevA.101.023617.

[94]T. Felser, P. Silvi, M. Collura and S. Montangero, "Two-Dimensional Quantum-Link Lattice Quantum Electrodynamics at Finite Density", Physical Review X **10**, 041040 (2020) 10.1103/PhysRevX.10.041040.

[95]G. Magnifico, T. Felser, P. Silvi and S. Montangero, "Lattice quantum electrodynamics in (3+1)-dimensions at finite density with tensor networks", Nature Communications **12**, 3600 (2021) 10.1038/s41467-021-23646-3.

[96]T. Felser, S. Notarnicola and S. Montangero, "Efficient Tensor Network Ansatz for High-Dimensional Quantum Many-Body Problems", Physical Review Letters **126**, 170603 (2021) 10.1103/PhysRevLett.126.170603.

[97]H.-G. Luo, M.-P. Qin and T. Xiang, "Optimizing Hartree-Fock orbitals by the density-matrix renormalization group", Physical Review B **81**, 235129 (2010) 10.1103/PhysRevB.81.235129.

[98]W. M. C. Foulkes, L. Mitas, R. J. Needs and G. Rajagopal, "Quantum Monte Carlo simulations of solids", Reviews of Modern Physics **73**, 33–83 (2001) 10.1103/RevModPhys.73.33.

[99]A. W. Sandvik and G. Vidal, "Variational Quantum Monte Carlo Simulations with Tensor-Network States", Physical Review Letters **99**, 220602 (2007) 10.1103/PhysRevLett.99.220602.

[100]F. Mezzacapo, N. Schuch, M. Boninsegni and J. I. Cirac, "Ground-state properties of quantum many-body systems: entangled-plaquette states and variational Monte Carlo", New Journal of Physics **11**, 083026 (2009) 10.1088/1367-2630/11/8/083026.

https://scholar.sun.ac.za

[101]H. J. Changlani, J. M. Kinder, C. J. Umrigar and G. K.-L. Chan, "Approximating strongly correlated wave functions with correlator product states", Physical Review B **80**, 245116 (2009) 10.1103/PhysRevB.80.245116.

[102]N. Schuch, M. M. Wolf, F. Verstraete and J. I. Cirac, "Simulation of Quantum Many-Body Systems with Strings of Operators and Monte Carlo Tensor Contractions", Physical Review Letters **100**, 040501 (2008) 10.1103/PhysRevLett.100.040501.

[103]A. Sfondrini, J. Cerrillo, N. Schuch and J. I. Cirac, "Simulating two- and three-dimensional frustrated quantum systems with string-bond states", Physical Review B **81**, 214426 (2010) 10.1103/PhysRevB.81.214426.

[104]K. H. Marti et al., "Complete-graph tensor network states: a new fermionic wave function ansatz for molecules", New Journal of Physics **12**, 103008 (2010) 10.1088/1367-2630/12/10/103008.

[105]G. Carleo and M. Troyer, "Solving the quantum many-body problem with artificial neural networks", Science **355**, 602–606 (2017) 10.1126/science.aag2302.

[106]K. Choo, G. Carleo, N. Regnault and T. Neupert, "Symmetries and Many-Body Excitations with Neural-Network Quantum States", Physical Review Letters **121**, 167204 (2018) 10.1103/PhysRevLett.121.167204.

[107]H. Saito, "Method to Solve Quantum Few-Body Problems with Artificial Neural Networks", Journal of the Physical Society of Japan **87**, 074002 (2018) 10.7566/JPSJ.87.074002.

[108]O. Sharir et al., "Deep Autoregressive Models for the Efficient Variational Simulation of Many-Body Quantum Systems", Physical Review Letters **124**, 020503 (2020) 10.1103/PhysRevLett.124.020503.

[109]Y. Levine, O. Sharir, N. Cohen and A. Shashua, "Quantum Entanglement in Deep Learning Architectures", Physical Review Letters **122**, 065301 (2019) 10.1103/PhysRevLett.122.065301.

[110]Y. Teng, S. Sachdev and M. S. Scheurer, "Clustering neural quantum states via diffusion maps", Physical Review B **108**, 205152 (2023) 10.1103/PhysRevB.108.205152.

[111]J. Carrasquilla and R. G. Melko, "Machine learning phases of matter", Nature Physics **13**, 431–434 (2017) 10.1038/nphys4035.

[112]S. R. Clark, "Unifying neural-network quantum states and correlator product states via tensor networks", Journal of Physics A: Mathematical and Theoretical **51**, 135301 (2018) 10.1088/1751-8121/aaaaf2.

https://scholar.sun.ac.za

[113]I. Glasser et al., "Neural-Network Quantum States, String-Bond States, and Chiral Topological States", Physical Review X **8**, 011006 (2018) 10.1103/PhysRevX.8.011006.

[114]Y. LeCun, Y. Bengio and G. Hinton, "Deep learning", Nature **521**, 436–444 (2015) 10.1038/nature14539.

[115]A. Krizhevsky, I. Sutskever and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks", in Advances in Neural Information Processing Systems, Vol. 25 (2012).

[116]B. Zoph and Q. V. Le, *Neural Architecture Search with Reinforcement Learning*, (15th Feb. 2017) http://arxiv.org/abs/1611.01578 (visited on 27/03/2023), pre-published.

[117]T. Elsken, J. H. Metzen and F. Hutter, *Neural Architecture Search: A Survey*, (26th Apr. 2019) 10.48550/arXiv.1808.05377, pre-published.

[118]E. Real, A. Aggarwal, Y. Huang and Q. V. Le, "Regularized Evolution for Image Classifier Architecture Search", Proceedings of the AAAI Conference on Artificial Intelligence **33**, 4780–4789 (2019) 10.1609/aaai.v33i01.33014780.

[119]B. Zoph, V. Vasudevan, J. Shlens and Q. V. Le, "Learning Transferable Architectures for Scalable Image Recognition", in (11th Apr. 2018), 10.48550/arXiv.1707.07012.

[120]L.-C. Chen et al., "Searching for Efficient Multi-Scale Architectures for Dense Image Prediction", in Advances in Neural Information Processing Systems, Vol. 31 (2018).

[121]H. Liu et al., "Hierarchical Representations for Efficient Architecture Search", in (23rd Feb. 2018).

[122]J. Gray, "Quimb: A python package for quantum information and many-body calculations", Journal of Open Source Software **3**, 819 (2018) 10.21105/joss.00819.

[123]E. Grant et al., "Hierarchical quantum classifiers", npj Quantum Information **4**, 65 (2018) 10.1038/s41534-018-0116-9.

[124]E. Real et al., "Large-scale evolution of image classifiers", in Proceedings of the 34th international conference on machine learning, Vol. 70, edited by D. Precup and Y. W. Teh, Proceedings of Machine Learning Research (6th–11th Aug. 2017), pp. 2902–2911.

[125]D. E. Goldberg and K. Deb, "A Comparative Analysis of Selection Schemes Used in Genetic Algorithms", in *Foundations of Genetic Algorithms*, Vol. 1, edited by G. J. E. Rawlins (Elsevier, 1st Jan. 1991), pp. 69–93, 10.1016/B978-0-08-050684-5.50008-2.

https://scholar.sun.ac.za

[126]H. J. Lipkin, N. Meshkov and A. J. Glick, "Validity of many-body approximation methods for a solvable model: (I). Exact solutions and perturbation theory", Nuclear Physics **62**, 188–198 (1965) 10.1016/0029-5582(65)90862-X.

[127]N. Meshkov, A. J. Glick and H. J. Lipkin, "Validity of many-body approximation methods for a solvable model: (II). Linearization procedures", Nuclear Physics **62**, 199–210 (1965) 10.1016/0029-5582(65)90863-1.

[128]A. J. Glick, H. J. Lipkin and N. Meshkov, "Validity of many-body approximation methods for a solvable model: (III). Diagram summations", Nuclear Physics **62**, 211–224 (1965) 10.1016/0029-5582(65)90864-3.

[129]S. Sachdev, *Quantum phase transitions*, Vol. 12 (Apr. 1999).

[130]C.-Y. Park, "Efficient ground state preparation in variational quantum eigensolver with symmetry-breaking layers", APL Quantum **1**, 016101 (2024) 10.1063/5.0186205.

[131]J. J. Meyer et al., "Exploiting Symmetry in Variational Quantum Machine Learning", PRX Quantum **4**, 010328 (2023) 10.1103/PRXQuantum.4.010328.

[132]S. Dusuel and J. Vidal, "Finite-Size Scaling Exponents of the Lipkin-Meshkov-Glick Model", Physical Review Letters **93**, 237204 (2004) 10.1103/PhysRevLett.93.237204.

[133]T. Barthel, S. Dusuel and J. Vidal, "Entanglement Entropy beyond the Free Case", Physical Review Letters **97**, 220402 (2006) 10.1103/PhysRevLett.97.220402.

[134]R. Orús, S. Dusuel and J. Vidal, "Equivalence of Critical Scaling Laws for Many-Body Entanglement in the Lipkin-Meshkov-Glick Model", Physical Review Letters **101**, 025701 (2008) 10.1103/PhysRevLett.101.025701.

[135]O. E. I. S., *Riordan triangle ((1-x)/(1-2x), x^2/(1-2x)), Entry A201701 in The On-Line Encyclopedia of Integer Sequences*, (30th Nov. 2024) https://oeis.org/A201701/internal.

[136]J. Herrmann et al., "Realizing quantum convolutional neural networks on a superconducting quantum processor to recognize quantum phases", Nature Communications **13**, 4144 (2022) 10.1038/s41467-022-31679-5.

[137]P. Smacchia et al., "Statistical mechanics of the cluster Ising model", Physical Review A **84**, 022304 (2011) 10.1103/PhysRevA.84.022304.

[138]F. D. M. Haldane, "Nonlinear Field Theory of Large-Spin Heisenberg Antiferromagnets: Semiclassically Quantized Solitons of the One-Dimensional Easy-Axis N\'eel State", Physical Review Letters **50**, 1153–1156 (1983) 10.1103/PhysRevLett.50.1153.

https://scholar.sun.ac.za

[139]M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition* (Cambridge University Press, 9th Dec. 2010), 10.1017/CBO9780511976667.

[140]M. Benedetti et al., "A generative modeling approach for benchmarking and training shallow quantum circuits", npj Quantum Information **5**, 45 (2019) 10.1038/s41534-019-0157-8.

[141]M. Cerezo et al., "Variational quantum algorithms", Nature Reviews Physics **3**, 625–644 (2021) 10.1038/s42254-021-00348-9.

[142]S. Mangini et al., "Quantum computing models for artificial neural networks", Europhysics Letters **134**, 10002 (2021) 10.1209/0295-5075/134/10002.

[143]K. Bharti et al., "Noisy intermediate-scale quantum algorithms", Reviews of Modern Physics **94**, 015004 (2022) 10.1103/RevModPhys.94.015004.

[144]M. Schuld, R. Sweke and J. J. Meyer, "Effect of data encoding on the expressive power of variational quantum-machine-learning models", Physical Review A **103**, 032430 (2021) 10.1103/PhysRevA.103.032430.

[145]Z. Holmes, K. Sharma, M. Cerezo and P. J. Coles, "Connecting Ansatz Expressibility to Gradient Magnitudes and Barren Plateaus", PRX Quantum **3**, 010313 (2022) 10.1103/PRXQuantum.3.010313.

[146]T. Haug, K. Bharti and M. Kim, "Capacity and Quantum Geometry of Parametrized Quantum Circuits", PRX Quantum **2**, 040309 (2021) 10.1103/PRXQuantum.2.040309.

[147]Y. S. Yordanov, V. Armaos, C. H. W. Barnes and D. R. M. Arvidsson-Shukur, "Qubit-excitation-based adaptive variational quantum eigensolver", Communications Physics **4**, 228 (2021) 10.1038/s42005-021-00730-0.

[148]A. G. Rattew et al., *A Domain-agnostic, Noise-resistant, Hardware-efficient Evolutionary Variational Quantum Eigensolver*, (23rd Jan. 2020) 10.48550/arXiv.1910.09694, pre-published.

[149]L. Zhu et al., "Adaptive quantum approximate optimization algorithm for solving combinatorial problems on a quantum computer", Physical Review Research **4**, 033029 (2022) 10.1103/PhysRevResearch.4.033029.

[150]M. Ostaszewski, E. Grant and M. Benedetti, "Structure optimization for parameterized quantum circuits", Quantum **5**, 391 (2021) 10.22331/q-2021-01-28-391.

[151]Z. Lu, P.-X. Shen and D.-L. Deng, "Markovian Quantum Neuroevolution for Machine Learning", Physical Review Applied **16**, 044039 (2021) 10.1103/PhysRevApplied.16.044039.

https://scholar.sun.ac.za

[152]D. Slabbert, M. Lourens and F. Petruccione, "Pulsar classification: comparing quantum convolutional neural networks and quantum support vector machines", Quantum Machine Intelligence **6**, 56 (2024) 10.1007/s42484-024-00194-9.

[153]T. Hur, L. Kim and D. K. Park, "Quantum convolutional neural network for classical data classification", Quantum Machine Intelligence **4**, 3 (2022) 10.1007/s42484-021-00061-x.

[154]S. Sim, P. D. Johnson and A. Aspuru-Guzik, "Expressibility and Entangling Capability of Parameterized Quantum Circuits for Hybrid Quantum-Classical Algorithms", Advanced Quantum Technologies **2**, 1900070 (2019) 10.1002/qute.201900070.

[155]R. M. Parrish, E. G. Hohenstein, P. L. McMahon and T. J. Martínez, "Quantum Computation of Electronic Transitions Using a Variational Quantum Eigensolver", Physical Review Letters **122**, 230401 (2019) 10.1103/PhysRevLett.122.230401.

[156]R. J. Lyon et al., "Fifty years of pulsar candidate selection: from simple filters to a new principled real-time classification approach", Monthly Notices of the Royal Astronomical Society **459**, 1104–1123 (2016) 10.1093/mnras/stw656.

[157]M. Schuld, *Supervised quantum machine learning models are kernel methods*, (17th Apr. 2021) 10.48550/arXiv.2101.11020, pre-published.

[158]Y. Liu, S. Arunachalam and K. Temme, "A rigorous and robust quantum speed-up in supervised machine learning", Nature Physics **17**, 1013–1017 (2021) 10.1038/s41567-021-01287-z.

https://scholar.sun.ac.za

Appendix A

Wildcard populate algorithm

Here I provide the explicit Python implementation of the wildcard populate procedure in Code. A.1 which is described in Section 2.4.3. The routine takes a base pattern containing the wild card characters `*` and `!` and incrementally replaces them with `0`s and `1`s. The final output is a bitstring that specifies a partition.

https://scholar.sun.ac.za


```
def wildcard_populate(self, pattern, length):
# Wildcard pattern
n_stars = pattern.count("*")
n_excl = pattern.count("!")
base = pattern
max_it = length
do_star = True if n_stars > 0 else False
just_changed = False
stars_found = 0
excls_found = 0
while len(base) < (length + n_stars + n_excl) and max_it > 0:
    if do_star:
        if stars_found % 2 == 0:
            idx = base.find("*")
        else:
            idx = base.rfind("*")
        stars_found += 1
        # Insert 0 next to it
        base = base[:idx] + "0" + base[idx:]
        do_star = False if n_excl > 0 else True
        just_changed = True
    if (not do_star) and (not just_changed):
        if excls_found % 2 == 0:
            idx = base.find("!")
        else:
            idx = base.rfind("!")
        excls_found += 1
        # Insert 1 next to it
        base = base[:idx] + "1" + base[idx:]
        do_star = True if n_stars > 0 else False
    just_changed = False
    max_it -= 1
base = base.replace("*", "")
base = base.replace("!", "")
return base
```

Code A.1: The wildcard populate algorithm python code